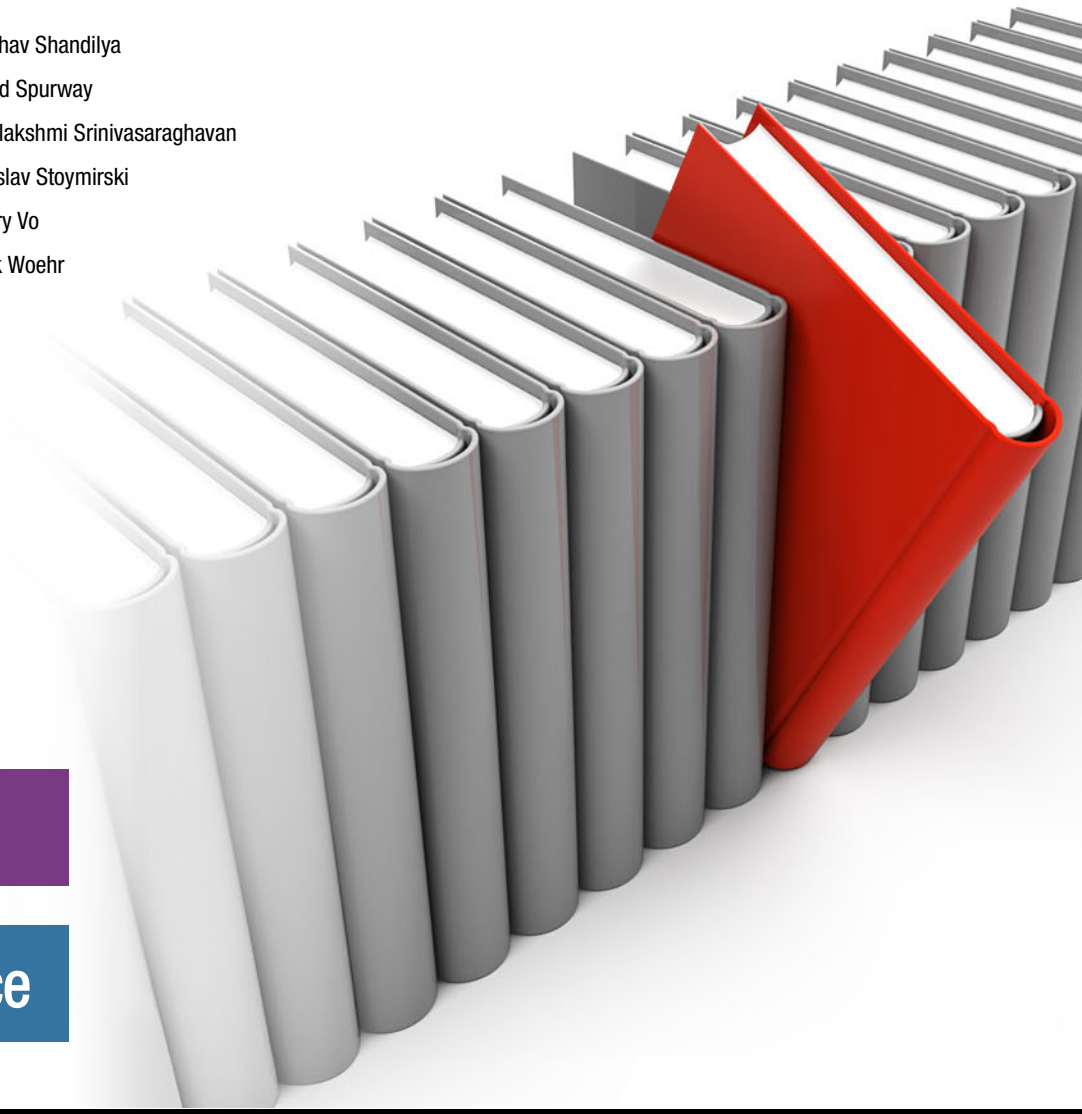
Redbooks

ibm.com/redbooks

Implementing AI on Power11

Introducing the IBM Spyre Accelerator

Tim Simon
Harshitha C
Anjil Reddy Chinnapatlolla
Ricardo Contreras
Florian Dahlitz
Abdul Haleem
Blake Hoskinson
Bijay Dev K M
Henrik Mader
Manish Mukul
Arnold Ness
Nnamdi Okore-Affia II
Praveen Kumar Pandey
Manvanthara Puttashankar
Daniel Schenker
Vaibhav Shandilya
David Spurway
Rajalakshmi Srinivasaraghavan
Borislav Stoymirski
Henry Vo
Jack Woehr



IBM Power

Artificial Intelligence

IBM

Redbooks



IBM Redbooks

Implementing AI on Power11: Introducing the IBM Spyre Accelerator

March 2026

SG24-8592-00

Note: Before using this information and the product it supports, read the information in “Notices” on page vii.

First Edition (March 2026)

This edition applies to the following products or services:
- IBM Spyre
- IBM Power11 servers
- Red Hat Enterprise Linux (RHEL) 9.6 or later
- Red Hat AI Inference Server 3.1

© Copyright International Business Machines Corporation 2026. All rights reserved.
Note to U.S. Government Users Restricted Rights -- Use, duplication or disclosure restricted by GSA ADP Schedule Contract with IBM Corp.

Contents

Notices vii
Trademarks viii

Preface ix
Authors ix
Now you can become a published author, too! xiii
Comments welcome xiii
Stay connected to IBM Redbooks xiii

Chapter 1. Introducing artificial intelligence on IBM Power11 1
1.1 Artificial intelligence and IBM Power11 2
1.2 Defining AI and understanding its limitations 2
1.2.1 Understanding AI techniques 3
1.2.2 Misconceptions about AI 4
1.3 Overview of AI trends in enterprise computing 4
1.4 Power11 as a strategic platform for AI 5
1.4.1 Power11 as a trusted enterprise server 6
1.4.2 Hardware-assisted AI features in Power11 6
1.4.3 IBM Power: Transforming business through seamless AI integration 8
1.4.4 AI services: Turnkey AI 10
1.5 Comparison with previous generations 10
1.5.1 Power9 11
1.5.2 Power10 12
1.6 Summary 13

Chapter 2. Use cases and industry applications 15
2.1 AI deployment patterns and use cases 16
2.1.1 Enterprise AI-ready deployment pattern on Power servers 16
2.1.2 Traditional AI and ML on Power servers: An on-premises AI and ML deployment pattern 19
2.1.3 Hybrid agentic AI with IBM watsonx Orchestrate and on-premises Power 22
2.1.4 Pre-built AI use cases with IBM Spyre for Power 24
2.2 AI for industries 26
2.2.1 Banking and finance 26
2.2.2 Government 27
2.2.3 Manufacturing 29
2.2.4 Healthcare 30
2.2.5 Airlines 31
2.2.6 Retail 32
2.2.7 Agriculture 33
2.3 Cross-industry AI 34
2.3.1 AI in IT operations 34
2.3.2 AI DevOps 38
2.3.3 Real-time inference 39

Chapter 3. On-chip acceleration 45
3.1 Overview of AI acceleration features 46
3.2 AI acceleration on IBM Power11: MMA and SIMD optimization 46
3.3 Training of machine learning models 47

© Copyright IBM Corp. 2026.

3.3.1 The training process 47
3.3.2 Computational complexity and hardware requirements 48
3.3.3 Hardware and energy considerations 49
3.3.4 Summary 49

Chapter 4. IBM Spyre Accelerator 51
4.1 Introducing the IBM Spyre Accelerator 52
4.1.1 Role in AI workload orchestration and optimization 52
4.2 Turnkey AI with Spyre 53
4.2.1 Architecture and design principles 53
4.2.2 Integration with IBM Spyre Accelerator for Power 54
4.2.3 Supported AI use cases 54
4.2.4 Deployment and operations 55
4.3 IBM Spyre Accelerator design 55
4.4 Power11 implementation of IBM Spyre Accelerator 56
4.4.1 Physical implementation 58
4.4.2 Expansion drawer support 60
4.5 Examples and supported workloads 66
4.6 Understanding LLM performance 68
4.7 Hybrid compute: Spyre and CPU 69
4.8 The IBM AI acceleration roadmap 70

Chapter 5. Artificial intelligence workload optimization on IBM Power11 71
5.1 Memory bandwidth and cache usage 72
5.2 NUMA-aware scheduling and resource allocation 72
5.3 Enhancements in Python performance on Power11 74
5.3.1 Power11 hardware enhancements 74
5.3.2 Compiler toolchain and library-level enhancements 74
5.3.3 Python Interpreter and runtime enhancements 75
5.3.4 Workload validation 75
5.4 Support for AI and ML frameworks 76
5.4.1 Overview of frameworks 77
5.5 Compatibility with OpenBLAS, NumPy, and other scientific libraries 78
5.5.1 Inspecting the OpenBLAS configuration 79
5.5.2 Inspecting the NumPy configuration 80
5.6 Developer productivity and tool improvements 84

Chapter 6. Artificial intelligence and security 85
6.1 Secure model deployment 86
6.2 On-premises compared to cloud deployment 87
6.3 Data privacy and encryption in AI pipelines 88
6.3.1 Data privacy threats in ML systems 89
6.3.2 Key measures for protecting ML systems 90
6.4 Security and AI on IBM Power 91
6.4.1 Why security is important 91
6.4.2 The IBM framework for securing generative AI 92
6.4.3 Resisting cyberattacks and recovering quickly in IBM Power 94
6.5 Securing AI on IBM Power 94
6.5.1 PowerSC: Security for hybrid cloud and AI 94
6.5.2 Expanding security with AI on IBM Power: Partner ecosystem 95
6.6 Controlling data and models in your AI ecosystem 95
6.6.1 Controlling data with AI on IBM Power 96
6.6.2 Controlling the model with AI on IBM Power 97

Chapter 7. Tools, frameworks, and ecosystem 101
7.1 Power Inference Microservice 102
7.1.1 PIM architecture 102
7.1.2 Creating a bootable container image to run on IBM Power 104
7.1.3 Using the Python ecosystem on IBM Power 105
7.2 IBM watsonx and AI APIs 106
7.3 Open-source tools and libraries 107
7.3.1 IBM i 108
7.4 Access to packages and high-level installation 110
7.4.1 Installing container tools and utilities on RHEL 9 110
7.4.2 Preparing the host environment 111
7.4.3 Red Hat AI Inference Server and API shapes 113
7.4.4 Obtaining and managing models 113
7.4.5 Environment variables and tuning 114
7.4.6 Running Red Hat AI Inference Server with Podman (examples) 115
7.4.7 Validating the server 117
7.4.8 Troubleshooting and best practices 118
7.5 Other tools 118
7.6 Using the Wallaroo AI Starter Kit 119

Abbreviations and acronyms 121

Related publications 123
IBM Redbooks 123
Online resources 123
Help from IBM 124

Notices

This information was developed for products and services offered in the US. This material might be available from IBM in other languages. However, you may be required to own a copy of the product or product version in that language in order to access it.

IBM may not offer the products, services, or features discussed in this document in other countries. Consult your local IBM representative for information on the products and services currently available in your area. Any reference to an IBM product, program, or service is not intended to state or imply that only that IBM product, program, or service may be used. Any functionally equivalent product, program, or service that does not infringe any IBM intellectual property right may be used instead. However, it is the user's responsibility to evaluate and verify the operation of any non-IBM product, program, or service.

IBM may have patents or pending patent applications covering subject matter described in this document. The furnishing of this document does not grant you any license to these patents. You can send license inquiries, in writing, to:
IBM Director of Licensing, IBM Corporation, North Castle Drive, MD-NC119, Armonk, NY 10504-1785, US

INTERNATIONAL BUSINESS MACHINES CORPORATION PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some jurisdictions do not allow disclaimer of express or implied warranties in certain transactions, therefore, this statement may not apply to you.

This information could include technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in new editions of the publication. IBM may make improvements and/or changes in the product(s) and/or the program(s) described in this publication at any time without notice.

Any references in this information to non-IBM websites are provided for convenience only and do not in any manner serve as an endorsement of those websites. The materials at those websites are not part of the materials for this IBM product and use of those websites is at your own risk.

IBM may use or distribute any of the information you provide in any way it believes appropriate without incurring any obligation to you.

The performance data and client examples cited are presented for illustrative purposes only. Actual performance results may vary depending on specific configurations and operating conditions.

Information concerning non-IBM products was obtained from the suppliers of those products, their published announcements or other publicly available sources. IBM has not tested those products and cannot confirm the accuracy of performance, compatibility or any other claims related to non-IBM products. Questions on the capabilities of non-IBM products should be addressed to the suppliers of those products.

Statements regarding IBM's future direction or intent are subject to change or withdrawal without notice, and represent goals and objectives only.

This information contains examples of data and reports used in daily business operations. To illustrate them as completely as possible, the examples include the names of individuals, companies, brands, and products. All of these names are fictitious and any similarity to actual people or business enterprises is entirely coincidental.

COPYRIGHT LICENSE:

This information contains sample application programs in source language, which illustrate programming techniques on various operating platforms. You may copy, modify, and distribute these sample programs in any form without payment to IBM, for the purposes of developing, using, marketing or distributing application programs conforming to the application programming interface for the operating platform for which the sample programs are written. These examples have not been thoroughly tested under all conditions. IBM, therefore, cannot guarantee or imply reliability, serviceability, or function of these programs. The sample programs are provided "AS IS", without warranty of any kind. IBM shall not be liable for any damages arising out of your use of the sample programs.

© Copyright IBM Corp. 2026.

Trademarks

IBM, the IBM logo, and ibm.com are trademarks or registered trademarks of International Business Machines Corporation, registered in many jurisdictions worldwide. Other product and service names might be trademarks of IBM or other companies. A current list of IBM trademarks is available on the web at “Copyright and trademark information” at https://www.ibm.com/legal/copytrade.shtml

The following terms are trademarks or registered trademarks of International Business Machines Corporation, and might also be trademarks or registered trademarks in other countries.

AIX®
Db2®
Granite®
Guardium®
IBM®
IBM Cloud®
IBM Cloud Pak®
IBM Cloud Satellite®
IBM Research®
IBM Security®
IBM Spyre™
IBM watsonx®
IBM Z®
Orchestrate®
Power Architecture®
Power9®
PowerVM®
QRadar®
Redbooks®
Redbooks (logo)  ®
Satellite®
Spyre™
watsonx®
watsonx Code Assistant®
watsonx Orchestrate®
watsonx.ai®
z/OS®
z/VM®

The following terms are trademarks of other companies:

Evolution, are trademarks or registered trademarks of Kenexa, an IBM Company.

Intel, Intel Xeon, Intel logo, Intel Inside logo, and Intel Centrino logo are trademarks or registered trademarks of Intel Corporation or its subsidiaries in the United States and other countries.

The registered trademark Linux® is used pursuant to a sublicense from the Linux Foundation, the exclusive licensee of Linus Torvalds, owner of the mark on a worldwide basis.

Microsoft, and the Windows logo are trademarks of Microsoft Corporation in the United States, other countries, or both.

Java, and all Java-based trademarks and logos are trademarks or registered trademarks of Oracle and/or its affiliates.

Red Hat, Ansible, Fedora, OpenShift, are trademarks or registered trademarks of Red Hat, Inc. or its subsidiaries in the United States and other countries.

VMware, and the VMware logo are registered trademarks or trademarks of VMware, Inc. or its subsidiaries in the United States and/or other jurisdictions.

Other company, product, or service names may be trademarks or service marks of others.

Preface

This IBM Redpaper publication presents the IBM® next-generation platform for enterprise artificial intelligence (AI) and shows how IBM Power11 strengthens on-chip acceleration, system design, and hybrid AI deployment. It outlines key AI concepts and industry trends, which emphasize the need for secure, high-performance inference close to mission-critical data.

A major highlight is IBM Spyre™ PCIe Gen5 x16 AI Accelerator Adapter (IBM Spyre Accelerator), which is a new reduced-power Peripheral Component Interconnect Express (PCIe) accelerator that is designed for scalable AI inference. With a highly parallel architecture and support for modern AI numerics, Spyre enables efficient running of large language models (LLMs), embedding, retrieval-augmented generation (RAG), and other enterprise AI workloads. Integrated with Red Hat AI and vLLM, it delivers on-premises AI with predictable performance and data-sovereign control.

The publication also addresses end-to-end lifecycle requirements, such as security, governance, machine learning operations (MLOps), and industry use cases, and shows how Power11 and Spyre support regulated, data-intensive environments. Combined with IBM's broader software and open-source ecosystem, the platform provides a complete blueprint for deploying enterprise-grade AI on IBM Power.

Authors

This book was produced by a team working at IBM Redbooks, Remote Center.

Tim Simon is an IBM Redbooks® Project Leader who is based in Tulsa, Oklahoma, US. He has more than 40 years of experience at IBM, primarily in technical sales, and has built a distinguished career helping clients design and implement IBM solutions that align with their business goals. He holds a Bachelor of Science degree in Mathematics from Towson University in Maryland. Tim’s expertise spans a broad range of IBM technologies, with deep experience in developing customer solutions that use IBM Power, IBM Storage, and IBM Z® platforms.

Harshitha C is a Performance Analyst at IBM India System Development Labs who specializes in end-to-end performance characterization and optimization of enterprise workloads on IBM Power Architecture®. Her work focuses on full-stack performance engineering for large-scale banking applications and includes analysis across the microarchitecture, runtime, middleware, and application layers to identify efficiency gaps and drive throughput, latency, and scalability improvements on the Power platform. Harshitha holds an M.S. by Research in Computer Science from IIT Palakkad, where her research focused on post-silicon validation workflows, including functional verification, performance correlation with pre-silicon models, and architecture-level anomaly detection by using trace analysis. She also holds a B.Tech. in Computer Science from the Cochin University of Science and Technology. Her professional interests span high-performance computing (HPC), microarchitecture-aware software optimization, workload characterization, and systems performance tuning.

© Copyright IBM Corp. 2026.

Anjil Reddy Chinnapatlolla is a Technical Lead for Power servers who is based in IBM India and has over 19 years of experience in systems software and performance engineering. He leads the AI Libraries development team at IBM, focused on accelerating AI workload performance on Power platforms. His work focuses on full-stack optimization from low-level kernel optimization, quantization, and hardware feature enablement through system software to AI frameworks to unlock CPU- and accelerator-based performance. He collaborates closely with architecture, research, and product teams to bring hardware innovations into production environments, improving performance, efficiency, and scalability for enterprise AI workloads.

Ricardo Contreras is an IBM Power Brand Technical Specialist who is based in Chile. With 6 years at IBM and more than 15 years in the IT industry, he designs and modernizes Power platforms for mission-critical workloads for IBM AIX®, open-source databases, and AI workloads (machine learning (ML) and LLM) on Linux by using hybrid architectures across on-premises and IBM Cloud® environments. He designs and implements solutions to meet stringent RTO and RPO objectives by using Power and cloud platforms to help ensure resilience, scalability, and performance. He was a LinuxONE mainframe specialist for South America and an HPC engineer at Chile's National Laboratory for High Performance Computing (NLHPC), supporting the scientific community and operating HPC environments.

Florian Dahlitz is an AI on IBM Power Technical Specialist in Germany. He has 8 years of experience in software engineering and has focused on AI for the past 3 years. He holds a master's degree in computer science from the Karlsruhe Institute of Technology. He has delivered numerous presentations about AI on IBM Power and helps open-source projects support the ppc64le architecture.

Abdul Haleem is a Senior Linux on Power Test Engineer at IBM India Systems Development Lab with over 15 years of experience in Linux testing, automation, and enterprise system validation across Power and IBM Z platforms. His expertise includes functional verification, I/O driver testing, and continuous integration and continuous deployment (CI/CD) infrastructure design for IBM hardware. He leads the IBM Spyre Functional Test team, where he drives test automation strategies for Power platforms to help ensure reliability and scalability for enterprise AI solutions.

Blake Hoskinson MBA, BS, is a Technical Sales Leader in the US. He has over 30 years of experience in enterprise IT, with a background in Power, IBM Power Virtual Server, IBM Storage, and business resiliency. Blake has led technical sales and service delivery teams, founded and operated a successful consulting firm, and played a key role in hundreds of system implementations. His expertise spans solution architecture, infrastructure modernization, and customer engagement across diverse industries.

Bijay Dev K M is a Hardware Developer at IBM India Systems Development Lab. He has over 7 years of experience in the information technology industry and holds a master's degree in Computer Science and Engineering from the National Institute of Technology Surathkal, India. His areas of interest include AI and ML, AI accelerators, computer networks, and cloud computing.

Henrik Mader is an AI on IBM Power subject matter expert who is based in Germany. He holds a B.Sc. in Business Informatics from the Technical University of Berlin with a specialization in Data Science, and an M.Sc. in Information Systems from the Technical University of Munich, focusing on AI. Henrik has worked on AI on IBM Power for the past 3 years, building technical assets, contributing to the open-source community, and supporting enterprise clients in deploying AI workloads and defining their AI strategy on Power servers. His work ranges from implementing modern AI architectures to optimizing LLM and ML workloads for on-premises environments. Before joining IBM, Henrik worked as a Data Engineer for a DAX-40 company. He completed his master's thesis with IBM Research®, focusing on ensemble methods and uncertainty estimation in ML.

Manish Mukul is a Senior Advisory Software Engineer at IBM India Systems Development Lab with over 17 years of experience developing advanced hardware validation tools for Power servers. He designs and delivers scalable, high-performance validation frameworks that help ensure the reliability, functional integrity, and performance of next-generation Power servers. Manish works closely with silicon design, firmware, and test engineering teams to build robust, reusable validation solutions as an integrated tool suite. His work enables seamless validation workflows across development teams and contributes to the quality and readiness of emerging Power platforms. Manish holds a bachelor's degree in Information Technology from Savitribai Phule Pune University. His professional interests include engineering excellence, automation, and next-generation server architecture.

Arnold Ness P.Eng., is the IBM Canada National Hybrid Cloud Technology Leader. He drives AI innovation and workload optimization for enterprises. With decades of experience at IBM and Ciena, he helps organizations use hybrid cloud, AI, and quantum-safe cybersecurity to gain competitive advantage. Arnold holds a MBA in Information Technology Management from Royal Roads University and a degree in electrical engineering from the University of Alberta. He completed IBM's Client Executive Program at Harvard and specializes in solution design and delivery across x86, Power, IBM Z, and cloud platforms. In 2022, Arnold received the Lou V. Gerstner Jr. Award for Client Excellence for delivering hybrid cloud solutions with the Government of Canada.

Nnamdi Okore-Affia II is a Technology Expert Labs Delivery Consultant in the UK. He has 7 years of experience with Linux on Power and holds a degree in computer science from Northumbria University. His areas of expertise include Red Hat Enterprise Linux (RHEL) systems administration and AI on Power. He has worked extensively on deep learning (DL) and AI workloads.

Praveen Kumar Pandey is a Senior Software Engineer at IBM with more than 15 years of experience in functional verification and performance engineering across Power and IBM Z platforms. He specializes in designing and implementing CI/CD infrastructure for IBM hardware systems development. Praveen is an active contributor and maintainer within several open-source communities, including the Linux kernel and Linux test automation projects. He is the designer of the Spyre CI/CD framework on Power11, accelerating development and validation workflows for next-generation IBM systems. In his current role, Praveen helps ensure the functional quality and reliability of supported Linux distributions on Power servers through automation and testing methodologies. He holds a bachelor's degree in Computer Science and Engineering from the Govind Ballabh Pant Institute of Engineering & Technology in Uttarakhand, India.

Manvanthara Puttashankar is a Linux on Power QA Architect at the IBM India Systems Development Lab in Bangalore. With over 22 years of experience, he has worked on operating systems (Linux, AIX, and IBM z/VM®), software development (firmware and kernel), server platforms (IBM Power, IBM Z, and other enterprise architectures), I/O drivers (network and storage), and Spyre AI acceleration. In his role as an AI Lead, he drives AI transformation across various IBM platforms. His contributions have been recognized with several awards, including the OTAA and Tech2025 honors. He advocates for Power and IBM Z platforms and collaborates with clients and vendors worldwide.

Daniel Schenker is a Software Performance Analyst and AI and Linux Toolchain Developer who is based in Texas, US with 3 years of experience at IBM. He holds a bachelor's degree in Computer Engineering from the Rochester Institute of Technology. His work focuses on software enablement for the Spyre Accelerator cards, where he develops, maintains, and validates end-to-end software stacks supporting AI workloads on Linux systems. His areas of expertise include AI performance benchmarking and analysis, containerized deployment workflows, CI/CD integration, and the development of automation and tools for running and measuring AI workloads. He contributes to and manages open-source and internal GitHub projects that support scalable AI software deployments.

Vaibhav Shandilya is a Technology Leader at IBM with over 28 years of industry experience, including more than 17 years at IBM. He currently leads APAC Client Engineering pilots and proof-of-concept initiatives, helping clients innovate with hybrid cloud, AI and ML, and generative AI solutions. Throughout his career, Vaibhav has held key leadership roles, including managing worldwide IBM Power Techline and Power assurance teams and enabling partners across global markets. His work has influenced multimillion-dollar sales opportunities by aligning technology strategies with business goals. Vaibhav's expertise spans cloud architecture, IBM Cloud Pak®, IBM watsonx.ai®, IBM watsonx.data, AI and ML, and generative AI. He holds a master's degree in Computer Science and has published numerous technical articles on emerging technologies and best practices.

David Spurway is the IBM Power AI and Security Principal for Europe, the Middle East, and Africa (EMEA). He works with customers across the region to demonstrate how Power servers and solutions can run mission-critical applications reliably while reducing costs. David presents regularly at events worldwide both in-person and virtual on Power technologies, security, and AI modernization. He helps customers understand how to resist cyberattacks and use AI to modernize operations and deliver new business functions.

Rajalakshmi Srinivasaraghavan is a Software Performance Analyst and AI Infrastructure Strategist at IBM. She works across architecture, performance analysis, and open-source enablement to integrate software architecture with low-level hardware features for AI acceleration and compute-intensive systems. Through her open-source contributions, she has delivered performance improvements for Power and fostered community adoption across the platform.

Borislav Stoymirski is a Hardware Product Engineer for Power at IBM Bulgaria who specializes in diagnosing and resolving complex hardware and software challenges across the Power ecosystem, AIX, VIOS, Hardware Management Console (HMC), IBM i, IBM PowerVC, IBM PowerVM® NovaLink, PowerVM, Linux on Power, and Red Hat OpenShift. Since joining IBM in 2015, he has provided technical support to clients worldwide, improving system reliability and performance across enterprise environments. Borislav has authored multiple publications and led training programs within IBM and externally. Borislav holds three master's degrees from Bulgarian universities: Computer and Software Engineering from the Technical University of Sofia, Transport Machinery and Technologies from the Technical University of Sofia, and Ecology and Environmental Protection from the University of Chemical Technology and Metallurgy. His professional interests include bioinformatics, AI, machine and deep learning, cybersecurity, cloud computing, and quantum computing, with a focus on sustainability and intelligent infrastructure.

Henry Vo is an IBM Redbooks Project Leader with more than 10 years of experience at IBM. He has technical expertise in business analysis, risk assessment, root-cause analysis, and technical planning. He has held multiple roles at IBM, including project management, ST/FT/ETE tester, back-end developer, and DOL agent for New York. Henry is certified in IBM z/OS® Mainframe Practices, IBM Z systems programming, Agile, and Telecommunication Development Jumpstart. He holds a master's degree in Management Information Systems (MIS) from the University of Texas at Dallas (2012).

Jack Woehr is an IBM Champion for Power and Quantum who is based in Colorado, US. As an independent consultant, his primary focus is IBM i modernization. With more than 40 years of experience in software development, he spent two decades as a contributing editor for Dr. Dobb's Journal. A frequent speaker at industry conferences, he has delivered numerous presentations on quantum computing and has spoken internationally on LLMs and AI.

Thanks to the following people for their contributions to this project:

Andrew Laidlaw, Senior Partner Technical Specialist - Power
IBM UK

Now you can become a published author, too!

Here's an opportunity to spotlight your skills, grow your career, and become a published author—all at the same time! Join an IBM Redbooks residency project and help write a book in your area of expertise, while honing your experience using leading-edge technologies. Your efforts will help to increase product acceptance and customer satisfaction, as you expand your network of technical contacts and relationships. Residencies run from two to six weeks in length, and you can participate either in person or as a remote resident working from your home base.

Find out more about the residency program, browse the residency index, and apply online at:

ibm.com/redbooks/residencies.html

Comments welcome

Your comments are important to us!

We want our books to be as helpful as possible. Send us your comments about this book or other IBM Redbooks publications in one of the following ways:

- Use the online **Contact us** review Redbooks form found at:

  ibm.com/redbooks

- Send your comments in an email to:

  redbooks@us.ibm.com

- Mail your comments to:

  IBM Corporation, IBM Redbooks
  Dept. HYTD Mail Station P099
  2455 South Road
  Poughkeepsie, NY 12601-5400

Stay connected to IBM Redbooks

- Find us on LinkedIn:

  https://www.linkedin.com/groups/2130806

- Explore new Redbooks publications, residencies, and workshops with the IBM Redbooks weekly newsletter:

  https://www.redbooks.ibm.com/subscribe

- Stay current on recent Redbooks publications with RSS Feeds:

  https://www.redbooks.ibm.com/rss.html

1

Introducing artificial intelligence on IBM Power11

This chapter introduces the evolving role of artificial intelligence (AI) in enterprise computing and positions IBM Power11 as a strategic platform for AI deployment. It outlines the foundational concepts of AI, including machine learning (ML), deep learning (DL), and natural language processing (NLP), and explains how these technologies transform industries through automation, decision-making, and personalization. The chapter also addresses common misconceptions about AI, such as its supposed autonomy or human-like reasoning, and emphasizes the importance of responsible deployment. It categorizes AI into types from narrow AI to speculative self-aware systems and highlights current enterprise applications such as predictive analytics, fraud detection, and customer engagement.

The chapter then explores how IBM Power11 advances AI capabilities beyond previous generations. Compared to IBM Power9® and IBM Power10, Power11 offers enhanced performance, broader AI precision support, and a mature ecosystem. Key innovations include an on-chip Matrix Math Accelerator (MMA), expanded single instruction, multiple data (SIMD) acceleration for vectorized numerical workloads, and the IBM Spyre PCIe Gen5 x16 AI Accelerator Adapter (IBM Spyre Accelerator). These features enable high-throughput inference and generative AI workloads with reduced latency and energy consumption. Power11 also delivers enterprise-grade reliability, quantum-safe security, and hybrid cloud flexibility, making it ideal for deploying AI where data is. With integration into Red Hat OpenShift AI and IBM watsonx®, Power11 supports AI lifecycle management and establishes the platform for mission-critical AI applications.

This chapter describes the following topics:

- Summary
- Defining AI and understanding its limitations
- Overview of AI trends in enterprise computing
- Power11 as a strategic platform for AI
- Comparison with previous generations
- Summary

© Copyright IBM Corp. 2026.

1.1 Artificial intelligence and IBM Power11

AI is the field of computer science that is focused on creating systems that can perform tasks that typically require human intelligence. These tasks include learning from data, reasoning, problem-solving, understanding natural language, and making decisions.

AI encompasses various techniques such as ML, DL, and NLP, enabling applications such as predictive analytics, computer vision, and generative AI. In enterprise computing, AI automates processes, enhances decision-making, and delivers personalized experiences at scale.

IBM Power11 accelerates enterprise AI adoption by delivering innovations in performance, security, and scalability. It features advanced AI acceleration through on-chip MMA accelerators, expanded SIMD vector processing, and the IBM Spyre Accelerator, providing high throughput for inference and generative AI workloads. Power11 also offers hybrid cloud flexibility, enabling consistent deployment across edge environments, on-premises infrastructure, and IBM Cloud through IBM Power Virtual Server (PowerVS). This approach allows enterprises to run AI workloads wherever data is, maintaining compliance and reducing latency.

Power11 delivers 99.9999% uptime with zero planned downtime (ZPD) and includes IBM Power Cyber Vault for ransomware detection in under a minute. Quantum-safe cryptography (QSC) protects against future threats. As an enterprise-grade AI platform, Power11 integrates with Red Hat OpenShift AI and IBM watsonx to provide model governance, transparency, and lifecycle management for mission-critical AI workloads.

1.2 Defining AI and understanding its limitations

AI is a broad field that encompasses many different technologies and approaches. Although the term AI is often used today to refer specifically to large language models (LLMs) and transformer-based systems for reasoning and user interaction, this publication adopts a wider view. Here, AI includes ML, intelligent systems, and the full range of foundational technologies that collectively make up the discipline.

AI can be categorized into several types, each with distinct capabilities and enterprise applications:

- Narrow AI (weak AI)

  Designed for specific tasks within a limited domain. Enterprise examples include chatbots for customer service, recommendation engines in e-commerce, and fraud detection systems in banking.

- General AI (strong AI)

  Theoretical AI that would perform a wide range of tasks similar to human intelligence. Enterprises envision using general AI in autonomous decision-making across multiple business functions, such as supply chain optimization and strategic planning.

- Superintelligent AI

  Hypothetical AI that would surpass human intelligence in all aspects. In an enterprise context, this form of AI might enable fully autonomous operations and innovation without human intervention. This form of AI remains speculative.

- Reactive machines

  Systems that respond to inputs without memory or learning. Enterprises use them in basic automation, such as rule-based quality checks in manufacturing or simple customer query routing.

- Limited memory AI

  Most modern AI systems fall into this category. Examples include predictive maintenance in industrial equipment, real-time fraud detection in financial transactions, and autonomous vehicles for logistics.

- Theory of mind AI

  Still in research, this type might understand emotions and intentions. Enterprises might use it for advanced customer engagement systems that adapt to emotional cues or virtual assistants for HR and employee wellness.

- Self-aware AI

  A futuristic concept in which AI systems would have consciousness. In theory, enterprises might use it for fully autonomous leadership roles, but this form of AI remains purely speculative and raises significant ethical concerns.

1.2.1 Understanding AI techniques

AI uses techniques such as ML, DL, and NLP to enable applications such as predictive analytics, computer vision, and generative AI. In enterprise computing, AI automates processes, improves decision-making, and delivers personalized experiences at scale.

Machine learning

ML is a subset of AI that enables systems to learn patterns from data and improve performance without being explicitly programmed. ML algorithms use statistical techniques to make predictions or decisions based on input data.

Enterprise ML applications include demand forecasting, fraud detection in banking, and customer churn prediction.

Deep learning

DL is a specialized branch of ML that uses artificial neural networks with multiple layers to model complex patterns in large datasets. It excels at tasks like image recognition, speech processing, and generative AI because it can handle unstructured data at scale.

Some example uses for DL are computer vision for quality control in manufacturing, autonomous vehicle navigation, and advanced recommendation systems.

Natural language processing

NLP focuses on enabling machines to understand, interpret, and generate human language. It combines linguistics and ML techniques to process text and speech. NLP powers applications like chatbots, sentiment analysis, and language translation.

Enterprise examples for NLP are AI-driven customer support chatbots, document summarization for legal firms, and voice assistants for call centers.

1.2.2 Misconceptions about AI

AI is transforming industries and reshaping how businesses operate, but misconceptions about what AI can and cannot do remain widespread. These misunderstandings often lead to unrealistic expectations or unnecessary fears, which can hinder effective adoption. From assumptions that AI thinks like humans to beliefs that it can learn without data or operate fully autonomously, these myths obscure the reality of AI's capabilities and limitations.

Here are some common misconceptions about AI:

- AI thinks like humans.

  Many people believe that AI has human-like reasoning or emotions. In reality, AI systems process data by using algorithms and statistical models. They do not possess consciousness, intuition, or feelings.

- AI can learn without data.

  AI models require large amounts of high-quality data to learn patterns. Without data, they cannot improve or make accurate predictions.

- AI is always objective.

  AI can inherit biases from the data that it is trained on. If the training data is biased, the model's outputs reflect those biases, making governance and fairness critical.

- AI will replace all jobs.

  AI automates repetitive tasks and augments human decision-making, but most roles evolve rather than disappear. New jobs emerge in AI development, governance, and maintenance.

- AI understands context perfectly.

  AI models often struggle with nuanced language, cultural context, or ambiguous scenarios. They rely on patterns, not true comprehension.

- Bigger models are always better.

  While large models like LLMs are powerful, they are not always the best solution. Smaller, specialized models can outperform them for domain-specific tasks.

- AI is fully autonomous.

  Most enterprise AI systems require human oversight for governance, compliance, and ethical decision-making. AI is a tool, not a self-governing entity.

Understanding the truth behind these misconceptions is essential for enterprises to deploy AI responsibly and maximize its value.

1.3 Overview of AI trends in enterprise computing

Enthusiasm for modern AI approaches has driven rapid adoption across the technology landscape. Early successes with transformer models and their ability to engage in human-like dialogue sparked significant investment in expanding the scope and capabilities of agent-based architectures.

However, this initial excitement has been tempered by recognition of the limitations in these architectures and the challenges of broadening the functions and scalability of LLM-based agents.

Here are some of the limitations:

- Difficulty in preventing exfiltration of data
- Emergent behavior (unexpected and startling outcomes)
- Technical, existential, and epistemological challenges to validation of output
- Nearly unfathomable security and trust issues
- Disappointments in automatic (“vibe”) coding
- Often unsatisfactory return on investment (ROI)[1]

Among the challenges is the failure of large-scale models that are trained on orders of magnitude more data than popular models yet yielding inferior or disappointing results relative to the investment in their development.[2, 3, 4]

At the time of writing during Q3 2025, the overall enterprise mood is one of caution, but expectations regarding AI outcomes remain high. Teams of engineers and system programmers explore tools and scenarios while awaiting developments, particularly in the IBM Power Architecture community.

Meanwhile, demonstrations of LLM and transformer architecture capabilities, which are combined with advances in deployment patterns, indicate that LLMs will significantly affect the delivery of goods and services and transform aspects of information technology and business operations.

Over the past decade, IBM enhanced the Power Architecture to support ML training and inference workloads. The maturity of the Power11 architecture for deploying AI workloads, along with deployment tools and techniques, is the subject of this publication.

1.4 Power11 as a strategic platform for AI

IBM Power11 combines enterprise reliability, advanced security, and integrated AI acceleration to support mission-critical workloads. As organizations expand their use of AI, hybrid cloud, and data analytics, Power11 provides a foundation for running core operations while extending into AI and data processing workloads. Power11 combines IBM i, IBM AIX, and Linux support with hardware-assisted AI capabilities, enabling organizations to modernize their infrastructure while maintaining operational continuity.

[1] “ROI of AI”, IBM in collaboration with Lopez Research, December 2024
[2] “Why Super intelligent AI Isn’t Taking Over Anytime Soon”, WSJ, 2025-06-13
[3] “The Fever Dream of Imminent Super intelligence Is Finally Breaking”, NYT, 2025-09-03
[4] “The Illusion of Thinking: Understanding the Strengths and Limitations of Reasoning Models via the Lens of Problem Complexity”

1.4.1 Power11 as a trusted enterprise server

IBM Power11 is the latest generation of enterprise computing, delivering reliability, security, and performance for mission-critical workloads. As organizations expand their use of AI, hybrid cloud, and advanced analytics, Power11 provides a foundation that combines enterprise resilience with integrated AI capabilities as follows:

- Enterprise-grade reliability

  Power11 delivers 99.9999% availability with ZPD and features such as dynamic firmware updates, enabling organizations to maintain continuous operations during maintenance cycles. This resilience helps ensure that critical applications remain available, supporting uninterrupted business processes.

- Security built for the future

  Security is at the core of the Power11 architecture. It incorporates quantum-safe cryptography (QSC) to protect against emerging threats and includes IBM Power Cyber Vault, which enables ransomware detection in under a minute. These capabilities help enterprises safeguard their data and workloads against current and future attack vectors.

- Performance for modern workloads

  Power11 is optimized for AI workloads with integrated on-chip MMA accelerators, expanded SIMD vector processing, and the IBM Spyre Accelerator, delivering high throughput for inference and generative AI tasks. Support for multiple precision formats helps ensure efficient model execution, while advanced memory bandwidth and I/O capabilities enable rapid data movement for analytics and AI applications.

- Hybrid cloud flexibility

  Enterprises can deploy Power11 consistently across on-premises environments, edge locations, and IBM Cloud through PowerVS. This flexibility enables organizations to run workloads where data is, reducing latency and helping ensure compliance with data sovereignty regulations.

- Integrated AI and open ecosystem

  Power11 integrates with Red Hat OpenShift AI and IBM watsonx, providing a platform for AI lifecycle management, governance, and transparency. This ecosystem supports AI deployment across mission-critical workloads.

1.4.2 Hardware-assisted AI features in Power11

AI adoption accelerated with transformer models and their ability to engage in human-like dialogue, driving rapid investment. Today, organizations focus on overcoming LLM limitations and improving deployment efficiency. Graphics processing units (GPUs) remain essential for training but pose latency and cost challenges for real-time enterprise inference.

In contrast, enterprise platforms such as IBM Power11 integrate on-chip AI accelerators and specialized hardware such as IBM Spyre to deliver high-throughput inference with reduced latency and improved energy efficiency. This approach enables organizations to run AI workloads closer to data, reducing dependency on external GPU clusters and supporting hybrid deployments across edge, on-premises, and cloud environments.

Matrix Math Accelerator on Power11

Matrix operations form the mathematical foundation of modern ML, enabling large-scale data transformations and computations. These operations are essential for tasks such as training neural networks, performing linear algebra calculations, and optimizing models for inference.

The Power11 CPU includes the MMA feature, originally introduced in Power10[5], to accelerate matrix computations in hardware. MMA enables high-performance matrix multiplication, a critical operation in AI workloads, through specialized instructions that reduce computational impact.

This capability is complemented by support for Vector Multimedia eXtension (VMX) and Vector Scalar eXtension (VSX) instructions, which provide optimized vector processing. Together, these features allow Power11 to deliver high throughput for ML and DL tasks, enabling enterprises to run complex AI models efficiently and at scale.

SIMD acceleration on Power11

Power11 enhances its SIMD capabilities, extending the vector-processing foundation of the Power ISA to support AI, analytics, and HPC workloads. Like MMA and Spyre, SIMD on Power11 accelerates the data-parallel computations in enterprise AI pipelines.

Power11 extends the Power Architecture vector capabilities with a wide SIMD engine within each core, providing higher per-cycle throughput for vector and tensor operations. This design delivers increased parallelism and improved bandwidth for vector instructions compared to Power10.

This wide SIMD design enables workloads including dense linear algebra, signal processing, encryption, and real-time analytics to use the Power11 increased clock speeds and improved memory subsystem efficiently.

VSX and Power ISA vector enhancements

Power11 supports and enhances VSX, which is IBM's SIMD model that is rooted in AltiVec and VMX. VSX on Power11 provides a unified set of 128-bit vector registers and instructions for floating-point, integer, and logical operations. This design enables compiler and intrinsic support for porting SIMD-optimized routines from other architectures such as x86 AVX or ARM NEON.

Key benefits of the VSX ecosystem include the following items:

- High portability for SIMD-accelerated libraries (Eigen, OpenBLAS, and FFT implementations)
- Efficient execution of vector loops, particularly when combined with compiler auto-vectorization
- Support for mixed integer and floating-point vector workloads for AI preprocessing and postprocessing operations

Complementing MMA's matrix-focused acceleration, VSX SIMD serves as the general-purpose vector engine for enterprise workloads.

Integration with Power11 memory and I/O bandwidth

Power11 SIMD performance benefits from memory subsystem enhancements, including a 32-channel DDR5 OMI architecture that provides higher bandwidth than earlier generations. This architecture delivers the sustained memory throughput that SIMD workloads require for vector-dense operations, especially when processing large tensors, streaming datasets, or real-time analytics pipelines.

High-speed memory and expanded SIMD width work together to reduce pipeline stalls and improve usage of vector execution units.

[5] MMA in IBM Power10 processor

Role of SIMD in the Power11 AI stack

The Power11 AI acceleration stack includes several components:

- MMA provides matrix math for inference and generative AI.
- SIMD and VSX accelerate the vector math that is used in activation functions, normalization, quantization and dequantization, and data layout transformations.
- Spyre provides off-chip AI acceleration for generative workloads.

SIMD serves as a flexible, general-purpose acceleration layer for AI tasks that are not offloaded to MMA or Spyre.

Here are some of the functions that benefit from Power11 enhanced SIMD capabilities:

- AI frameworks (PyTorch, TensorFlow) through optimized back-end libraries
- Enterprise analytics through faster vector scanning, filtering, and transformation
- Cryptographic operations such as hashing and block-cipher acceleration
- High-performance scientific workloads where vector operations dominate compute kernels

These improvements make SIMD a part of the Power11 value proposition for enterprises seeking scalable, CPU-centric AI performance.

IBM Spyre Accelerator

The IBM Spyre Accelerator, which is integrated into Power11 servers, provides AI hardware acceleration for enterprise workloads. Designed for generative and agentic AI, Spyre enables reduced-latency inference and real-time responsiveness for AI applications that require dynamic decision-making and multi-model processing.

Spyre is a PCIe-attached system-on-a-chip that is built on 5-nm technology and features 32 accelerator cores and 25.6 billion transistors. Each card consumes 75 watts and can be clustered (up to 16 cards in a Power11 server) to scale AI capabilities[6]. This clustering enables Power11 servers to handle complex AI workloads, including LLMs, without compromising throughput or system resilience.

Unlike previous generations that relied on external GPUs, Spyre brings on-premises AI acceleration into the Power11 architecture. This integration helps enterprises maintain data locality, enhancing security and compliance by keeping sensitive data on-premises.

Spyre supports multi-model AI processing, enabling systems to use different model types together to improve inference accuracy. For example, initial predictions can be handled by the Power11 processor while more complex or uncertain cases are escalated to Spyre for deeper analysis.

The Spyre Accelerator enables Power11 to run AI workloads with improved efficiency, scalability, and security for enterprise data-driven operations.

1.4.3 IBM Power: Transforming business through seamless AI integration

IBM Power servers support enterprise computing by integrating AI into business operations. Power provides the performance, security, and flexibility to run AI in transactional systems, scale generative models, and deploy applications across hybrid environments.

[6] https://newsroom.ibm.com/2025-10-07-ibm-introduces-the-spyre-accelerator-for-commercial-availability

AI is where your data lives

In distributed data environments spanning on-premises, cloud, and edge locations, AI must be strategically positioned. IBM Power brings AI inference to where data is, eliminating the need to move sensitive information across networks or rely on external GPUs. This approach reduces latency, enhances security, and reduces operational costs. With the IBM Spyre Accelerator and IBM watsonx.data on Power servers, organizations can combine data lakehouse capabilities with off-chip acceleration for faster inference.

Continuous security for AI workloads

AI workloads must be protected from evolving threats. IBM Power servers provide security across the stack, including transparent memory encryption that protects data without compromising performance. For tasks such as generative AI, Power enables scalable inference. The platform also features IBM Power Cyber Vault, which detects ransomware threats in under a minute[7].

Hybrid flexibility without friction

Deploying AI across hybrid environments requires integration. Power servers support enterprise software for hybrid cloud deployment, enabling workloads to move between on-premises systems and PowerVS. Power provides the flexibility to train models in the cloud or run inference on-premises. With support for more than 130 open-source AI tools and packages, teams can build, deploy, and scale AI solutions.

Sustainable performance on demand

IBM Power11 delivers twice the performance per watt compared to x86-based servers, enabling organizations to reduce energy consumption[8]. The Energy Efficient mode on Power11 servers offers up to 28% better energy efficiency compared to Maximum Performance mode[9], helping enterprises reduce the environmental impact of AI operations.

Acceleration without compromise

Power servers run AI workloads at scale in real time. With high parallelism, large memory capacity, and integrated on-chip acceleration, Power enables AI to be embedded into business processes. Data scientists can run workloads without refactoring code. For LLMs, IBM Power S1022 processes up to 42% more batch queries per second than comparable x86 servers under peak load[10] while maintaining subsecond inference latency, enabling faster insights and real-time responsiveness.

---

[7] This guarantee covers only the displaying of an alert in less than one minute.

[8] Based on Quantitative Performance Index (QPI) data as of 15 May 2025 from IDC, which is available at https://www.idc.com/about/qpi. IBM Power E1150 (4x30c Power11 at 3.0-4.1 GHz) QPI of 241,000E versus HPE Compute Scale-up Server 3200 (4x60-core Intel cores at 1.9 GHz) QPI of 208,898 and usage of 75% for Power E1150 based on IBM Power Performance Utilization Guarantee and 40% for x86.

[9] Based on IBM measurements of performance per watt on servers comparing Maximum Performance Mode to Energy Efficient Mode while running compute-, disk-, and memory-based workloads on Power11 servers with fully configured sockets and memory as follows: Power E1180 with 4x10c / 64x64GB DDIMM, Power E1150 with 4x16c / 64x32GB DDIMM, IBM Power S1124 with 2x16c / 32x32GB DDIMM, IBM Power S1122 with 2x16c / 32x32GB DDIMM.

[10] Comparison based on IBM internal testing of question and answer inferencing by using the PrimeQA model (https://github.com/primeqa), which is based on Dr. Decr and ColBERT models. These results are valid as of Aug 22, 2023, and conducted under laboratory conditions. Individual results can vary based on workload size, usage of storage subsystems, and other conditions. Comparison is based on total throughput in score (inferences) per second on Power S1022 (1x20-core at 512 GB) running SMT8 versus Intel Xeon Platinum 8468 V-based (1x48-core at 512 GB) systems.

1.4.4 AI services: Turnkey AI

Spyre for Power is a portfolio of AI software products that are optimized for the IBM Spyre Accelerator. The portfolio includes the following features:

- Pre-built AI services (IBM Open-Source AI Foundation for Power)
- An inference platform (Red Hat AI Inference Server on IBM Power)
- Accelerator infrastructure software (IBM Spyre Enablement Stack for Power)
- Enablement stack updates (IBM Spyre Stack Updates for Power)

This portfolio provides AI services, deployment capabilities, and acceleration for enterprise AI on Power servers. Turnkey AI supports the transition from pilot to production, integrates with enterprise workflows such as enterprise resource planning (ERP) and IT systems, and maintains data sovereignty through on-premises infrastructure.

AI services are part of the IBM Open-Source AI Foundation for Power for Power11 servers. They provide pre-built service components that integrate with inference platforms such as Red Hat AI Inference Server. AI services support the IBM Spyre Accelerator.

AI services enable enterprises to deploy AI capabilities on Power servers. Each AI Service is a self-contained deployment unit with inference capabilities. The portfolio supports models, which include LLMs, embedding models, and reranking models.

With IBM Spyre for Power, organizations can scale AI workloads for LLM inference by using turnkey AI. For more information, see 4.2, “Turnkey AI with Spyre” on page 53.

1.5 Comparison with previous generations

The Power11 platform advances enterprise AI on Power servers. Power9 supported AI workloads, high memory bandwidth, and GPU integration. Power10 introduced enhanced processor acceleration and architectural improvements. Power11 delivers increased performance, expanded AI precision capabilities, and a mature ecosystem for modern AI applications.

Table 1-1 shows a comparison of AI-ready characteristics between the Power11 platform and previous generations.

Table 1-1 Technology comparison of the Power11 platform to previous generations

Characteristics	Power11	Power 10	Power9
Processor technology	7 nm	7 nm	14 nm
Maximum cores	16	15	12
Maximum simultaneous multithreading (SMT) threads per core	SMT8	SMT8	SMT8
Maximum frequency	3.8 – 4.4 GHz	3.75 – 4.15 GHz	3.9 – 4.0 GHz
AI precision support	FP64/32/16, Bf16, INT8 (FP16/8 with Spyre)	FP64/32/16, Bf16, INT8	FP64/32 (with GPU), FP16
Memory support	DDR5 and DDR4	DDR5 and DDR4	DDR4 and DDR3
Peak memory bandwidth	Up to 512 GBps	Up to 409 GBps	170 GBps

Characteristics	Power11	Power 10	Power9
L2 cache	2048 KB per core	2048 KB per core	512 KB per core
L3 cache	8 MB of L3 cache per core with each core having access to the full 128 MB of L3 cache, on-chip high-efficiency SRAM	8 MB of L3 cache per core with each core having access to the full 120 MB of L3 cache, on-chip high-efficiency SRAM	10 MB of L3 cache per core with each core having access to the full 120 MB of L3 cache, on-chip high-efficiency eDRAM
I/O interconnect	PCIe Gen 5	PCIe Gen 5	PCIe Gen 4, and NVLink 2.0
On-chip acceleration	Improved MMA engine and SIMD vector instructions	MMA engine	SIMD, compression and encryption (AES and SHA), and GZIP engine
Off-chip acceleration	Up to 8 IBM Spyre Accelerator adapters in an expansion drawer	N/A	Up to 6 NVIDIA GPUs in a server (NVLink 2.0)
Python and ML ecosystem	Improved Python performance, and AI and ML stack fully optimized (IBM Open-Source AI Foundation for Power)	Optimized Python, and PyTorch and TensorFlow for Power10 MMA (rocketce)	Standard Python, scientific libraries, and open-source frameworks (Open Cognitive Environment (OpenCE))
Target AI workloads	Generative AI, real-time inference, and retrieval-augmented generation (RAG)	Hybrid cloud AI, and mission-critical AI and ML inference	GPU-assisted DL, and HPC.

1.5.1 Power9

Power9 supported enterprise AI, HPC, and data analytics. Its integration with NVIDIA GPUs through NVLink 2.0 enabled fast data exchange for DL workloads. With high memory bandwidth and multi-threading capabilities, Power9 handled parallel AI tasks but relied on external accelerators for peak performance.

Power11 expands native support for AI data types, including FP64, BF16, and INT8, for training and inference across diverse models. Power11 incorporates on-chip AI acceleration, reducing dependency on external GPUs and improving performance and energy efficiency. It includes an expanded software ecosystem with optimized AI frameworks and enhanced security features for enterprise AI deployments.

GPUs

Power9 uses GPUs as its primary accelerator. The IBM Power AC922 was the first server to use NVLink 2.0 technology for DL workloads. DL models are trained by using GPUs such as the NVIDIA Tesla V100, available with either 16 GB or 32 GB of memory. Each GPU delivers up to 7.8 teraflops (TFLOPs) of double-precision performance, 15.7 TFLOPs for single precision, and up to 125 TFLOPs for DL workloads by using Tensor Cores. A water-cooled Power AC922 server (GTX model) can accommodate up to six GPUs.

NVLink 2.0

Developed jointly by IBM, NVIDIA, and the OpenPOWER Foundation, NVLink 2.0 offers up to five times the bandwidth between CPUs and GPUs compared to PCIe Gen 3 connections.

This higher bandwidth accelerates data transfer between system memory and GPUs, and between GPUs.

Each Power9 processor that is connected through NVLink 2.0 provides up to 300 GBps of bidirectional bandwidth with attached GPUs, compared to 32 GBps with PCIe Gen 3.

Figure 1-1 compares Power9 NVLink 2.0 with processor interconnects that use PCIe and first-generation NVLink. For more information about GPUs and NVLink on Power9, see *IBM Power System AC922 Technical Overview and Introduction*, REDP-5494.

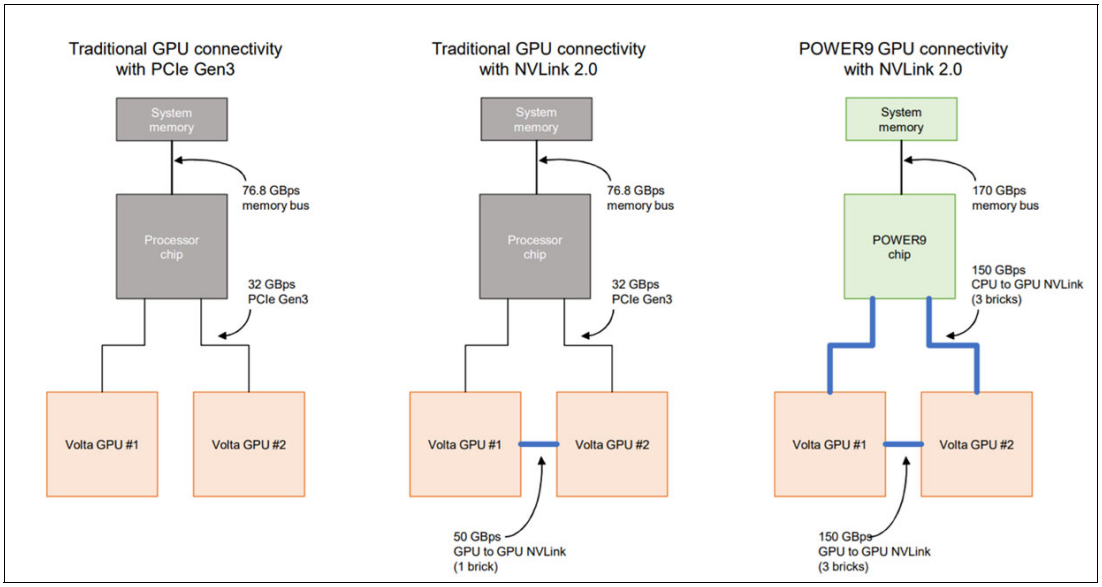


Figure 1-1 Comparison between traditional CPU and GPU interconnects and Power9 using NVLink 2.0

Target workloads

The Power9 platform, when paired with NVIDIA GPUs and NVLink 2.0 interfaces, boosts processing capabilities to meet the demands of HPC, high-performance data analytics (HPDA), and AI.

1.5.2 Power10

Power10 improved upon Power9 in AI acceleration and architectural efficiency. It featured on-chip matrix math engines to accelerate AI inference and training, reducing dependency on external GPUs. Power10 supported AI data types including BF16 and INT8, which improved performance and reduced power consumption for DL workloads. Its memory bandwidth and security enhancements supported enterprise AI deployments in regulated industries.

Power11 refines these capabilities with improved AI performance and ecosystem maturity. It supports mixed-precision AI operations and delivers higher throughput for training and inference tasks. Power11 enhances the on-chip acceleration that was introduced in Power10 for improved efficiency and scalability. The platform includes an expanded software stack and broader framework support for deploying and managing AI workloads. Compared to Power10, Power11 provides improved performance per watt and enhanced integration with AI tools for enterprise AI applications.

Matrix Math Accelerator

The MMA in Power10 accelerates compute-intensive operations such as matrix multiplication and convolutions. It uses a dense math engine (DME) microarchitecture for ML and AI inference workloads. For more information about MMA design and performance, see *Matrix-Multiply Assist Best Practices Guide*, REDP-5612.

SIMD

Power10 delivers 2× the general-purpose SIMD performance relative to Power9, stemming from improvements in execution width, instruction throughput, and ISA expansions. For matrix math and AI-oriented SIMD operations, Power10 achieves up to 4× the performance of Power9.[11] This approach aligns with IBM's push toward AI-optimized instruction sets and hardware acceleration integrated directly into the cores. Power10 includes AI-focused instruction set additions, expanding SIMD and matrix math capabilities beyond what Power9 supported, especially for matrix multiplications commonly used in machine learning workloads.

Power10 delivers two times general and four times matrix SIMD performance relative to Power9, expanding SIMD bandwidth and execution width for AI, HPC, and analytics workloads. These enhancements work with the VSX instruction set of the Power ISA, enabling compiler auto-vectorization and execution of scalar, vector, and mixed-precision operations.

These features make SIMD a core component of the Power10 acceleration approach, complementing the on-chip MMA engine for enterprise workloads.

1.6 Summary

AI transforms enterprise computing by enabling automation, advanced analytics, and personalized experiences. Power11 serves as a platform for AI deployment, delivering improved performance, security, and scalability compared to previous generations. Key features include on-chip MMA, expanded SIMD vector acceleration, the IBM Spyre Accelerator, and support for mixed-precision operations for training and inference workloads.

This chapter covered AI fundamentals including ML, DL, and NLP, along with common misconceptions and enterprise applications. It addressed industry trends, noting the initial excitement around LLMs and the subsequent focus on overcoming their limitations in scalability and performance.

Power11 provides enterprise reliability, quantum-safe security, and hybrid cloud flexibility, integrating with Red Hat OpenShift AI and IBM watsonx® for lifecycle management and governance. Compared to Power9 and Power10, Power11 delivers higher throughput, improved energy efficiency, and an expanded ecosystem for mission-critical AI deployments.

[11] https://www.hc32.hotchips.org/assets/program/conference/day1/HotChips2020_Server_Processors_IBM_Starke_POWER10_v33.pdf

2

Use cases and industry applications

This chapter explains how AI is integrated into enterprise environments by using IBM Power servers. It introduces deployment patterns that combine hardware acceleration, middleware, and application layers to embed AI into mission-critical workloads while maintaining security and compliance. These patterns support traditional AI and ML and generative AI, enabling organizations to improve operational efficiency.

The chapter covers practical applications and cross-industry use cases including predictive analytics, natural language processing (NLP), computer vision, process automation, and generative AI. It describes emerging trends such as AI for IT operations (AIOps) for automating IT operations and AI-driven DevOps for accelerating software delivery. The chapter also describes real-time inference and edge computing, demonstrating how Power11 servers deliver reduced-latency AI processing for scenarios including fraud detection, Internet of Things (IoT), and autonomous systems.

This chapter describes the following topics:

- AI deployment patterns and use cases
- AI for industries
- Cross-industry AI

© Copyright IBM Corp. 2026.

2.1 AI deployment patterns and use cases

This section describes enterprise AI deployment patterns and use cases on Power servers. The patterns define the infrastructure and integration strategy for deploying AI across Power servers and cloud environments.

They integrate accelerated hardware with traditional AI and ML and generative AI capabilities. These patterns enable organizations to embed AI into mission-critical enterprise processes.

By keeping data where it is, these patterns help ensure data privacy and regulatory compliance.

The framework supports generative AI capabilities including entity extraction and retrieval-augmented generation (RAG)-based digital assistants for enterprise use. The framework also supports pre-built AI services from the IBM catalog, including turnkey AI for enterprise workloads.

2.1.1 Enterprise AI-ready deployment pattern on Power servers

This pattern provides a blueprint for integrating AI into mission-critical workloads that use the Power infrastructure. It includes hardware acceleration and enterprise integration capabilities.

The deployment pattern supports organizations running mission-critical workloads on Power servers that want to integrate AI while maintaining on-premises control and data security.

Figure 2-1 shows the components of the deployment pattern.

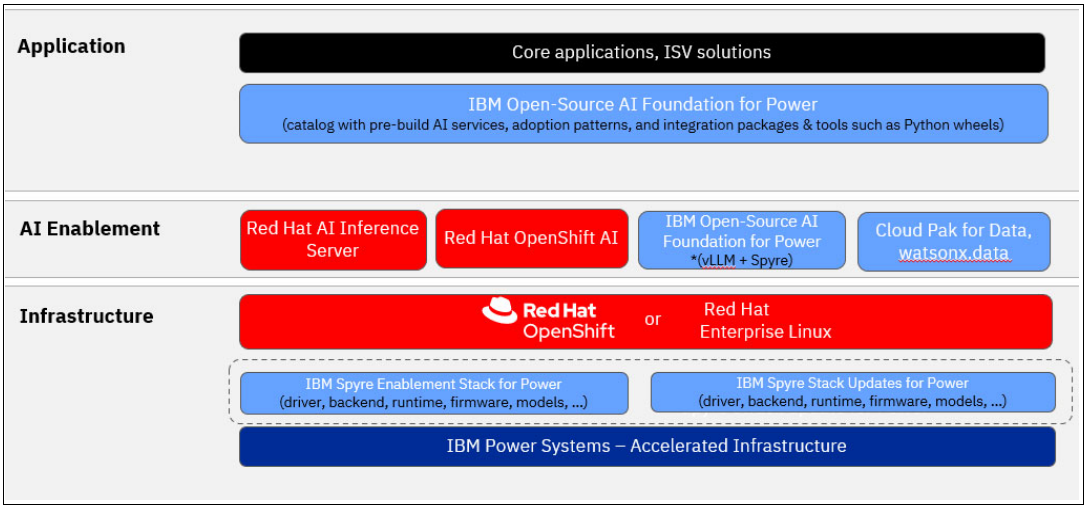


Figure 2-1 Enterprise AI-ready deployment pattern

The AI stack is organized into three logical layers: infrastructure, AI enablement, and application.

Infrastructure layer

This layer provides the foundational computing power and operating system services that are required to support AI workloads.

Here are the key components of this layer:

- Power servers – Accelerated AI Infrastructure

  Power servers provide a foundation for AI acceleration through specialized hardware:

  - Matrix Math Accelerator (MMA)

    The Power11 integrated on-chip matrix engines accelerate high-density tensor and matrix operations, delivering higher throughput for DL inference and generative AI workloads.

  - Single instruction, multiple data (SIMD) acceleration

    High-throughput vector processing integrated into each Power11 core, accelerating parallel numeric operations for AI preprocessing, feature extraction, and real-time analytics.

  - IBM Spyre PCIe Gen5 x16 AI Accelerator Adapter (IBM Spyre Accelerator)

    An off-chip architecture that supports generative AI inference and large model execution beyond the chip.

  - Middleware

    The enterprise AI stack includes middleware components that enable higher-level software to use the underlying infrastructure.

  - IBM Spyre Enablement Stack for Power

    Required for using the IBM Spyre Accelerator from higher-level software stacks. Enables integration of IBM Spyre Accelerator capabilities into AI frameworks and applications. The stack includes a PyTorch back end, run times, a device driver, card firmware, and pre-compiled models with Spyre support.

  - IBM Spyre Stack Updates for Power

    Provides regular updates for the IBM Spyre Enablement Stack for Power, including drivers, firmware, AI models, and other runtime components as needed.

- Operating systems

  - Red Hat Enterprise Linux (RHEL) on logical partition (LPAR)

    Enables execution of open-source AI foundations, including Python-based inference frameworks like vLLM, delivering high-performance inferencing on Power servers.

  - Red Hat OpenShift (with Red Hat CoreOS nodes)

    Hosts containerized enterprise AI platforms such as IBM Cloud Pak for Data (CP4D), IBM watsonx.data, and Red Hat OpenShift AI, providing orchestration and lifecycle management across the Power infrastructure.

AI enablement layer: Inference server components

This layer provides AI capabilities for model deployment, inference, and decision-making. It integrates IBM and open-source components, such as the following ones:

- Red Hat AI Inference Server

  Designed for generative AI inference and optimized for the IBM Spyre Accelerator. Red Hat AI Inference Server provides a lightweight deployment option with reduced impact for initial deployments on a single LPAR.

- Red Hat OpenShift AI

  A platform for building, deploying, and managing AI applications by using Kubernetes-native tools. Red Hat OpenShift AI includes hardware acceleration support and enterprise features such as high availability (HA), user management, multi-tenancy, and scalable infrastructure for production AI workloads.

- IBM Open-Source AI Foundation for Power – Native integration

  Native integration of vLLM with the IBM Spyre Accelerator within the IBM Open-Source AI Foundation for Power. This open-source layer includes pre-built AI services and packages and supports vLLM integration with the IBM Spyre Accelerator for open-source inference.

- CP4D

  CP4D runs on Red Hat OpenShift and serves as a platform that integrates AI and data services for the data-to-AI lifecycle.

- watsonx.data

  A data lakehouse for analytics and AI workloads. It is deployed with CP4D to support data-intensive inference use cases.

Application layer

This layer integrates AI into enterprise applications, providing business solutions that use the underlying AI and infrastructure layers.

This layer consists of the following items:

- IBM Open-Source AI Foundation for Power (application integration)

  Provides a catalog of pre-built AI services and integration packages, which includes the following tools:

  - Python wheels for object detection and image classification
  - PDF2Text and OCR for document processing

- ISV solutions (industry-specific or AI use-case specific)

  Prepackaged AI solutions for core enterprise applications. The targeted domains include the following ones:

  - Core banking systems
  - Enterprise resource planning (ERP)
  - Enterprise Content Management (ECM)

Data sources and AI readiness flow

Enterprise data comes from transactional systems and databases, such as the following ones:

- IBM Db2® (on IBM AIX and IBM i)
- Oracle
- PostgreSQL
- SAP HANA

Depending on the use case, data exists in various formats:

- For structured analytics, structured data flows from source databases to data warehouses for reporting and BI.
- For cold or archival data, data can be offloaded to watsonx.data.
- For unstructured data, data is loaded into watsonx.data for RAG solutions.

watsonx.data provides data transformation engines for AI:

- Spark and Presto

  Transform structured data for predictive AI.
- OpenSearch

  Vectorize unstructured data for RAG workflows.

The workflow supports traditional and generative AI models.

2.1.2 Traditional AI and ML on Power servers: An on-premises AI and ML deployment pattern

This deployment pattern supports enterprises running mission-critical workloads on Power servers that want to integrate traditional AI and ML capabilities while maintaining on-premises control and data security for real-time decision-making.

Figure 2-2 shows the deployment pattern with the AI and ML pipeline and inference flow in the top layer and the infrastructure components that host the functions in the bottom layer.

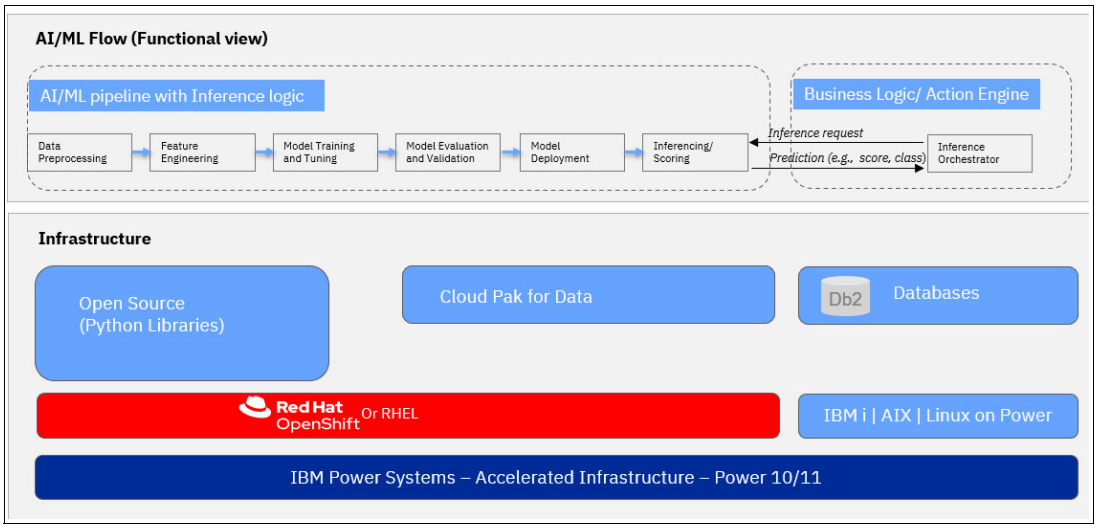


Figure 2-2 On-premises AI and ML deployment pattern

This section describes real-time AI and ML applications that integrate with source data. Although the emphasis is on real-time use cases, the underlying logic also applies to batch inference scenarios.

The transaction control logic typically is on the AIX or IBM i LPAR. When a transaction record is inserted or updated in the database, a trigger or application logic initiates a synchronous API call to a classification inference endpoint that is exposed by the AI and ML pipeline running on Linux LPARs. The model evaluates the transaction in real time and returns a risk score or classification, which determines the action (for example, approve, flag, or hold the transaction).

AI and ML model training and inference are performed locally on Power servers. Enterprise data remains on-premises, which helps ensure security for sensitive information and compliance with data residency and regulatory requirements.

Real-time AI and ML applications, such as fraud detection, operate on source data to meet reduced-latency requirements. Other applications rely on curated data from warehouses or lakehouses (for example, watsonx.data) for model training and batch inference.

Infrastructure layer

The infrastructure layer is composed of the following parts:

- IBM Power servers

  IBM Power10 and Power11 servers provide infrastructure for AI acceleration by using math libraries that are optimized for on-chip MMAs.

- AIX and IBM i LPARs

  These operating system platforms host mission-critical workloads that include core banking applications, ERP systems (for example, SAP and JD Edwards), and logistics and supply chain platforms.

- Databases (for example, Db2) on AIX and IBM i

  Databases that store structured transactional data, such as customer activity logs, payment records, and ERP and core banking transactions.

- RHEL LPAR or Red Hat OpenShift

  These containers provide environments for building and running AI and ML pipelines and exposing inference endpoints:

  - RHEL LPARs provide Linux environments on Power servers. These partitions run Python-based ML models and host frameworks such as TensorFlow and Scikit-learn.
  - Red Hat OpenShift provides a Kubernetes-based platform for container orchestration and supports AI and ML workflows through containerized deployments. It integrates with IBM Cloud Pak for Data (CP4D) for model development, deployment, and management.

- Open-source Python libraries on RHEL

  The Python open-source ecosystem provides libraries such as Scikit-learn for ML algorithms, Pandas for data manipulation and analysis, and TensorFlow for DL that support the AI and ML pipeline. These tools enable developers to handle tasks from data preprocessing and feature engineering to model training, evaluation, and deployment.

- CP4D on Red Hat OpenShift

  CP4D offers a suite of integrated modules that streamline the AI and ML lifecycle. The AutoAI module automates key stages of model development, including data preparation, algorithm selection, and pipeline generation.

- The Watson Studio Learning module provides a collaborative environment for data scientists to develop notebooks, visualize data, and build models efficiently.
- The Watson Machine Learning module serves as the runtime engine for training, deploying, and managing ML models.
- IBM Knowledge Catalog can be integrated for data governance, cataloging, and metadata management during data preparation.

AI and ML flow (functional view)

This layer represents the logical and functional architecture of how tasks run across systems. A typical ML pipeline includes the following steps:

1. Data preprocessing: Clean raw data to make it usable.
   - Handle missing values.
   - Convert data types (strings to numerics).
   - Encode categorical variables (one-shot, label encoding).
2. Feature engineering: Create or select the most informative features.
   - Combine or split columns.
   - Extract time and date features.
   - Apply domain logic (flag anomalies).
   - Drop irrelevant or highly correlated features.
3. Model training + tuning; Train models on preprocessed data.
   - Choose an algorithm, such as RandomForest, XGBoost, or LightGBM.
   - Tune hyperparameters (for example, depth and learning rate).
   - Do cross-validation.
   - Perform model evaluation validation.
   - Use metrics like accuracy, precision, recall, F1-score, and ROC AUC.
   - Check for overfitting or underfitting.
4. Model deployment: Make the model accessible for inference.
   - Export as `.pkl`, `.pmml`, or other formats.
   - Deploy to a RESTful endpoint (for example, Flask, FastAPI, or Watson ML).
5. Inference: Generate predictions from new data.
   - Accepts raw input, preprocesses (if pipeline is included), and returns output.
   - Real-time (RESTful API) or batch inference (CSV, database, or others).

Business logic and action trigger

This component includes enterprise business applications running on AIX and IBM i, integrated with AI and ML inference orchestration. Enterprise systems such as ERP, core banking, and logistics on AIX and IBM i platforms connect with AI capabilities by using APIs and event bridges (for example, RESTful APIs, message queues, or Kafka).

An Inference Orchestrator connects AI models with business logic. It transforms data formats into model-ready inputs and routes AI predictions back to enterprise systems.

Key use cases include the following ones:

- Fraud detection in banking
- Demand forecasting in logistics
- Customer churn prediction
- Credit scoring
- Inventory optimization

A traditional AI and ML pipeline build approach on Power

A traditional AI and ML pipeline is implemented on a Power infrastructure by using the following commonly used approaches:

- Open-source

  Open-source Python library-based solutions provide control over pipeline design and development for prototyping and experimentation. However, this approach lacks built-in governance and automation and requires in-house expertise in ML and DevOps to scale.

- CP4D

  CP4D solutions provide ML pipelines with governance, compliance, and production capabilities. They integrate with enterprise data systems and business processes, supporting auditability, explainability, and scalability. CP4D supports organizations requiring security, collaboration, and lifecycle management.

- Hybrid approach

  Organizations combine open-source tools (for example, Python, Scikit-learn, and Jupyter) with CP4D capabilities. Data scientists develop scripts within CP4D's notebook environment by using flexibility and governance features.

Note: For experimentation, data science teams use synthetic, sample, or public datasets that are external to enterprise databases for initial model training and validation. The model is later fine-tuned and validated with production data from the database (for example, Db2).

2.1.3 Hybrid agentic AI with IBM watsonx Orchestrate and on-premises Power

This pattern provides a hybrid architecture combining cloud-native orchestration (IBM watsonx Orchestrate®) with on-premises compute. It combines structured ML pipelines with generative AI inference.

This deployment pattern supports enterprises running mission-critical workloads on Power servers that want to integrate generative agentic AI capabilities while maintaining on-premises control and data security for real-time decision-making.

Architecture overview

watsonx Orchestrate on IBM Cloud provides agentic AI capabilities for back-end systems running on IBM Power (IBM i and AIX). Hybrid infrastructure connectivity between cloud and on-premises environments uses connectivity solutions such as IBM Cloud Satellite®.

Generative AI modernization provides workflow automation and decision support for traditional enterprise workloads.

For customer-initiated flows, an AI agent presents as a chatbot interface in a web page. A watsonx Orchestrate® agentic workflow retrieves data from on-premises Power servers for contextual processing.

For system-initiated flows, an on-premises event (for example, a transaction) pushes to IBM watsonx Orchestrate through an API. watsonx Orchestrate initiates workflows, performs generative AI tasks, and sends messages.

The pattern exposes subsets of on-premises enterprise data to AI agents running in IBM Cloud. Industry examples include fraud detection and prevention, predictive maintenance, and customer support with context-aware communication in banking, finance, and healthcare.

Figure 2-3 shows how watsonx Orchestrate operates in hybrid enterprise environments.

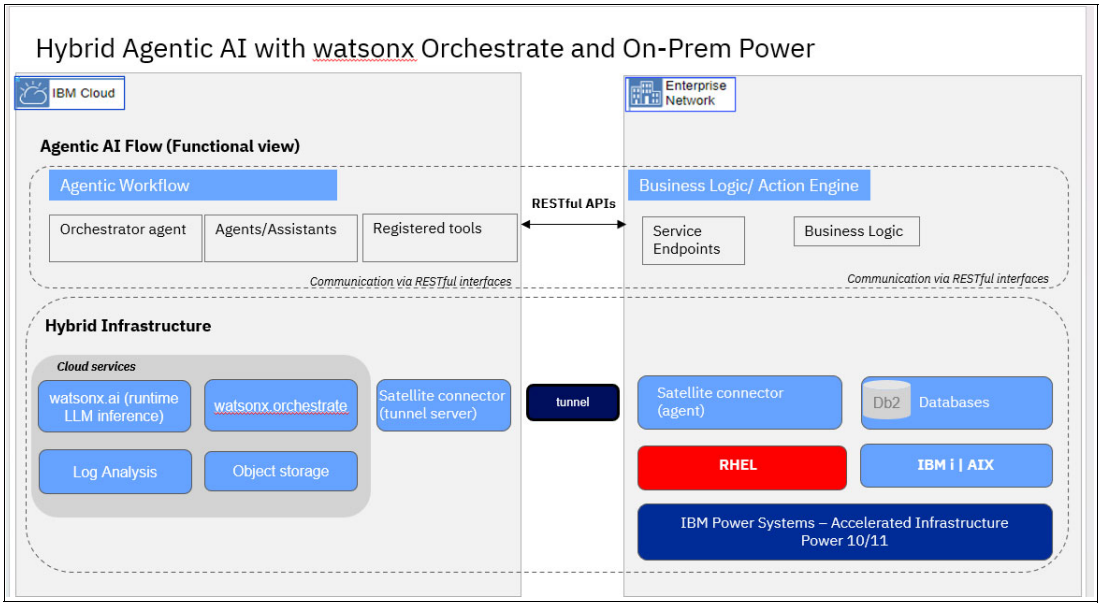


Figure 2-3 Hybrid agentic AI with watsonx Orchestrate and on-premises Power

Hybrid infrastructure layer

This layer defines the physical and network topology for secure communication across cloud and on-premises environments.

- The cloud side consists of the following items:
  - IBM watsonx Orchestrate

    Coordinates agentic workflows.
  - IBM watsonx.ai

    Provides an LLM run time for generative AI tasks (for example, summarization, classification, and decision support).
  - IBM Satellite® Connector (Tunnel Service)

    Enables cloud agents to interact with on-premises services through secure tunnels without exposing internal networks.
- The on-premises enterprise side consists of the following items:
  - IBM Power10 and Power11 servers

    Provide infrastructure for AI acceleration by using math libraries that are optimized for on-chip MMAs and IBM Spyre Accelerators.
  - RHEL

    Hosts the Satellite Connector Agent, which establishes outbound tunnels to the cloud.
  - AIX and IBM i with Db2

    Enterprise systems running mission-critical workloads and exposing APIs or services for orchestration.

Agentic AI flow layer (functional view)

This layer represents the logical architecture for task execution across systems.

- Cloud side
  - Orchestrator Agent

    Manages workflow execution and coordination.
  - Agents and assistants

    AI agents that perform tasks such as data extraction, decision-making, or user interaction.
  - Registered tools

    Service endpoints (APIs and connectors) that agents use to perform actions.
- On-premises side
  - Business logic

    Application logic (for example, RPG programs, COBOL routines, and Java services) that drives enterprise processes.
  - Service endpoints (tool implementations)

    RESTful APIs or service wrappers exposing business logic to the orchestration layer.

2.1.4 Pre-built AI use cases with IBM Spyre for Power

IBM Spyre for Power provides AI capabilities for enterprises, including digital assistants. The offering includes AI solutions for enterprise use cases. The foundation is the AI services layer, which includes capabilities such as document digitization, entity extraction, and vector search. These services support adoption patterns. The adoption pattern layer combines AI services into enterprise-ready workflows.

Applications deliver business value through use cases for specific enterprise functions such as IT service management (ITSM), HR automation, and supply chain optimization.

IBM Spyre for Power includes pre-built AI services that integrate into enterprise processes following adoption patterns.

Figure 2-4 on page 25 shows example use cases.

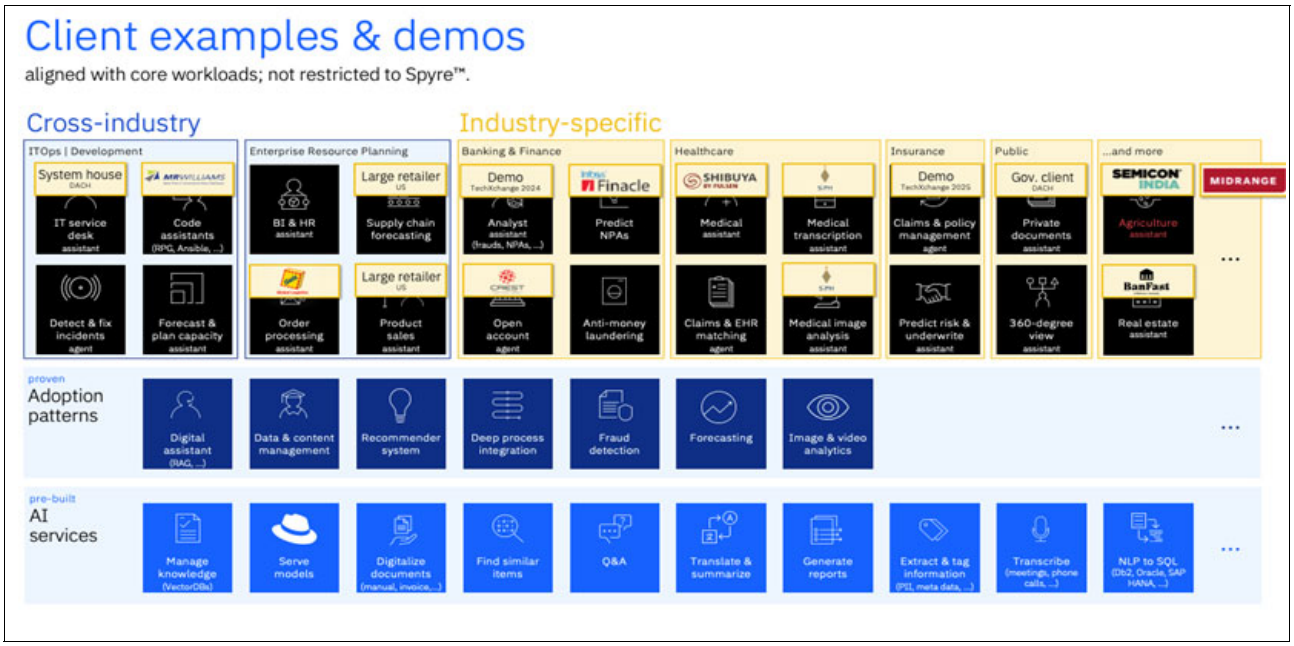


Figure 2-4 Example use cases

For example, a manual ERP order process can be automated. AI services read incoming order emails, extract details, and populate ERP fields faster than manual processing.

For the initial released of IBM Spyre for AI, the focus is on digital assistants (RAG pipelines), which include AI services such as document digitization from PDF to Markdown by using Python libraries and vector database creation. More use cases are being developed for future releases.

Figure 2-5 shows this configuration.

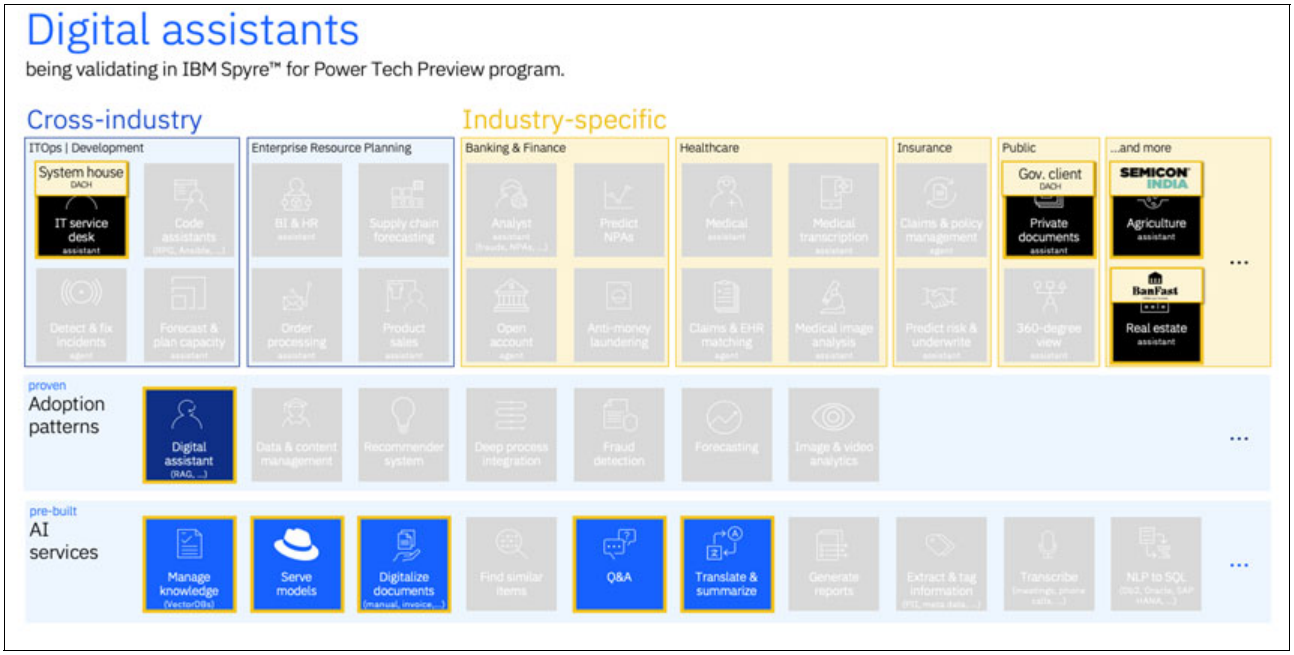


Figure 2-5 Digital assistant use cases

Entity extraction is an AI service that integrates into existing workflows.

Figure 2-6 shows these use cases.

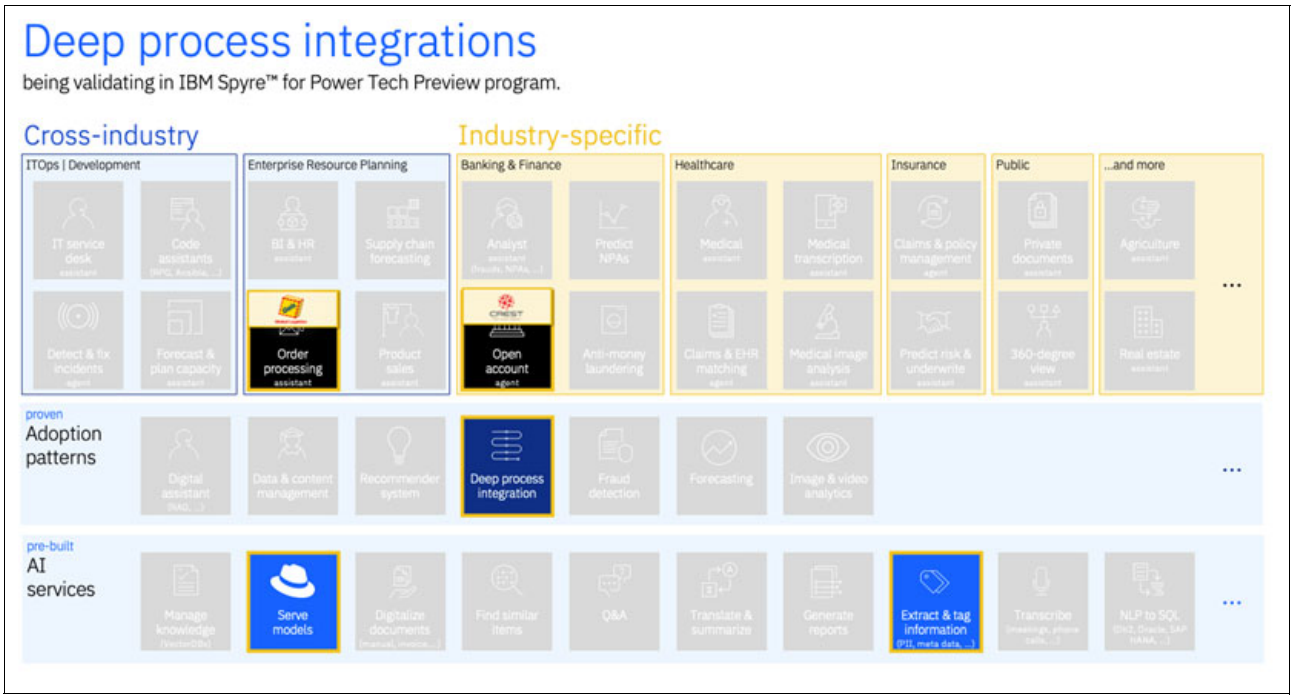


Figure 2-6 Process integration use cases

2.2 AI for industries

AI addresses sector-specific needs across industries. From banking and finance to healthcare, manufacturing, and government, AI provides capabilities for challenges such as fraud detection in financial services, predictive maintenance in manufacturing, personalized treatment in healthcare, and citizen service automation in government. Using analytics, ML, and generative AI, organizations can improve decision-making and processes. Industry-focused solutions align with regulatory, operational, and business requirements.

2.2.1 Banking and finance

AI and ML have changed the banking and finance industry, improving efficiency, customer experiences, security, and risk management. Banks deploy AI across business functions.

AI adoption began in the 1980s with expert systems for fraud detection and credit scoring. Early ML algorithms supported risk assessment, trading, and compliance monitoring as data sources expanded. Digital banking and cloud computing after 2010 provided more data for AI development.

Traditional ML evolved into DL and generative AI, enabling conversational chatbots, predictive analytics, and real-time fraud detection. Regulatory technology (RegTech), robo-advisors, and algorithmic trading moved AI from back-office tasks to customer-facing and mission-critical roles.

Customer service

Chatbots and virtual assistants are used in the industry for account queries and financial advice. They improve operational efficiency and customer satisfaction.

Fraud detection

AI systems monitor transactions in real time to detect unusual behaviors that might signal fraud or money laundering. Trained on historical data, these models establish normal user patterns and combine them with identifiers such as email, phone number, and IP address to detect anomalies. This approach reduces fraud losses and false positives.

Credit scoring

Banks use AI that analyzes hundreds of data points, including alternative and real-time data to deliver more accurate and inclusive credit decisions. This process enables lenders to tailor loan terms, interest rates, and repayment plans to a borrower's financial situation.

Compliance and regulatory reporting (RegTech)

Financial institutions use AI to automate suspicious activity reports, verify know-your-customer (KYC) documents, and perform compliance checks. NLP systems interpret new regulations and adjust workflows, reducing risk and manual effort during audits.

Investment management

AI systems analyze market trends, run trades, and update portfolio strategies in real time. Generative AI simulates market scenarios, enabling advisors and clients to make decisions for portfolios, hedging, and regulatory stress tests. Robo-advisors provide wealth management by tailoring strategies, automating rebalancing, and synthesizing research.

Operations

AI reconciles transactions, monitors operational risk, and reduces errors in reconciliations and settlements across institutions.

Summary

AI supports banking and finance operations through fraud detection, credit scoring, investment management, and operational efficiency. AI enables institutions to provide services while maintaining security and compliance requirements.

Generative AI provides capabilities for financial advisory, investment strategies, and scenario modeling. Quantum computing might improve risk modeling and portfolio optimization. AI-driven RegTech continues to evolve, automating compliance processes.

Financial institutions must address algorithmic transparency, bias mitigation, and data privacy to meet regulatory requirements. Collaboration between banks, technology providers, and regulators is needed to establish frameworks and governance standards for AI use in finance.

2.2.2 Government

AI is embedded in government functions, including citizen services, tax compliance, policing, and infrastructure management. Local, regional, and national agencies use AI for multiple applications.

Citizen services

Government agencies handle large volumes of routine inquiries, similar to corporate call centers. AI chatbots respond to questions about taxes, permits, policies, and procedures, reducing staff workload and improving response times.

Transportation and infrastructure

Governments use AI to monitor traffic conditions, analyze flow patterns, and adjust signals in real time, reducing congestion and improving safety. The same data supports predictive maintenance for infrastructure, helping prevent failures. AI analyzes sensor data from public vehicles such as buses, garbage trucks, and mail carriers to schedule maintenance, minimizing downtime. Local governments apply AI to optimize energy use in facilities, adjust street lighting, and manage water supply, driving sustainability and cost savings. Planning agencies use AI to evaluate demand, demographics, and environmental data, enabling improved project prioritization and funding allocation for roads, utilities, and public buildings.

Law enforcement and public safety

Police departments use predictive analytics to forecast crime hotspots and allocate patrols, improving public safety and resource efficiency. ML models analyze historical incident data to identify trends and guide deployment decisions, complementing traditional crime analysis. Public agencies use computer vision and pattern recognition to monitor infrastructure and public spaces, improving threat detection. These applications raise governance and privacy concerns.

Border security and immigration

The European Union (EU) iBorderCtrl system combines automated questioning, facial analysis, and risk-scoring to prescreen travelers, processing low-risk crossings and focusing human attention on higher-risk cases. U. S. Customs and Border Protection apply AI to screen cargo at ports of entry, validate identities through the CBP One app, and improve threat awareness at the border. AI baggage screening uses object detection and image classification to identify prohibited items in carry-on luggage. Immigration agencies use AI to streamline services, reducing paperwork by consolidating customer information from multiple systems.

Tax administration and fraud detection

AI models assist tax authorities in selecting samples for audits and identifying returns likely to contain errors or unpaid taxes. Canada's tax agency uses ML to cross-check filings against financial data, uncovering patterns of evasion and focusing audits on high-risk cases. The Australian Taxation Office uses AI to analyze returns, flag anomalies, and guide compliance strategies, improving accuracy and efficiency. Governments use AI to detect suspicious benefit claims by identifying patterns such as repeated contact details or inconsistent information, reducing welfare fraud. Real-time monitoring of transactions in taxation, procurement, and social programs helps prevent and investigate fraud.

Environmental regulation

The U. S. Environmental Protection Agency uses AI to analyze air quality data in real time, informing pollution control policies and enabling cities to respond to environmental risks. Municipalities use AI to integrate environmental and demographic data into urban planning, green space allocation, and resilience projects, aligning infrastructure with climate and population trends. Regulators use AI to process large volumes of filings, comments, and technical documents, helping staff identify key issues and patterns for developing or enforcing regulations.

Patent examination and intellectual property

DesignVision, an AI tool, searches US and international industrial design collections by using images as queries. It provides access to design patents, registrations, trademarks, and industrial designs from more than 80 global registers. The U.S. Patent and Trademark Office uses AI to help examiners locate relevant documents, streamlining the search and adjudication of new patent applications.

Summary

AI improves government operations through applications in citizen engagement, infrastructure management, law enforcement, and environmental regulation. AI supports decision-making and risk management. AI introduces challenges in privacy, transparency, and governance.

2.2.3 Manufacturing

AI and ML have improved manufacturing efficiency through predictive maintenance, supply chain optimization, and product design support. Manufacturers deploy AI across production lines, quality control, and logistics.

AI adoption in manufacturing began with rule-based systems for process automation and quality checks in the 1980s. As sensor technology and industrial IoT matured, ML algorithms enabled predictive maintenance and real-time monitoring of equipment health. Industry 4.0 initiatives after 2010 increased AI integration by using data from connected machines and enterprise systems.

Traditional ML evolved into DL and generative AI, enabling applications such as defect detection through computer vision, dynamic scheduling, and autonomous robotics. These technologies support mission-critical production and operations.

Predictive maintenance

AI models analyze sensor data from machines to predict failures before they occur, reducing downtime and maintenance costs. By identifying patterns in vibration, temperature, and pressure readings, these systems enable interventions and extend equipment life.

Quality control

Computer vision systems that use DL inspect products in real time, detecting defects. This approach improves consistency, reduces waste, and helps ensure compliance with quality standards.

Supply chain optimization

AI systems forecast demand, optimize inventory, and improve logistics. By integrating data from suppliers, production schedules, and market trends, these systems reduce delays and operational costs.

Robotics and automation

Collaborative robots (cobots) that use AI adapt to tasks on the factory floor, improving flexibility and productivity. Generative AI supports robotic programming by simulating scenarios for improved performance.

Product design and innovation

AI speeds design cycles by analyzing customer feedback, material properties, and performance data. Generative design algorithms propose solutions that balance cost, durability, and sustainability.

Summary

AI supports manufacturing through predictive maintenance, quality control, supply chain optimization, and robotics. Generative AI provides capabilities for design and simulation. Edge AI enables decision-making on production floors. Manufacturers must address data security, workforce training, and AI governance.

2.2.4 Healthcare

AI use in healthcare includes expert systems from the 1960s, computer-aided diagnosis systems from the 1970s, image recognition systems from the 1980s, and NLP systems from the 1990s that could “understand” medical texts. AI use in healthcare continues to increase, improving patient care, supporting treatment development, and increasing efficiency.

Drug development

Drug discovery is a complex, costly, and time-consuming process that typically takes 10 - 15 years to bring a single drug to market. Success rates are low, with only about 10% of candidates reaching the market.

ML and generative AI help researchers identify and optimize drug candidates, accelerating the process and improving efficiency. AI analyzes large datasets of scientific literature, genetic information, and chemical compounds to identify potential drug candidates. LLMs are used to design proteins, predicting how changes in an amino acid sequence affect its function.

AI makes clinical trials more efficient. By analyzing electronic health records, AI can identify trial participants based on factors such as age, medical history, and genetic makeup. AI can optimize patient stratification and identify medical “twins” for control groups.

AI is used to repurpose existing medicines by reviewing health records and scientific journals to discover potential new applications for existing drugs.

Cancer drug research

AI supports cancer drug discovery by analyzing large datasets of genomic information, clinical trial results, and chemical compound libraries. ML models identify drug candidates by predicting molecular interactions and assessing toxicity profiles, reducing research time and cost. DL techniques enable virtual screening of compounds, while generative AI designs molecules optimized for efficacy and safety. AI supports personalized oncology by matching therapies to individual genetic profiles, improving treatment outcomes. AI platforms integrate real-world evidence and simulation models for cancer research.

Patient care

AI analyzes medical images such as X-rays and CT scans to detect abnormalities. In emergency situations, faster diagnosis can improve patient outcomes. In routine scans, AI can identify early signs of medical conditions that are unrelated to the current examination.

Patients are monitored in facilities or with remote devices to collect data such as heart rate, oxygen levels, and glucose levels. AI analyzes that data to identify early signs of problems.

AI robotic systems enable surgeons to perform surgeries with improved precision and control. These procedures can be less invasive with fewer complications and shorter patient recovery times.

AI analyzes a patient's medical history, lifestyle, and genetic information to create treatment plans, which include matching medications to patients that are more likely to be effective or cause fewer side effects.

Administration

NLP systems are used to transcribe spoken words into notes. This technology can analyze patient conversations in real time and extract relevant information for clinical notes.

Administrative tasks such as scheduling, coding, room assignments, and billing can be automated, which reduce costs.

AI can analyze historical data, news reports, and social media to predict demand, enabling administrators to forecast admission rates and staffing requirements.

Key challenges

AI in healthcare raises ethical concerns due to sensitive data and high-stakes decisions.

AI models might show biases from training data that is not representative of populations. This situation is an issue in medicine, with women and minorities underrepresented in medical research and studies. Organizations must carefully select data for AI training.

Security and ownership of medical data must be addressed. Patient information is sensitive and protected by laws such as HIPAA and the General Data Protection Regulation (GDPR). Data must be protected during AI training and use.

Overreliance on AI in diagnosis and patient care can reduce empathy and trust between providers and patients, which raises concerns about declining clinical skills and reasoning among healthcare professionals.

2.2.5 Airlines

AI supports airline operations through improvements in efficiency, passenger experience, safety, and maintenance. Airlines deploy AI for scheduling, operations, and customer service.

Evolution of AI in airlines

Early AI adoption focused on revenue management and flight scheduling by using rule-based systems. As big data and cloud computing developed, ML enabled predictive analytics for demand forecasting and dynamic pricing. DL and generative AI support applications such as flight optimization, travel planning, and predictive maintenance.

Predictive maintenance

AI models analyze sensor data from aircraft engines and systems to predict component failures before they occur, which reduces downtime, improves safety, and lowers maintenance costs.

Flight operations optimization

AI systems optimize flight paths, fuel consumption, and crew scheduling by analyzing weather patterns, air traffic, and operational constraints, which helps airlines reduce delays and improve on-time performance.

Customer experience

Chatbots and virtual assistants support booking, check-in, and flight status inquiries. Generative AI enables personalized offers, loyalty program recommendations, and travel itineraries.

Revenue management and dynamic pricing

AI algorithms forecast demand and adjust ticket prices based on factors such as seasonality, competitor pricing, and booking trends, which increases revenue while maintaining competitive pricing.

Safety and compliance

AI systems monitor flight data and operational logs to detect anomalies and help ensure compliance with aviation regulations. NLP analyzes incident reports and regulatory updates.

Crew and resource scheduling

AI optimizes crew assignments and resource allocation, which reduces bottlenecks and improves workforce efficiency.

Summary

AI supports airline operations through predictive maintenance, flight optimization, passenger services, and dynamic pricing. Generative AI provides capabilities for operational planning and customer engagement. Edge AI enables in-flight decision-making. Airlines must address data privacy, algorithmic transparency, and cybersecurity.

2.2.6 Retail

AI supports retail operations through personalized shopping experiences, supply chain optimization, and operational efficiency improvements. Retailers deploy AI for merchandising, customer engagement, and inventory management.

Evolution of AI in retail

Early AI adoption focused on product recommendations and demand forecasting by using rule-based systems. As e-commerce and omnichannel retail developed, ML enabled dynamic pricing, personalized marketing, and real-time inventory optimization. DL and generative AI support applications such as virtual shopping assistants, automated store operations, and customer experiences.

Personalized recommendations

AI analyzes customer behavior, purchase history, and preferences to deliver product recommendations across online and in-store channels.

Dynamic pricing and promotions

AI algorithms adjust prices and promotions based on demand, competitor pricing, and inventory levels.

Inventory and supply chain optimization

AI forecasting predicts demand, reducing stockouts and overstock. AI systems optimize logistics and warehouse operations.

Customer service and engagement

Chatbots and virtual assistants support inquiries, returns, and product searches. Generative AI creates marketing content and shopping experiences.

Fraud detection and security

AI monitors transactions and user behavior to detect anomalies, preventing payment fraud and protecting customer data.

Visual search and augmented reality

Computer vision enables customers to search for products by using images, while augmented reality (AR) applications allow virtual try-ons.

Summary

AI supports retail operations through dynamic pricing, inventory optimization, and shopping experiences. Generative AI provides capabilities for marketing and virtual store environments. Edge AI supports analytics in physical stores. Retailers must address data privacy, AI governance, and system integration.

2.2.7 Agriculture

AI supports agricultural operations through precision farming, crop yield improvement, and resource optimization. Farmers and agribusiness deploy AI for soil analysis, irrigation management, and supply chain efficiency.

Evolution of AI in agriculture

Early AI adoption focused on automation for irrigation and pest control. As IoT sensors and satellite imagery developed, ML enabled predictive analytics for crop health and weather forecasting. DL and computer vision support applications such as automated harvesting, disease detection, and yield prediction.

Precision farming

AI systems analyze soil conditions, moisture levels, and nutrient requirements to optimize planting and fertilization schedules.

Crop monitoring and disease detection

Computer vision and drone-based imaging detect early signs of disease, pests, and nutrient deficiencies.

Weather forecasting and risk management

AI models predict weather patterns and climate risks for agricultural planning.

Automated irrigation and resource management

AI irrigation systems adjust water distribution based on soil and weather data.

Supply chain optimization

AI forecasts demand and optimizes logistics for transporting produce.

Robotics and autonomous equipment

AI robots perform tasks such as planting, weeding, and harvesting.

Summary

AI supports agricultural operations through precision farming, crop monitoring, irrigation, and supply chain optimization. Generative AI provides capabilities for crop modeling and breeding strategies. Edge AI enables decision-making in the field. Farmers must address data privacy, infrastructure requirements, and workforce training.

2.3 Cross-industry AI

AI supports business operations across sectors through applications such as the following ones:

- Predictive analytics for trend forecasting and risk management
- NLP for chatbots, virtual assistants, and document processing
- Computer vision for image and video analysis in security, quality control, and content moderation
- Recommendation systems for products, services, and content
- Process automation for tasks such as data entry and reporting
- Anomaly detection for fraud prevention, cybersecurity, and operational monitoring
- Generative AI for creating text, images, and code for marketing, design, and development
- Optimization algorithms for supply chains, resource allocation, and scheduling
- AIOps for incident detection, root cause analysis, and system performance optimization
- AI-driven DevOps for software delivery through testing, deployment automation, and failure analysis

2.3.1 AI in IT operations

Enterprise IT environments have become more complex. Hybrid cloud architectures, containerized applications, and microservices have increased operational data volume and speed. Traditional monitoring tools cannot process this data effectively.

Organizations use AIOps to improve IT operations efficiency and responsiveness.

What is AIOps

AIOps platforms analyze telemetry data, including logs, metrics, events, traces, and configuration details. Using ML and statistical techniques, these platforms can detect anomalies, correlate related events, and predict potential service disruptions, which enable IT teams to reduce downtime and accelerate issue resolution.

Key benefits

AIOps provides the following capabilities for IT operations:

- Alert reduction

  AIOps groups similar notifications and filters non-actionable alerts.

- Root cause analysis

  AI maps system dependencies and identifies underlying issues in distributed environments.

- Automation

  Tasks such as restarting services, scaling resources, or applying updates can run automatically, reducing manual effort.
- Capacity planning and optimization

  AIOps analyzes historical data and usage trends to support resource allocation and cost control.
- Security and compliance

  Continuous monitoring and automated responses support security and policy adherence.
- ITSM enhancement

  AI enables automated ticket handling, sentiment analysis, and chatbot interfaces.

Why AIOps now

IT ecosystems spanning hybrid clouds, microservices, and edge computing generate logs, metrics, and events across network, compute, storage, and application layers. Traditional monitoring systems that are designed for static infrastructures cannot process this data volume. They produce excessive alerts, lack event correlation, and require manual intervention.

AIOps addresses these limitations by applying analytics, ML, and automation to IT operations data. AIOps platforms ingest diverse data sources, normalize them, and use AI models to detect anomalies, correlate events, and predict failures.

This approach reduces Mean Time to Recovery (MTTR), minimizes service disruptions, and provides insights for capacity planning and performance optimization.

Core capabilities of AIOps

AIOps provides capabilities for proactive and automated IT operations.

Anomaly detection

Anomaly detection is a capability of AIOps. AIOps uses ML models to learn normal system behavior and identify deviations. These models analyze historical and streaming data such as logs, metrics, and traces to establish baselines for expected performance. When patterns deviate, such as CPU spikes, memory leaks, or network latency changes, the system flags these anomalies. This approach enables IT teams to intervene before service degradation occurs.

Event correlation

IT environments generate thousands of alerts daily, creating noise and alert fatigue for operations teams. AIOps addresses this issue through event correlation. AIOps uses algorithms to group related alerts and identify the root cause, converting multiple alerts into a single incident. For example, instead of treating 50 alerts from different microservices as separate problems, AIOps can correlate them to a single failing database node. This approach reduces alert volume and accelerates troubleshooting, improving MTTR.

Intelligent alert management

Alert management reduces alert volume by clustering similar alerts, suppressing non-actionable notifications, and ranking incidents by severity and potential disruption. This approach enables engineers to address critical issues.

Root cause analysis

Identifying the root cause of an issue in distributed infrastructure can be challenging. AIOps uses AI to map system dependencies and analyze event patterns across multiple layers of the IT stack. This approach identifies the source of a problem, whether it is a misconfigured load balancer, a failing microservice, or a network bottleneck. This approach accelerates root cause identification and reduces downtime.

Predictive analytics

AIOps uses predictive analytics to forecast infrastructure states. Using historical data and ML models, AIOps predicts issues such as disk capacity exhaustion or network congestion during peak loads. These insights support capacity planning and resource allocation. Predictive analytics supports resource scaling based on anticipated demand.

Automated remediation

Automation connects detection and resolution in AIOps. When anomalies or failures are identified, AIOps triggers workflows or scripts to resolve issues. Examples include restarting failed services, scaling resources during traffic surges, or applying configuration updates. Automated remediation reduces manual effort.

Architecture overview of AIOps

The AIOps architecture processes IT operations data, applies AI analytics, and automates remediation. It consists of three layers: data sources, the AI and ML pipeline, and the integration layer.

Figure 2-7 shows the AIOps architecture.

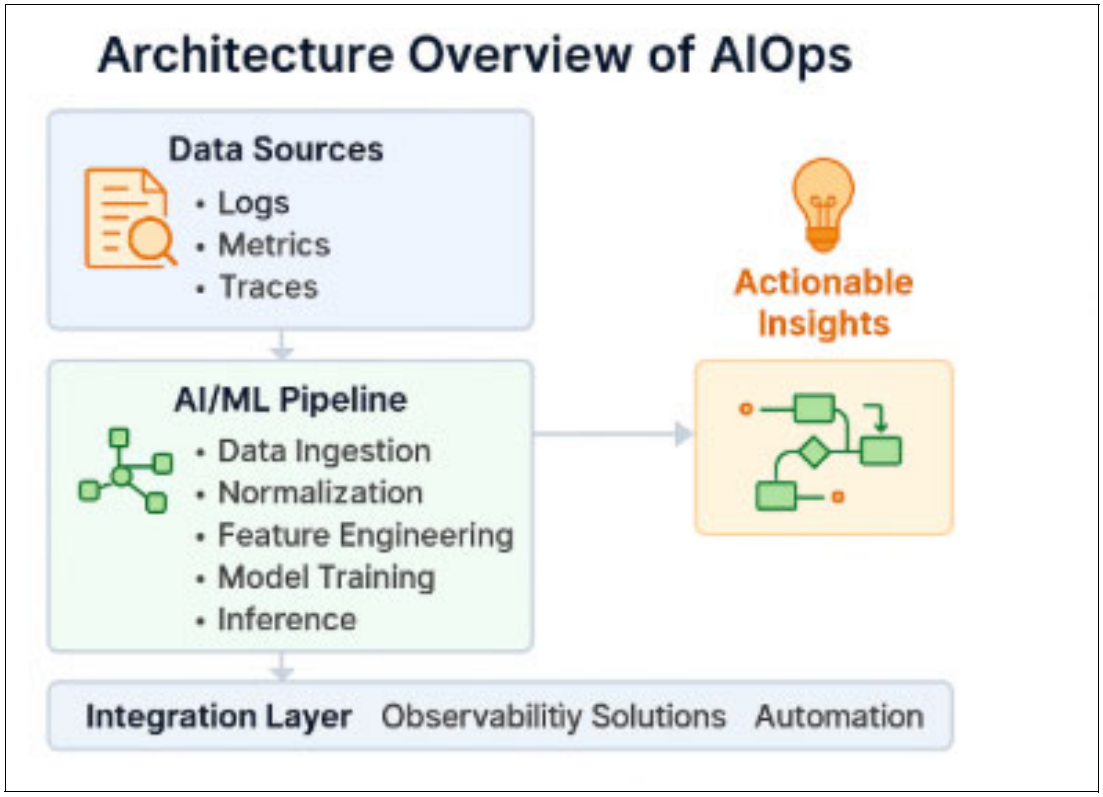


Figure 2-7 AIOps architecture

Data sources

IT environments generate data from multiple layers and components, including logs, metrics, traces, and alerts. Logs such as application, system, and security logs provide event details, and metrics capture resource usage such as CPU, memory, network throughput, and storage performance. Distributed tracing data from microservices and containerized workloads tracks transactions, and alerts from monitoring tools and observability platforms indicate issues. These data streams are unstructured or semi-structured and come from sources such as cloud platforms, on-premises systems, and edge devices. AIOps platforms ingest and normalize this data.

AI and ML pipeline

The AIOps data processing and ML pipeline includes several stages:

- Data ingestion

  Collecting data from multiple sources by using connectors and APIs
- Normalization

  Converting heterogeneous data into a unified format
- Feature engineering

  Extracting relevant features for model training
- Model training

  Building ML models for anomaly detection, event correlation, and predictive analytics
- Inference

  Applying trained models to real-time data streams to generate insights

AIOps platforms use technologies such as Apache Kafka for streaming, Apache Spark for distributed processing, and ML frameworks such as TensorFlow or PyTorch for model development and deployment.

Integration layer

AIOps integrates with IT management ecosystems, which include ITSM tools such as ServiceNow or Remedy for incident and change management; observability solutions such as Prometheus, Grafana, or OpenTelemetry for metrics and traces; and automation engines such as Ansible or Chef for remediation workflows. These integrations enable insights from AIOps to trigger automated actions.

End-to-end flow

The AIOps workflow includes these steps:

1. Data collection

   Logs, metrics, and traces are collected.
2. Processing and analysis

   AI and ML models detect anomalies, correlate events, and predict failures.
3. Insights

   Alerts are enriched with context and sent to ITSM tools for prioritization.
4. Automation

   Remediation scripts or workflows are triggered automatically.

Best practices

AIOps adoption requires planning that addresses technology, data, and business alignment. The following practices support AIOps implementation:

- Start with specific use cases.

  Focus on use cases such as reducing alert volume or accelerating incident resolution. This approach demonstrates value and supports broader adoption.

- Help ensure data readiness and governance.

  AIOps uses data from diverse sources. Establish data governance practices for accuracy, consistency, and security. This approach includes normalizing data formats, managing access controls, and maintaining compliance with regulatory requirements.

- Retrain models with feedback.

  ML models improve when they incorporate operational feedback. Implement processes that retrain models based on operational outcomes and system behaviors. This approach supports accurate anomaly detection, event correlation, and predictive analytics.

- Align AIOps with business objectives

  Align initiatives with business goals such as reducing downtime and optimizing costs. Define KPIs and metrics to measure impact.

2.3.2 AI DevOps

AI DevOps applies DevOps principles to the development, deployment, and maintenance of AI models and applications. It integrates data science with IT operations for AI solution delivery.

Evolution of AI DevOps

DevOps automates software delivery pipelines, enabling continuous integration and continuous deployment (CI/CD). AI introduces complexities such as model training, data versioning, and performance monitoring. AI DevOps extends these principles to include the following activities:

- Continuous training

  Updating models as new data becomes available

- Model versioning

  Tracking changes in model architecture and parameters

- Data governance

  Managing datasets for compliance, quality, and reproducibility.

Key components of AI DevOps

AI DevOps requires specialized practices beyond traditional DevOps. AI involves managing large datasets, monitoring model performance, and helping ensure compliance with ethical and regulatory standards. AI DevOps includes components for the lifecycle from data preparation and model training to deployment and governance. These components support workflow automation, transparency, and scaling AI solutions in hybrid environments.

Automated pipelines for AI

AI DevOps pipelines integrate data ingestion, feature engineering, model training, validation, and deployment. Tools such as Kubeflow, MLflow, and Red Hat OpenShift AI enable automation across these stages.

Monitoring and observability

AI systems require monitoring beyond uptime and latency. Metrics include model accuracy, drift detection, and fairness indicators. Monitoring supports model reliability in production.

Scalable infrastructure

AI workloads require compute resources. Platforms such as IBM Power11 provide integrated accelerators and hybrid cloud capabilities for training and inference.

Security and compliance

AI DevOps includes security throughout the process, from encrypted data pipelines to quantum-safe cryptography (QSC) for model storage. Compliance frameworks support compliance with regulations such as GDPR and industry-specific standards.

Collaboration and governance

AI DevOps supports collaboration among data scientists, developers, and operations teams. Governance tools manage model lifecycle, audit trails, and explainability.

Benefits of AI DevOps

AI DevOps provides benefits that include the following items:

- Deployment of AI models into production
- Reliability and scalability for applications
- Security and compliance for data
- Automated retraining and monitoring

Summary

AI DevOps supports AI adoption by providing automation, governance, and scalability for the AI lifecycle. AI DevOps supports deployment of AI services in hybrid environments.

2.3.3 Real-time inference

Real-time inference is the process of running an AI or ML model and generating predictions or insights as new data is received. Real-time inference delivers outputs with reduced latency for applications such as fraud detection, autonomous vehicles, and personalized recommendations.

Real-time inference applies to AI models as data is generated. This approach differs from the training process, which requires resources to build models. Organizations deploy pre-trained models, such as LLMs and ML models, in distributed environments near the data source.

Enterprises process data volumes in real time, and real-time inference enables insights and decisions as data arrives.

Real-time inference requires the following items:

- AI models that run with reduced latency
- Systems that handle concurrent requests
- Resilient infrastructure that does not depend on cloud connectivity

Real-time inference examples include the following items:

- Fraud detection in financial transactions

  Banks and payment processors use real-time inference to analyze transactions as they occur, flagging suspicious activity to prevent fraud.

- Autonomous vehicles

  Self-driving cars use real-time inference to process sensor data and make decisions for navigation and safety.

- Personalized recommendations

  Streaming platforms and e-commerce sites deliver product or content recommendations that are based on user behavior and context.

- Predictive maintenance in manufacturing

  IoT sensors on equipment feed data to AI models that predict failures or maintenance needs.

- Healthcare monitoring

  Wearable devices and hospital systems use real-time inference to detect anomalies in vital signs and alert medical staff.

- Cybersecurity threat detection

  AI models monitor network traffic and identify potential breaches or malware to protect enterprise systems.

Edge computing

Edge computing is a distributed IT architecture that processes data near where it is generated. Instead of sending data to centralized data centers, edge computing processes it locally. This approach reduces latency and improves bandwidth efficiency. Edge computing supports scenarios such as IoT, autonomous systems, and remote operations.

Edge computing is used to accomplish the following tasks:

- Reduce latency by avoiding data transfer to data centers.
- Reduce bandwidth costs and storage requirements.
- Support compliance with data governance and privacy requirements for local data processing.

Power11 provides memory bandwidth and MMA accelerator support for real-time inference workloads. Power11 reduces latency in edge environments and supports operations with limited network connectivity.

Power11 capabilities for edge computing

The Scale-Out Power11 models IBM Power S1124, IBM Power S1122, and IBM Power S1112 support real-time inference at the edge. These systems provide capabilities for distributed environments:

- Reduced latency and high throughput

  Power11 servers include a memory subsystem with DDR5 and Open Memory Interface (OMI) interfaces, increasing bandwidth compared to the previous generation. Each core includes IPC improvements and SMT8 for concurrent execution of multiple threads. This memory bandwidth and core design supports inference models.

- Integrated AI acceleration: MMA and Spyre

  Power11 integrates MMA on the CPU for dense linear algebra operations such as matrix multiplication, convolution, and attention kernels. This approach reduces latency per token or frame without external accelerators.

  For workloads with higher throughput requirements, such as multiple video or image streams, Power11 serves support the IBM Spyre Accelerator to extend compute capacity beyond a single socket. This approach uses MMA for small batch inference and the IBM Spyre Accelerator for parallel or large batch inference.

- Security, resiliency, and reliability

  Edge environments operate with limited connectivity to data centers. Power11 servers provide Secure Boot, memory encryption, and LPAR isolation. The platform supports zero planned downtime (ZPD), allowing firmware and OS updates without interrupting workloads.

Example edge computing architecture

Edge computing architecture includes three layers, as shown in Figure 2-8:

- Far edge

  End devices, sensors, and micro-CPUs near data sources that capture and filter data.

- Near edge

  Micro data centers with computing and storage for real-time inference. These systems run ML and LLM models with data caching and maintain connections to core data centers.

- Data center or core

  On-premises or cloud central systems with computing and storage for orchestration, disaster recovery, and data collection from edge systems.

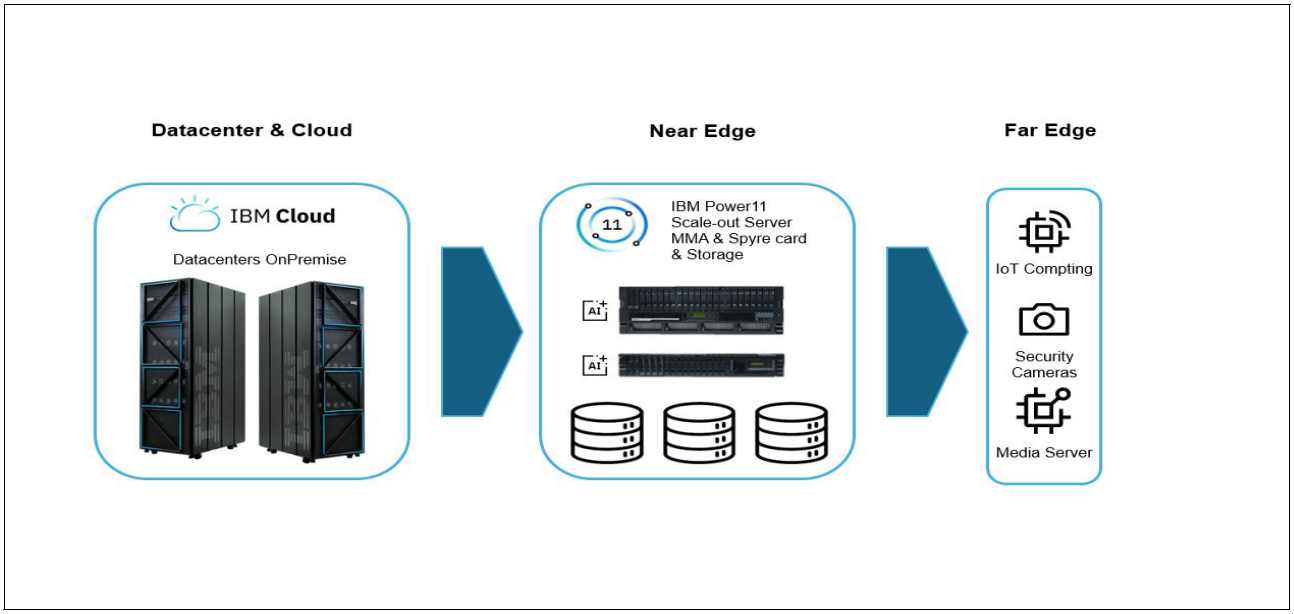


Figure 2-8 Example edge computing architecture

Data flow begins near the data source: devices at the edge capture signals and events, and micro-computers filter data to reduce volume while preserving context (timestamps, asset identity, and location). This layer uses reduced power and operates continuously, even with intermittent connectivity.

Data is routed to a nearby compute node (a micro data center with compute and I/O density) where real-time inference runs and multiple data streams are correlated. This layer balances proximity with compute density, reduces WAN traffic, and operates during outages by using persistent queues.

Higher data volumes and compliance requirements favor processing data near the source, transporting only summaries, embeddings, and events. Processing location decisions depend on target latency, data volume, local retention requirements, and compliance.

Inference and correlation run at the edge with local compute and storage. Training, historical analytics, and orchestration run at the core data center. This approach separates time-critical processing from analytical processing and supports operations in environments with variable connectivity.

Use cases

This section describes edge computing use cases across industries:

- Banking

  Here are two edge computing examples in banking that use Power11 servers:

  - For pre-authorization and anti-fraud at POSs and ATMs, authorization decisions must be made quickly to avoid transaction latency. Terminals operate as the far edge for capture and filtering. Inference runs at the near edge in branches or regional hubs to correlate transactions and risk signals. The core consolidates history, recalibrates models, and manages global rules.

    Power11 servers provide reduced latency inference with MMA for concurrent processing, LPAR isolation for compliance, and updates without interruption for continuous operation.

  - The near edge runs document classification pipelines, PII detection, field validation, risk rules, and LLM processing for intent and summary. The core consolidates metrics and retrains models with anonymized data.

    Power11 servers provide concurrent processing for multiple streams, supporting LLM inference and validations. LPAR isolation protects data by branch, and ZPD supports operations during maintenance.

- Retail

  Here are two edge computing examples in retail that use Power11 servers:

  - In shelf availability, cameras capture images, and the store runs image analysis to detect stock shortages and planogram deviations. The core consolidates multi-store KPIs and retrains models for inference at each store.

    Power11 servers support inference with MMA for camera footage analysis in stores.

  - In stores where the self-checkout model is implemented, sensors and cameras deliver signals for processing. Power11 servers with MMA process data to detect risks such as concealment or non-payment.

    Power11 servers provide reduced latency for payment processing. An LPAR isolates failures to maintain store availability.

Real-time inference processes data where it is generated to meet latency requirements, reduce network costs, and maintain control of sensitive data. Edge computing deploys processing near endpoints such as sensors, cameras, and IoT devices to filter data and make decisions without transferring data to data centers. This approach provides predictable performance without network limitations. Local processing maintains data privacy.

Power11 provides reduced latency with on-chip MMA for small language models (SLMs) or ML models and resilience for operating remote sites. Power11 scale-out configurations provide computing per watt and parallelism with SMT8 to meet SLAs.

3

On-chip acceleration

This chapter describes on-chip acceleration technologies in IBM POWER processors and the capabilities in IBM Power11. It describes how IBM integrated hardware accelerators for AI, analytics, and high-throughput workloads.

The chapter describes how these capabilities are exposed through open software stacks, including ONNX, PyTorch, and TensorFlow, and how they integrate with IBM's AI ecosystem, including IBM watsonx® and Red Hat OpenShift.

This chapter describes the following topics:

- Overview of AI acceleration features
- AI acceleration on IBM Power11: MMA and SIMD optimization
- Training of machine learning models

© Copyright IBM Corp. 2026.

3.1 Overview of AI acceleration features

IBM Power11 provides AI acceleration for enterprise workloads. Traditional AI deployments rely on external graphics processing units (GPUs) for training and inference, which can introduce latency, increase costs, and complicate data governance. Power11 integrates acceleration technologies into its architecture for AI processing with reduced latency and improved energy efficiency.

The Matrix Math Accelerator (MMA) accelerates matrix operations for machine learning (ML) and deep learning (DL) in hardware. MMA reduces computational processing for tasks such as neural network training and inference.

Power11 runs AI workloads where data is, reducing dependency on external GPU clusters and supporting hybrid deployments across edge, on-premises, and cloud environments. With support for mixed-precision operations such as INT8 and FP16, Power11 supports training and inference for AI applications.

3.2 AI acceleration on IBM Power11: MMA and SIMD optimization

Power11 provides AI acceleration capabilities. The MMA accelerates matrix operations for ML and DL in hardware. MMA supports matrix multiplication operations, reducing computational processing for neural network training and inference.

Starting with IBM POWER7, the Vector Scalar eXtension (VSX) capabilities were introduced to accelerate computation. VSX supports vector and scalar binary floating-point operations, extending and unifying the 32 floating-point registers (FPRs) of the Power ISA to 128 bits. These operations are combined with the existing thirty-two 128-bit Vector Multimedia eXtension (VMX) registers to create a single register file that is known as the Vector Scalar Register (VSR).

Figure 3-1 shows the construction of the VSR to enable single instruction, multiple data (SIMD) instructions.

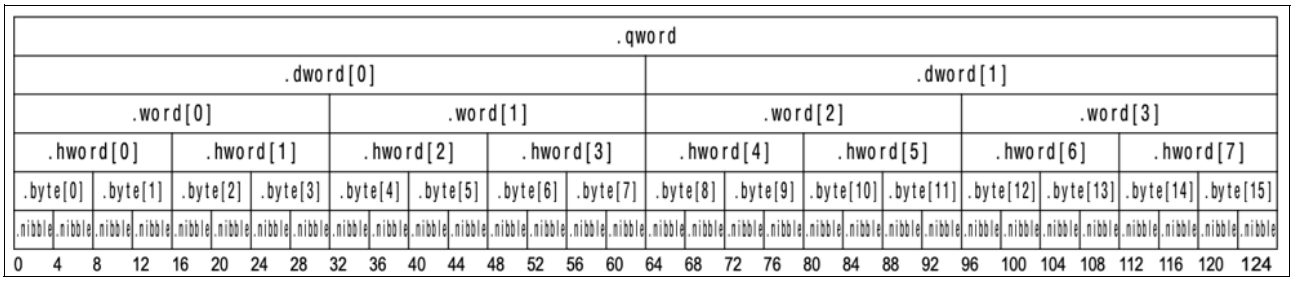


Figure 3-1 Vector Scalar Register

Operations run in parallel across all elements in a VSR, supporting SIMD computations. Depending on the data type (byte, halfword, or word), elements are interpreted as signed, unsigned, single-precision, or double-precision. VSX supports operations such as Vector Splat, Vector Permute, and Vector Arithmetic instructions for AI workloads.

For AI use cases, computations such as vector sum and vector multiply-add operations support linear algebra routines in DL frameworks. These operations support tensor operations on Power11 for inference and training.

Example 3-1 shows how VSX instructions accelerate an AI computation by calculating the squared Euclidean distance between two vectors.

Example 3-1 Example code

```
for (i = 0; i < d; i++) {
    const float tmp = x[i] - y[i];
    res += tmp * tmp;
}
```

Compiled for Power11 with VSX optimization, this loop produces the instructions that are shown in Example 3-2.

Example 3-2

```
lvx         v2, r24, r29        // Load vector
xvsubsp     vs46, vs28, vs51    // Subtract vectors (single precision)
xvmaddasp   vs1, vs13, vs13     // Multiply-add for accumulation
```

These SIMD instructions run multiple operations in parallel, improving performance compared to scalar execution.

3.3 Training of machine learning models

Training is the phase of the ML workflow where models learn patterns, structures, or relationships in data. The goal is to adjust model parameters so the model can generalize from training data to unseen examples. This process involves minimizing a loss function through optimization techniques such as stochastic gradient descent (SGD) or its variants.

Depending on model architecture, dataset size, and computational resources, training can range from seconds on personal computers to weeks on distributed clusters with hardware accelerators. Training scales from small models on personal computers to LLMs that require GPU clusters.

3.3.1 The training process

The training process includes these stages:

1. Data preparation

   Raw data is collected, cleaned, and transformed into numerical representation for model input. This approach might include normalization, feature extraction, or tokenization.

2. Model initialization

   Model parameters (weights and biases) are initialized (often randomly) to begin the learning process. Initialization affects convergence efficiency.

3. Forward pass

   Input data is propagated through the model to produce output (prediction). This output is compared to the true label by using a loss function (for example, mean squared error for regression, or cross-entropy for classification).

4. Backward pass (backpropagation)

   The model computes gradients of the loss function regarding its parameters. These gradients indicate how to update parameters to minimize loss.

5. Parameter update

   Optimization algorithms such as SGD, Adam, or RMSProp update model parameters iteratively. The process repeats across epochs until convergence.

3.3.2 Computational complexity and hardware requirements

The computational demands of training ML models vary depending on model type, model size, and data volume. Simple models can be trained on CPUs, but more complex architectures, such as DL models, require hardware such as GPUs or Tensor Processing Units (TPUs) to achieve training times.

The following sections describe model types, their complexity, and the hardware resources for training.

Linear and logistic regression

Linear and logistic regression models are simple ML models. They learn parameters to fit relationships between input features and target output.

- Training Effort

  Low. Completes in seconds to minutes.

- Hardware requirements

  Standard CPUs are sufficient for large datasets. Training can be performed on a laptop or single server.

- Scalability

  These models scale to large datasets but do not capture complex nonlinear relationships.

Decision trees and ensemble models

Decision trees divide data into branches based on feature values. Ensemble methods such as Random Forests or Gradient Boosted Trees build multiple trees to improve accuracy. These models handle more complex patterns than linear models and require more computation and memory.

- Training Effort

  Moderate. Seconds to hours depending on the number and depth of trees.

- Hardware requirements

  Multi-core CPUs are sufficient. Training uses parallel processing. GPUs are not required.

- Scalability

  Ensemble models support moderate to large datasets but can be memory-intensive when building hundreds or thousands of trees.

Support Vector Machines

Support Vector Machines (SVMs) can model complex decision boundaries by using nonlinear kernels. They become computationally expensive as the data size grows.

- Training effort

  High for large datasets. Can take hours or days.

- Hardware requirements

  CPUs with large memory. GPUs are not commonly used.

- Scalability

  Suited for smaller datasets.

Neural networks and deep learning models

Neural networks, such as deep architectures including CNNs and Transformers (used in LLMs), are more computationally demanding. They can contain millions or billions of parameters and require large datasets.

- Training effort

  High. Can take hours to weeks depending on model size and data volume.

- Hardware requirements
  - GPUs perform matrix operations in parallel for DL.
  - Distributed training across multiple GPUs or nodes supports large models.

- Scalability

  Scales but requires hardware and optimization techniques (for example, mixed-precision training, or data parallelism).

3.3.3 Hardware and energy considerations

Hardware choice affects training speed and energy consumption:

- CPUs support small to medium-scale models.
- GPUs accelerate training of neural networks for computer vision and natural language processing (NLP).
- Large models consume megawatt-hours of electricity during training.
- Hardware optimization, model compression, and optimization techniques support efficient AI development.

3.3.4 Summary

The computational complexity of training ML models varies across model types. Simple models such as linear regressors can be trained in seconds on standard hardware, but deep neural networks might require thousands of GPU hours.

Understanding these computational requirements supports system design for AI infrastructure. Training efficiency depends on algorithmic optimization, hardware selection, and parallelization strategies.

4

IBM Spyre Accelerator

The IBM Spyre PCIe Gen5 x16 AI Accelerator Adapter (IBM Spyre Accelerator) supports AI workloads, including generative AI. Each IBM Spyre Accelerator module is a PCIe card with 32 accelerator cores. Co-developed by IBM Research and IBM Infrastructure, the Spyre architecture enables data flow between compute engines, reducing memory transfers.

IBM Spyre Accelerator supports low-precision numeric formats that reduce memory footprint and power usage for AI inference tasks. This approach supports deploying AI models in enterprise environments.

With IBM Power servers, the IBM Spyre Accelerator offloads computations from CPUs. The IBM Spyre Accelerator design enables data flow between processor and memory for AI workloads.

This chapter describes the following topics:

- Introducing the IBM Spyre Accelerator
- Turnkey AI with Spyre
- IBM Spyre Accelerator design
- Power11 implementation of IBM Spyre Accelerator
- Examples and supported workloads
- Understanding LLM performance
- Hybrid compute: Spyre and CPU
- The IBM AI acceleration roadmap

© Copyright IBM Corp. 2026.

4.1 Introducing the IBM Spyre Accelerator

The IBM Spyre Accelerator supports AI workloads. Data and instructions are transferred between processor and memory in computing systems. The IBM Spyre Accelerator supports matrix and vector computations for AI.

This architecture enables data transfer between compute engines. The IBM Spyre Accelerator supports inference and training tasks.

Graphics processing units (GPUs) are used for AI workloads but present challenges:

- Latency

  GPU systems rely on external interconnects and discrete memory pools, which increase latency when moving data between compute units and system memory.

- Cost and complexity

  Deploying GPUs requires infrastructure including cooling, power delivery systems, and networking to support data movement between accelerator nodes.

To address these limitations, The IBM Spyre Accelerator integrates into the IBM Power Architecture and uses internal interconnects.

Figure 4-1 shows the IBM Spyre Accelerator PCIe attached card.

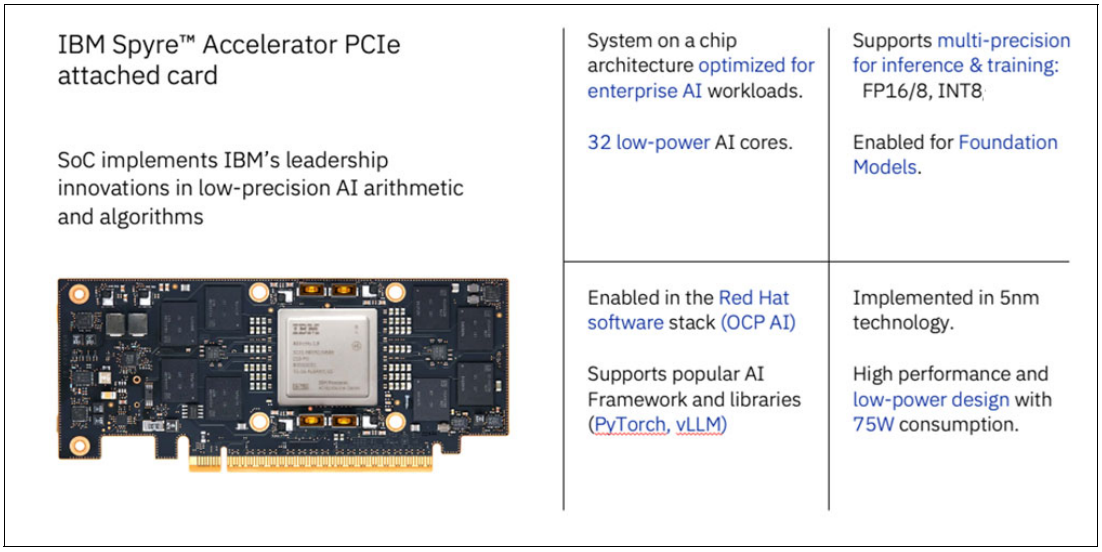


Figure 4-1 IBM Spyre Accelerator PCIe attached card

4.1.1 Role in AI workload orchestration and optimization

Off-chip accelerators such as the IBM Spyre Accelerator support scaling and optimization of AI workloads. Integration into system architecture provides advantages:

- Specialized compute for AI tasks

  Off-chip accelerators support operations such as matrix multiplications, convolutions, and data movement. These accelerators enable the following items:

  - Inference and training cycles.
  - Power consumption per operation.
  - Throughput for parallelized tasks.

- Workload offloading and balancing

  Offloading operations such as model inference, preprocessing, or feature extraction to accelerators provides the following benefits:

  - Frees CPU resources for orchestration, control logic, and services.
  - Enables workload distribution across compute nodes.

- Orchestration across heterogeneous resources

  AI orchestration frameworks such as Red Hat OpenShift AI route workloads to compute resources based on the following aspects:

  - Model size, latency requirements, and data locality.
  - Policies for energy efficiency, throughput, or cost.
  - Colocation of AI inference with transactional or database workloads on the same server.

- Scalability and modularity

  Off-chip accelerators can be clustered and scaled independently of the host CPU, which leads to the following benefits:

  - Multiple IBM Spyre Accelerators in an I/O drawer can scale AI compute.
  - Supports deployment models from edge to cloud.

4.2 Turnkey AI with Spyre

AI Services for IBM Power provide a turnkey framework for deploying AI workloads on Power11 servers. As part of the IBM Open-Source AI Foundation for Power, AI services provide pre-built, containerized components for model inference, retrieval-augmented generation (RAG), embedding pipelines, and reranking tasks in Power environments. These services run on the Power11 platform and support the IBM Spyre Accelerator.

4.2.1 Architecture and design principles

AI services use a modular, open-source architecture for deployment in Power data centers. Each service is a self-contained deployment unit that includes an AI model, runtime configuration, and API interface. This architecture supports integration of model capabilities in enterprise environments.

Key design principles include the following items:

- Turnkey operation

  AI services require minimal setup and integrate with the Red Hat AI Inference Server and the IBM Spyre Accelerator.

- Open and extensible

  Built on open-source foundations with support for IBM Granite® models, embedding models, and third-party inference frameworks.

- Power11 support

  Each service uses Power11 vector processing, high memory bandwidth, and hardware-assisted AI features.

- Accelerator support

  Integration with the IBM Spyre Accelerator enables services to route inference workloads and use FP16 execution, continuous batching, and multi-card scaling.

4.2.2 Integration with IBM Spyre Accelerator for Power

AI services integrate with the IBM Spyre Enablement Stack for Power, including drivers, vLLM-based back ends, and inference run times. This integration provides the following benefits:

- Accelerator access through Virtual Function I/O (VFIO) and zero-copy data movement
- Continuous batching support for LLM workloads
- Multi-card scaling for RAG pipelines and large-context inferencing
- Model caching and runtime optimizations

4.2.3 Supported AI use cases

AI services provide implementations for enterprise AI tasks. These services use Granite and open-source models for functions:

- LLM inference

  Text generation, summarization, and conversational tasks by using Granite instruction models.

- Embedding generation

  Vectorization of documents for semantic search, knowledge-base indexing, and RAG pipelines.

- RAG pipelines

  RAG by using embedding services, vector databases, and LLM-based answer synthesis.

- Reranking

  Ranking of retrieved results by using reranking models for search and retrieval systems.

- Entity extraction

  Structured data extraction from unstructured sources by using transformer-based natural language processing (NLP) models.

Table 4-1 shows the supported use cases.

Table 4-1 Use cases supported at initial GA.

Use case	Model name	Batch size	Maximum input context size	Maximum output context size	Number of cards per container
Entity extraction	Granite3.3-8b-instruct	16	3 K	3 K	1
RAG embedding	Granite-Embedding-125m-English Granite-Embedding-278m-multilingual	Up to 256	512	Vector of size 768	1
RAG embedding	Granite-Embedding-30m-English Granite-embedding-107m-multilingual	Up to 256	512	Vector of size 384	1
RAG inferencing	Granite3.3-8b-instruct	32	32 K (batch * context equals less than 128 K)	32 K (batch * context equals less than 128 K)	4
Reranker	beg-reranker-v2-m3	Up to 4	8 K	8 K	1

Each AI service provides APIs, configuration examples, and deployment patterns for workloads supported by the IBM Spyre Accelerator and Red Hat AI Inference Server.

4.2.4 Deployment and operations

AI services are containerized and can be deployed as Podman containers for development, testing, and lightweight environments, or as Podman Quadlets for production scenarios. They integrate with the Red Hat AI Inference Server, which supports model downloads, environment preparation, and resource delegation. Each service includes runtime logic to support operations across multiple IBM Spyre Accelerator cards and monitoring through the Spyre metrics infrastructure. Services use **`systemd`**, **`journald`**, and Podman tools for logs and diagnostics and apply SELinux and system-level security controls consistent with Red Hat practices.

AI services provide a turnkey approach for AI deployment. Organizations can deploy AI pipelines with minimal configuration. This turnkey capability includes the following items:

- Integrated models and serving run times
- Defined API interfaces for inference and data processing.
- Compatibility with governance, security, and observability frameworks.
- Automated scaling on Power11 infrastructure.

Enterprises can embed AI into business processes and use a Power11 infrastructure for inferencing. Organizations can run workloads such as RAG where data is.

For more information about AI services from the IBM Open-Source AI Foundation for Power, see IBM Open-Source AI Foundation for Power.

4.3 IBM Spyre Accelerator design

The IBM Spyre Accelerator is a PCIe Generation 5 x16 adapter for AI workloads. The adapter uses a Samsung 5LPE chip with 32 accelerator cores and 128 GB of LPDDR5 memory. The IBM Spyre Accelerator is supported in IBM Z17 and IBM Power11 servers.

The IBM Spyre Accelerator provides compute capabilities in a 75 W envelope. Each IBM Spyre Accelerator includes 34 accelerator cores, of which 32 are active. The cores deliver over 300 trillions of operations per second (TOPS)[1]. The IBM Spyre Accelerator supports numeric formats for AI inference workloads.

Each accelerator core includes two corelets that are coupled to an L3 scratchpad memory of 2 MB. Each corelet has an 8×8 Processing Tile (PT) array of floating-point processors for AI and deep learning (DL) workloads by using precisions such as FP8 and FP16.

Matrices are divided into smaller chunks that are called *tiles*, which are stored in the PT array and loaded into memory slices within the core. The PT array runs Multiply And Accumulate operations on these tiles. The Processing Elements (PE) and the Special Function PE (SFP) array handle math functions for AI models, such as activations, normalization layers, and other non-matrix-multiply operations. A 128-byte data ring interconnects all accelerator cores with LPDDR5 memory for tile data movement across the subsystem[2].

[1] https://www.nextplatform.com/2024/08/27/ibm-shows-off-next-gen-ai-acceleration-on-chip-dpu-for-big-iron/

[2] https://hc2024.hotchips.org/assets/program/conference/day1/04_HC2024.IBM.CBerry.final.pdf

Figure 4-2 shows this architecture.

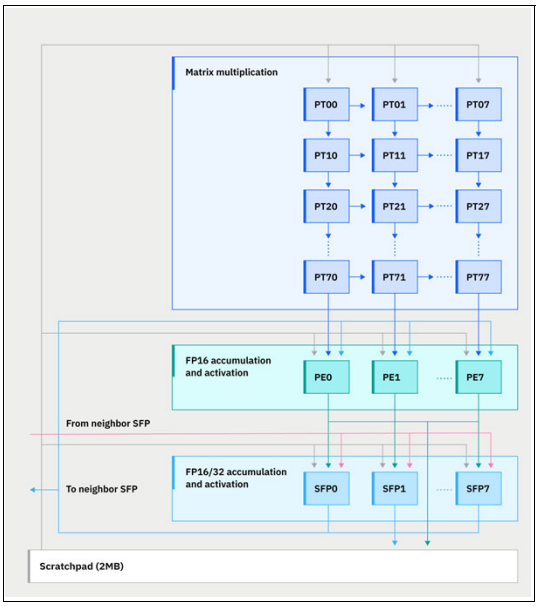


Figure 4-2 IBM Spyre Accelerator architecture

With 128 GB of LPDDR5 memory, the IBM Spyre Accelerator supports LLM inference. The software stack is compatible with AI frameworks such as PyTorch and vLLM.

The IBM Spyre Accelerator supports up to eight cards clustered within a single I/O drawer, providing 256 accelerator cores and 1 TB of memory. This configuration provides 1.6 TBps memory bandwidth. The software stack supports model sharding across multiple adapters for larger model sizes. For example, encoder models can run on a single IBM Spyre Accelerator, and decoder models such as Granite-3.3-8B-Instruct can use four IBM Spyre Accelerators.

4.4 Power11 implementation of IBM Spyre Accelerator

The IBM Spyre Accelerator supports Power11 servers by providing acceleration for AI workloads. Using virtualization and workload consolidation features of Power servers, applications that use the IBM Spyre Accelerator can be collocated with services such as databases and transactional systems on the same server.

Figure 4-3 on page 57 shows the architecture and integration of the IBM Spyre Accelerator adapters within the Power server environment. Eight IBM Spyre Accelerator adapters are in a dedicated I/O drawer that is connected to the Power server. Communication between the Power server and the IBM Spyre Accelerator adapters uses a dedicated Linux logical partition (LPAR).

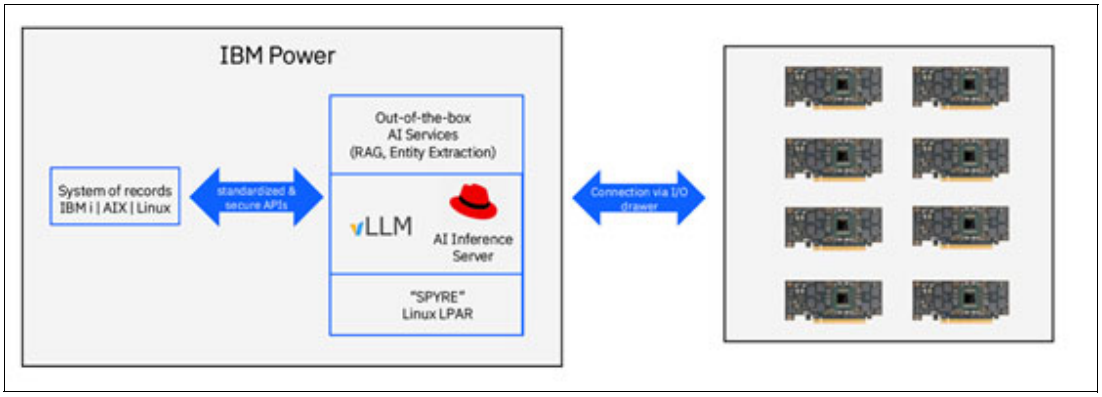


Figure 4-3 IBM Spyre Accelerator implementation

The Red Hat AI Inference Server is software that is required to use the IBM Spyre Accelerator with AI software stacks on Power servers. It provides components to integrate the IBM Spyre Accelerator into AI workflows. Built on technologies such as vLLM and model compression, Red Hat AI Inference Server supports deployment of LLMs in hybrid cloud environments. On Power servers, it includes HA, security features, and hardware-accelerated inference.

Red Hat AI Inference Server includes the following features:

- A vLLM inference server with a PyTorch back end for the IBM Spyre Accelerator
- Runtime libraries for execution across accelerator cores
- Device driver for communication between the host system and the IBM Spyre Accelerator
- Firmware for the IBM Spyre Accelerator
- Precompiled AI models for the IBM Spyre Accelerator

vLLM provides an OpenAI-compatible API. Applications that are built for the OpenAI API can run on vLLM, enabling deployment of OpenAI cloud applications on Power servers with the IBM Spyre Accelerator.

To move an application from OpenAI cloud to the vLLM server running on-premises, change the client initialization with the new `base_url`. Prompts, message handling, and model calls remain the same.

Figure 4-4 shows the original code.

```python
from openai import OpenAI

# Create client for OpenAI cloud API
client = OpenAI(api_key="YOUR_OPENAI_API_KEY")

# Example: Chat completion
response = client.chat.completions.create(
    model="gpt-4",
    messages=[
        {"role": "user", "content": "Summarize the key benefits of vLLM."}
    ],
)

print(response.choices[0].message.content)
```

Figure 4-4 Original API call

Figure 4-5 shows the changed code pointing to the Spyre interface.

```python
from openai import OpenAI

# Create client pointing to vLLM server running on-prem
client = OpenAI(base_url="http://ip-of-spyre-lpar:8000/v1")  # vLLM endpoin

# Example: Chat completion
response = client.chat.completions.create(
    model="gpt-4",
    messages=[
        {"role": "user", "content": "Summarize the key benefits of vLLM."}
    ],
)

print(response.choices[0].message.content)
```

Figure 4-5 API call to the Spyre interface

These components support AI inference deployment on Power platforms. The Red Hat AI Inference Server is updated regularly with drivers, firmware, AI models, and components.

Prerequisites and support

A Spyre inference environment consists of a Power11 host system with eight or more IBM Spyre Accelerators that are connected through an ENZ0 PCIe Gen 4 Expansion I/O drawer. The host system runs Red Hat Enterprise Linux (RHEL) 9.6 or later or Red Hat OpenShift and includes the following software components:

- Red Hat AI Inference Server
- IBM Open-Source AI Foundation for Power

With this configuration, users can run inference workloads on the IBM Spyre Accelerator.

4.4.1 Physical implementation

The IBM Spyre Accelerator is supported in ENZ0 PCIe Gen 4 Expansion Drawers that are configured with ENZA fanout modules.

Here are some IBM Spyre Accelerator design concepts:

- Built by using 5-nm fabrication, the IBM Spyre Accelerator operates at 75 W.
- AI compute.
  - 32 AI cores for parallel processing
  - Multi-precision support for inference
- Foundation model support.

  Integrates with Red Hat OpenShift AI for foundation model deployment.
- Framework compatibility.

  Supports AI frameworks and libraries such as PyTorch and vLLM.

Table 4-2 shows the specifications of the IBM Spyre AI Accelerator adapter.

Table 4-2 Specifications of the IBM Spyre AI Accelerator adapter

Description	Item
Adapter FRU number	03PN188.
I/O bus architecture	PCIe Gen5 x16.

Description	Item
Operating voltage	3.3 V and 12 V.
Form factor	Full-height, half-length.
Maximum supported adapters	Up to 4 adapters per ENZA fanout module. Up to 8 adapters total per ENZ0 PCIe Gen 4 Expansion Drawer.

The dedicated, enterprise-grade AI acceleration chip is on a 75 W PCIe adapter with 128 GB of LPDDR5 memory to support LLMs on Power servers.

Figure 4-6 shows the IBM Spyre Accelerator.

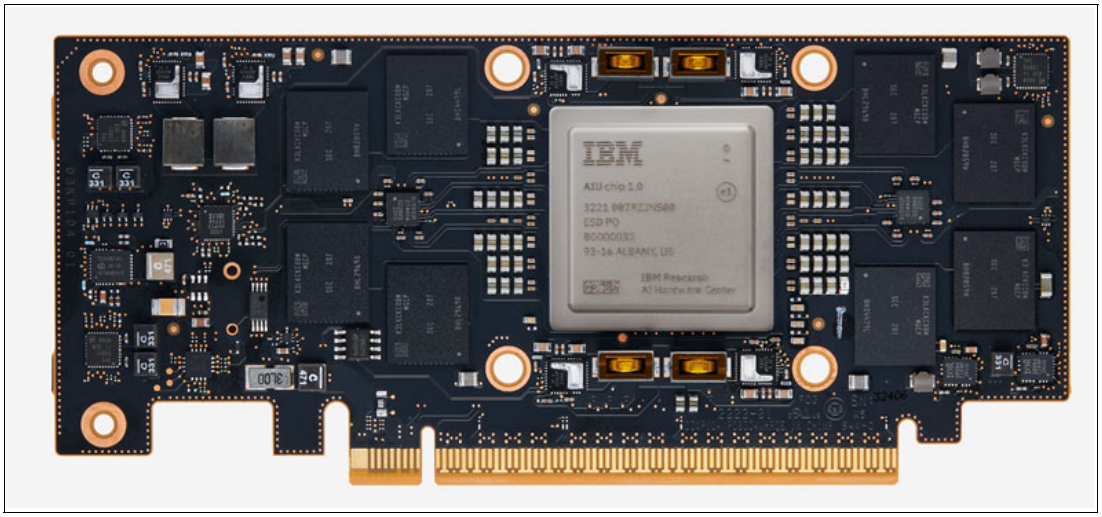


Figure 4-6 IBM Spyre Accelerator

A single IBM Spyre Accelerator provides limited compute capacity for enterprise AI workloads. IBM uses ENZ0 PCIe Gen 4 I/O Expansion Drawer technology with eight IBM Spyre Accelerator adapters to form a cluster. The firmware on these adapters manages workload distribution and coordinates data transfers across the cluster. This cluster appears as a single compute engine, with 1 TB of on-adapter memory and 1.6 TBps of memory bandwidth.

Figure 4-7 shows an IBM Spyre Accelerator cluster.

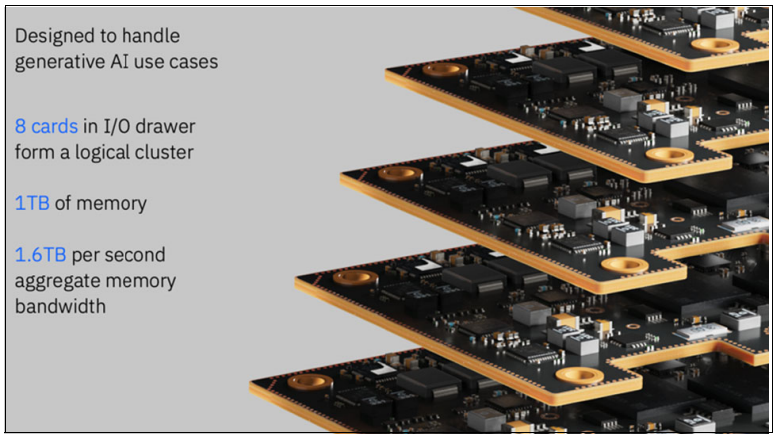


Figure 4-7 IBM Spyre Accelerator cluster in an expansion drawer

4.4.2 Expansion drawer support

Expansion drawers with IBM Spyre Accelerator adapters support Power11 servers. The number of supported drawers varies by system model. At GA, the IBM Spyre Accelerator configuration includes eight adapters in a PCIe Gen 4 I/O Expansion Drawer.

Table 4-3 shows the supported Power11 servers.

Table 4-3 Supported Power11 servers

Server model	Number of drawers that are supported	Number of adapters that are supported
IBM Power L1122	1	8
IBM Power L1124	1	8
IBM Power S1122	1	8
IBM Power S1124	1	8
IBM Power E1150	1	8
IBM Power E1180	1 or 2	8 or 16

The configuration requires extra power and cooling. Components such as power supplies and fanout modules are integrated into the expansion drawer for operation with multiple IBM Spyre Accelerator adapters.

The supported software and hardware at GA include the following items:

- Firmware: FW1110.10
- Hardware Management Console (HMC): Version HMC1111
- Operating system: RHEL 9.6
- Spyre Software Stack Container
- Red Hat OpenShift AI (technology preview: Q4 2025, General Availability: Q1 2026)

ENZ0 PCIe Gen 4 I/O Expansion Drawer

The ENZ0 PCIe Gen 4 I/O Expansion Drawer supports PCIe adapters in Power servers. It supports up to eight adapters by using two 6-slot fanout modules with Feature Code ENZA or Feature Code ENZF.

Adapter placement and slot priorities

Figure 4-8 shows the rear view of the ENZ0 PCIe Gen 4 Expansion Drawer and the location codes for adapter slots within the PCIe Gen 4 6-slot fanout module.

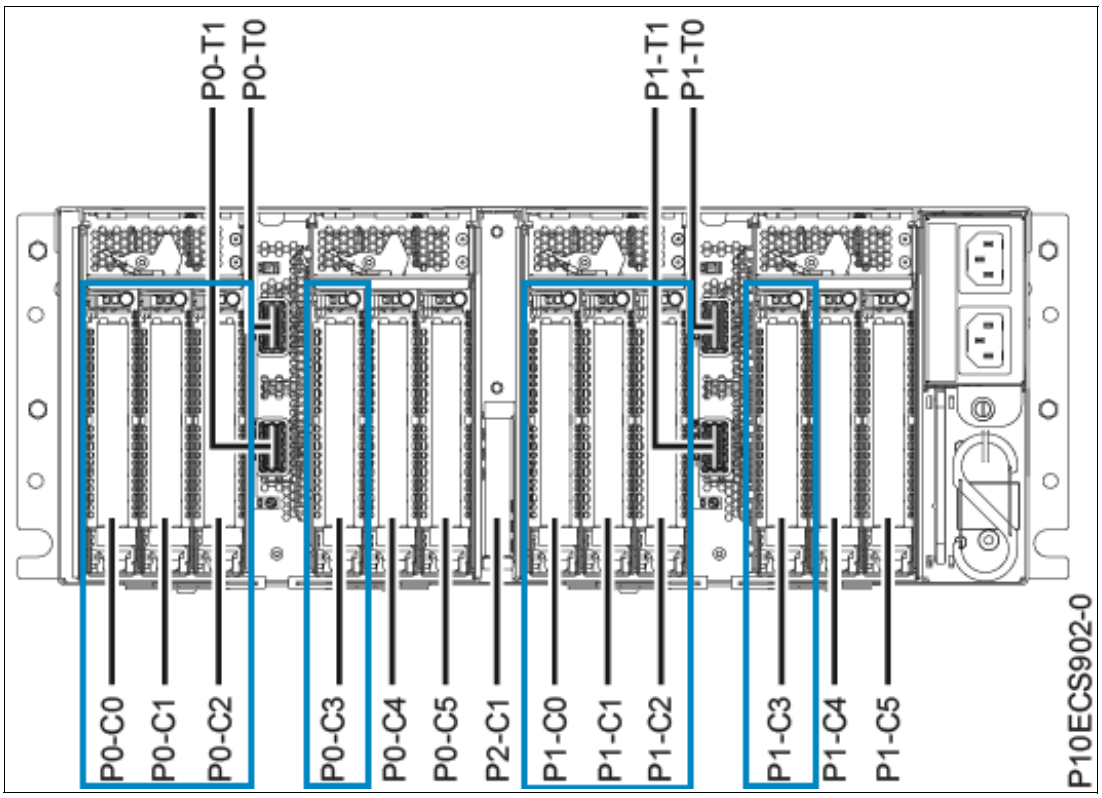


Figure 4-8 The rear view of the ENZ0 PCIe Gen 4 Expansion Drawer

For IBM Spyre Accelerator installation, use the following slot priorities:

- Slot priorities
  - P0-C0 through P0-C3
  - P1-C0 through P1-C3

  These slots must be populated when installing IBM Spyre Accelerator adapters.
- Fanout module compatibility
  - ENZF fanout modules support adapter types except for Feature Code ECSE
  - ENZA fanout modules support Feature Code ECSE only

Table 4-4 illustrates the slot priorities and maximum supported IBM Spyre Accelerator adapters in the ENZ0 PCIe Gen4 Expansion Drawer.

Table 4-4 Slot priorities and maximum supported IBM Spyre Accelerator adapters in ENZ0 PCIe Gen4 Expansion Drawer

Feature Code[a]	Description	ENZ0 PCIe4 expansion drawer	
		Slot priorities[b]	Maximum number of adapters supported[c]
ECSE	PCIe5 x16 AI Accelerator adapter (Feature Code ECSE and CCIN 2E1F. Adapter part number 03PN188.	P0-C0 through P0-C3 and P1-C0 through P1-C3 must be fully populated.	4.

a. With ENZA fanout modules, the ENZ0 PCIe Gen 4 Expansion Drawer supports Feature Code ECSE only.
b. The slot priority sequence applies to the ENZ0 PCIe Gen 4 Expansion Drawer with two PCIe Gen 4 6-slot fanout modules.
c. Maximum adapters per PCIe Gen 4 6-slot fanout module.

Note: When the PCIe Gen 5 x16 AI Accelerator Adapter (Feature Code ECSE and CCIN 2E1F) is installed in the ENZ0 PCIe Gen 4 I/O Expansion Drawer, the adapter operates at PCIe Gen 4 speeds.

Limitations and requirements

Limitations and requirements for the IBM Spyre Accelerator and ENZA fanout module include the following items:

- Feature Code ENZA (PCIe Gen 4 FOM) cannot be mixed with Feature Code ENZF in Feature Code ENZ0 Expansion Drawer.
- Feature Code ENZA is required to enable the IBM Spyre Accelerator (Feature Code ECSE).
- Feature Code ENZA supports IBM Spyre Accelerator (Feature Code ECSE) only; no other adapters are supported.
- Each Feature Code ENZA supports up to four IBM Spyre Accelerator adapters (Feature Code ECSE).
- Feature Code ENZ0 Expansion Drawer has a distinct CCIN for accelerator configuration.
- Cable card attachment for Feature Code ECSE in ENZ0 requires x16 system slots.
- Field upgrades for accelerator functions in ENZ0 are not supported.
- ENZ0 accelerator configuration is not supported by eSCM processor modules with 4-core or 10-core configurations on the Power S1122 server.
- ENZ0 accelerator configuration is not supported on systems with a single 16-core processor module on the Power S1124 server.
- 1 TB of memory is required when ordering the IBM Spyre Accelerator.

Installing an IBM Spyre Accelerator in an ENZA fanout module

To install an IBM Spyre Accelerator into an ENZA fanout module within an ENZ0 PCIe Gen 4 Expansion Drawer, complete the following steps:

1. Insert the adapter: With the tail stock clamp open, insert the adapter into the tail stock retaining channel.
2. Secure the adapter: Rotate the adapter upward and close the tail stock clamp, as shown in Figure 4-9 on page 63.

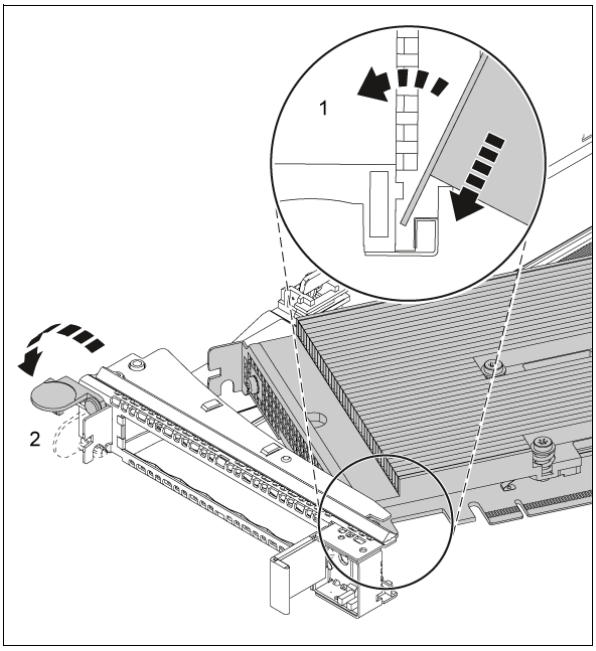


Figure 4-9 Placing the ECSE adapter in the cassette

3. Lock the retainers: Slide the adapter retainers toward the adapter, parallel to the cassette. Lock the top retainers by rotating them to a vertical position.
4. Adjust the clip assembly: Slide the double clip assembly until it aligns flush with the adapter's tabbed feature, then rotate the tail stock clamp to the closed position.
5. Verify retainer clips: Ensure that all retainer clips are in the locked (vertical) position. Clips in the horizontal position are unlocked. For more information, see Figure 4-10.

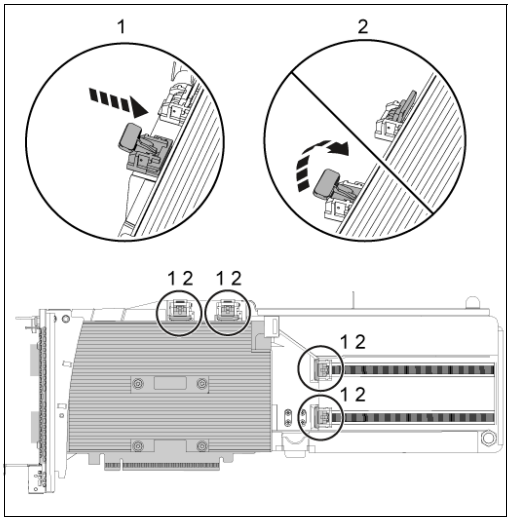


Figure 4-10 Locking the retainer clips

Verifying the link speed

To confirm the PCIe link speed and device information, run the command that is shown in Example 4-1.

Example 4-1 Validating the configuration of the IBM Spyre Accelerator

```
# lspci -vvv -d 1014:06a7
182:60:00.0 Processing accelerators [1200]: IBM Spyre Accelerator [1014:06a7] (rev 02)
   Physical Slot: U5B61.001.WZS09NJ-P1-C1
   Device tree node: /sys/firmware/devicetree/base/pci@800000020000182/pci1014,6a7@0
   Control: I/O- Mem+ BusMaster- SpecCycle- MemWINV- VGASnoop- ParErr+ Stepping- SERR+ FastB2B-
DisINTx-
   Status: Cap+ 66MHz- UDF- FastB2B- ParErr- DEVSEL=fast >TAbort- <TAbort- <MAbort- >SERR-
<PERR- INTx-
   Interrupt: pin A routed to IRQ 0
   NUMA node: 0
   IOMMU group: 0
   Region 0: Memory at 41082000000 (64-bit, prefetchable) [size=4M]
   Region 2: Memory at 41000000000 (64-bit, prefetchable) [size=2G]
   Region 4: Memory at 41080000000 (64-bit, prefetchable) [size=32M]
   Capabilities: [40] Power Management version 3
      Flags: PMEClk- DSI- D1- D2- AuxCurrent=375mA PME(D0-,D1-,D2-,D3hot-,D3cold-)
      Status: D0 NoSoftRst+ PME-Enable- DSel=0 DScale=0 PME-
   Capabilities: [50] MSI: Enable- Count=1/8 Maskable+ 64bit+
      Address: 0000000000000000  Data: 0000
      Masking: 00000000  Pending: 00000000
   Capabilities: [70] Express (v2) Endpoint, MSI 00
      DevCap:MaxPayload 512 bytes, PhantFunc 0, Latency L0s unlimited, L1 unlimited
         ExtTag+ AttnBtn- AttnInd- PwrInd- RBE+ FLReset+ SlotPowerLimit 0.000W
      DevCtl:CorrErr- NonFatalErr+ FatalErr+ UnsupReq+
         RlxdOrd+ ExtTag+ PhantFunc- AuxPwr- NoSnoop- FLReset-
         MaxPayload 512 bytes, MaxReadReq 4096 bytes
      DevSta:CorrErr- NonFatalErr- FatalErr- UnsupReq- AuxPwr- TransPend-
      LnkCap:Port #0, Speed 32 GT/s, Width x16, ASPM L1, Exit Latency L1 <64us
         ClockPM- Surprise- LLActRep- BwNot- ASPMOptComp+
      LnkCtl:ASPM Disabled; RCB 64 bytes, Disabled- CommClk-
         ExtSynch- ClockPM- AutWidDis- BWInt- AutBWInt-
      LnkSta:Speed 16GT/s (downgraded), Width x16 (ok)
         TrErr- Train- SlotClk+ DLActive- BWMgmt- ABWMgmt-
      DevCap2: Completion Timeout: Range ABCD, TimeoutDis+ NROPrPrP- LTR-
          10BitTagComp+ 10BitTagReq- OBFF Not Supported, ExtFmt- EETLPPrefix-
          EmergencyPowerReduction Not Supported, EmergencyPowerReductionInit-
          FRS- TPHComp- ExtTPHComp-
          AtomicOpsCap: 32bit- 64bit- 128bitCAS-
      DevCtl2: Completion Timeout: 50us to 50ms, TimeoutDis+ LTR- OBFF Disabled,
          AtomicOpsCtl: ReqEn-
      LnkCap2: Supported Link Speeds: 2.5-32GT/s, Crosslink- Retimer+ 2Retimers+ DRS-
      LnkCtl2: Target Link Speed: 32GT/s, EnterCompliance- SpeedDis-
          Transmit Margin: Normal Operating Range, EnterModifiedCompliance- ComplianceSOS-
          Compliance De-emphasis: -6dB
      LnkSta2: Current De-emphasis Level: -6dB, EqualizationComplete+ EqualizationPhase1+
          EqualizationPhase2+ EqualizationPhase3+ LinkEqualizationRequest+
          Retimer- 2Retimers- CrosslinkRes: Upstream Port
   Capabilities: [100 v2] Advanced Error Reporting
      UESta:DLP- SDES- TLP- FCP- CmpltTO- CmpltAbrt- UnxCmplt- RxOF- MalfTLP- ECRC- UnsupReq-
ACSViol-
```

```
      UEMsk:DLP- SDES- TLP- FCP- CmpltTO- CmpltAbrt- UnxCmplt- RxOF- MalfTLP- ECRC- UnsupReq-
ACSViol-
      UESvrt:DLP+ SDES+ TLP- FCP+ CmpltTO- CmpltAbrt- UnxCmplt- RxOF+ MalfTLP+ ECRC- UnsupReq-
ACSViol-
      CESta:RxErr- BadTLP- BadDLLP- Rollover- Timeout- AdvNonFatalErr-
      CEMsk:RxErr- BadTLP- BadDLLP- Rollover- Timeout- AdvNonFatalErr+
      AERCap:First Error Pointer: 00, ECRCGenCap+ ECRCGenEn+ ECRCChkCap+ ECRCChkEn+
         MultHdrRecCap- MultHdrRecEn- TLPPfxPres- HdrLogCap-
      HeaderLog: 00000000 00000000 00000000 00000000
   Capabilities: [148 v1] Alternative Routing-ID Interpretation (ARI)
      ARICap:MFVC- ACS-, Next Function: 0
      ARICtl:MFVC- ACS-, Function Group: 0
   Capabilities: [158 v1] Secondary PCI Express
      LnkCtl3: LnkEquIntrruptEn- PerformEqu-
      LaneErrStat: 0
   Capabilities: [188 v1] Physical Layer 16.0 GT/s <?>
   Capabilities: [1b8 v1] Lane Margining at the Receiver <?>
   Capabilities: [200 v1] Extended Capability ID 0x2a
   Capabilities: [230 v1] Single Root I/O Virtualization (SR-IOV)
      IOVCap:Migration-, Interrupt Message Number: 000
      IOVCtl:Enable- Migration- Interrupt- MSE- ARIHierarchy-
      IOVSta:Migration-
      Initial VFs: 255, Total VFs: 255, Number of VFs: 0, Function Dependency Link: 00
      VF offset: 1, stride: 1, Device ID: 06a8
      Supported Page Size: 00000553, System Page Size: 00000010
      Region 0: Memory at 0000000000000000 (64-bit, prefetchable)
      VF Migration: offset: 00000000, BIR: 0
   Capabilities: [270 v1] Vendor Specific Information: ID=0002 Rev=4 Len=100 <?>
   Capabilities: [370 v1] Vendor Specific Information: ID=0001 Rev=1 Len=038 <?>
   Capabilities: [3a8 v1] Data Link Feature <?>
   Kernel driver in use: vfio-pci
```

In this example, the **lspci** command lists all PCI devices:

- The **vvv** flag provides verbose output with detailed device capabilities and status.
- The **d 1014:06a7** flag filters for devices with Vendor ID 1014 (IBM) and Device ID 06a7 (IBM Spyre Accelerator). IBM Spyre Accelerator adapters can use either **1014:06a7** or **1014:06a**.

In the output, `182:60:00.0` specifies the exact PCI domain, bus, device, and function.

From the command output, look for the following lines to verify the link speed:

```
LnkCap: Port #0, Speed 32GTps, Width x16, ASPM L1, and Exit Latency L1 <64us
LnkSta: Speed 16GT/s (downgraded), Width x16 (ok)
```

`LnkCap` shows the maximum supported speed (PCIe Gen5: 32 GTps). `LnkSta` shows the operational speed (PCIe Gen4: 16 GTps).

Note: The adapter operates at PCIe Gen 4 speeds in a PCIe Gen 4 drawer. The IBM Spyre Accelerator (Feature Code ECSE) supports PCIe Gen 5 speeds but operates at PCIe Gen 4 speeds in the PCIe Gen 4 drawer.

4.5 Examples and supported workloads

The following sections describe how to run inference with vLLM. Examples include RAG, entity extraction, and embedding. Details in examples are subject to change.

Running RAG and entity extraction

To run RAG or entity extraction workloads, complete the following steps:

1. Set the required environment variables:

AIU_IDS	Space-separated list of AI Accelerator Unit (AIU) IDs for the container to use.
AIU_WORLD_SIZE	Number of AIUs for the container to use.
HOST_MODELS_DIR	Path to the model directory on the host.
VLLM_MODEL_PATH	Path to the model within the container environment.
MAX_MODEL_LEN	Maximum model length and context size. Refers to the maximum sum of the prompt length and output tokens
MAX_BATCH_SIZE	Maximum batch size of the vLLM server.

Example 4-2 shows setting the variables for a RAG environment.

Example 4-2 Variables for the RAG environment

```
export AIU_IDS="0301:50:00.0 0302:60:00.0 0303:70:00.0 0304:80:00.0"
export AIU_WORLD_SIZE=4
export HOST_MODELS_DIR=/models
export VLLM_MODEL_PATH=/models/granite-3.3-8b-instruct
export MAX_MODEL_LEN=32768
export MAX_BATCH_SIZE=32
```

Example 4-3 shows setting the variables for an entity extraction environment.

Example 4-3 Entity extraction settings

```
export AIU_IDS="0301:50:00.0"
export AIU_WORLD_SIZE=1
export HOST_MODELS_DIR=/models
export VLLM_MODEL_PATH=/models/granite-3.3-8b-instruct
export MAX_MODEL_LEN=3072
export MAX_BATCH_SIZE=16
```

2. Start the container by using the command that is shown in Example 4-4.

Example 4-4 Starting the container

```
podman run -d \
      --device=/dev/vfio \
      -v ${HOST_MODELS_DIR}:/models \
      -e AIU_PCIE_IDS="${AIU_IDS}" \
      -e VLLM_SPYRE_USE_CB=1 \
      --pids-limit 0 \
      --userns=keep-id \
      --group-add=keep-groups \
      --memory 200G \
      --shm-size 64G \
      -p 8000:8000 \
```

```
      <container url>:<container tag> \
         --model "${VLLM_MODEL_PATH}" \
         -tp "${AIU_WORLD_SIZE}" \
         --max-model-len "${MAX_MODEL_LEN}" \
         --max-num-seqs ${MAX_BATCH_SIZE}
```

3. View the container by running the **podman ps** command. The container starts a vLLM server. Monitor the progress by using the **podman logs <cid>** command, where **<cid>** is the container ID.
4. When the vLLM server is running, run the **curl** command in Example 4-5 to interact with the model.

Example 4-5 The curl command

```
curl -X POST -H "Content-Type: application/json" -d '{
      "model": "/models/granite-3.3-8b-instruct",
      "prompt": "What is the capital of France?",
      "max_tokens": 50
}' http://<your_server_ip>:8000/v1/completions | jq
```

Running embedded workloads

To run the embedded workloads, complete the following steps:

1. Set the required environment variables, as shown in Example 4-6.

Example 4-6 Setting the environment variables

```
AIU_IDS -> Space-separated list of AIU IDs for the container to use
AIU_WORLD_SIZE -> Number of AIUs for the container to use
HOST_MODELS_DIR -> Path to model directory on host
VLLM_MODEL_PATH -> Path to wanted model within container environment
PROMPT_LENS -> Max length of input prompts in tokens
BATCH_SIZES -> Batch size of vLLM server
```

Example 4-7 shows an example.

Example 4-7 Setting the environment variables

```
export AIU_IDS="0301:50:00.0"
export AIU_WORLD_SIZE=1
export HOST_MODELS_DIR=$HOME/models
export VLLM_MODEL_PATH=/models/granite-embedding-125m-english/
export PROMPT_LENS=512
export BATCH_SIZES=4
```

2. Start the container by running the command that is shown in Example 4-8.

Example 4-8 Starting the container

```
podman run -d \
      --device=/dev/vfio \
      -v ${HOST_MODELS_DIR}:/models \
      -e AIU_PCIE_IDS="${AIU_IDS}" \
      -e VLLM_SPYRE_WARMUP_BATCH_SIZES="${BATCH_SIZES}" \
      -e VLLM_SPYRE_WARMUP_PROMPT_LENS="${PROMPT_LENS}" \
      --userns=keep-id \
      --group-add=keep-groups \
```

```
--pids-limit 0 \
--memory 200G \
--shm-size 64G \
-p 8000:8000 \
<container url>:<container tag> \
--model "${VLLM_MODEL_PATH}" \
-tp "${AIU_WORLD_SIZE}"
```

3. View the container by running the **podman ps** command. The container starts a vLLM server. Monitor the progress by using the **podman logs <cid>** command, where **<cid>** is the container ID.
4. When the vLLM server is running, run the **curl** command in Example 4-9 to interact with the model.

Example 4-9 Interacting with the model

```
curl http://<your_server_ip>:8000/v1/embeddings -H "Content-Type:
applica-tion/json" -d '{ "model": "/models/granite-embedding-125m-english/",
"input": ["What is the capital of Paris?"] }'
```

4.6 Understanding LLM performance

LLMs operate through two phases during inference:

- Prompt intake (also called prefill or prompt processing) is when the model receives and processes the input prompt. During this phase, the model computes attention scores across all input tokens in parallel through matrix multiplications.
- Token generation (also called decode or autoregressive generation) generates tokens sequentially. The model generates one token at a time. Each generated token is used to produce the next token until the response is complete. This sequential process differentiates token generation from the parallel processing of prompt intake.

The two phases have different performance characteristics:

- Prompt intake is compute-intensive because it processes many tokens simultaneously through matrix multiplications that use compute cores and arithmetic units. Key metrics include floating-point operations per second (FLOPS), accelerator usage, and throughput in tokens per second.
- Token generation is memory bandwidth-bound because it generates one token at a time, loading weight matrices from memory for each token prediction. The accelerator waits for data transfer from memory. Critical metrics for token generation include memory bandwidth usage (GBps), latency per token, and arithmetic intensity ratio.

The IBM Spyre Accelerator supports use cases such as RAG and entity extraction. The IBM Spyre Accelerator integrates with Power servers. Enterprise databases on Power infrastructure enable AI workload execution at the data source. This approach supports on-premises AI processing.

Figure 4-11 on page 69 shows a performance comparison of the Power10 Matrix Math Accelerator (MMA) and the IBM Spyre Accelerator.

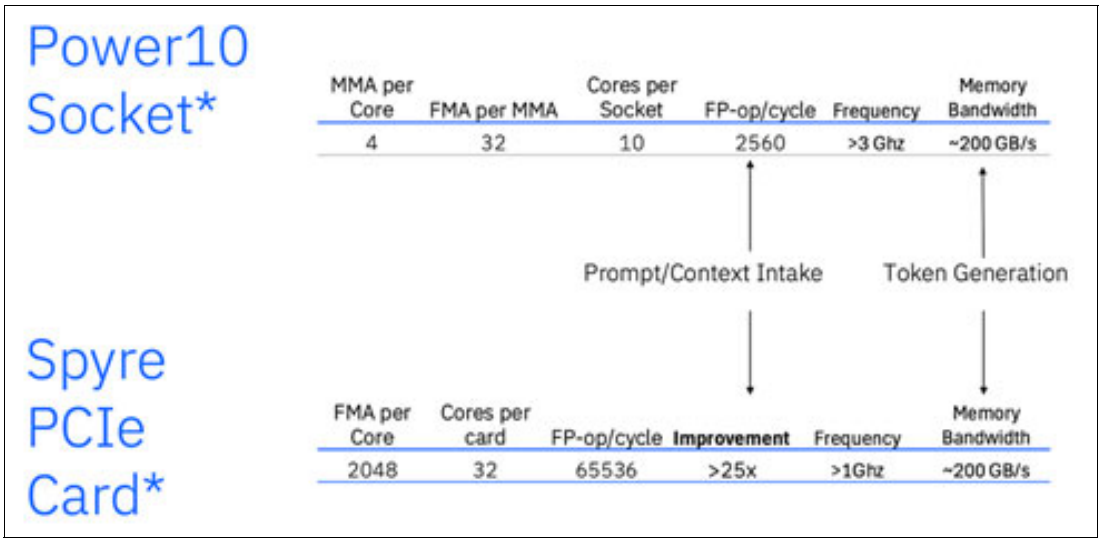


Figure 4-11 Performance guidance

4.7 Hybrid compute: Spyre and CPU

AI systems use a hybrid compute architecture with on-chip and off-chip resources. The Power11 processor includes MMA and single instruction, multiple data (SIMD) for inference tasks. With the IBM Spyre Accelerator, Power11 servers support AI inference workloads.

On-chip workloads run in the processor by using components such as the following ones:

- MMA
- SIMD units and vector engines
- Caches and memory

Off-chip workloads use accelerators such as the IBM Spyre Accelerator. These workloads include the following items:

- Model inference and training
- Agentic workloads with multi-modal reasoning or long-context processing

In RAG pipelines, workload distribution across compute resources improves performance. Preprocessing tasks such as document ingestion and embedding generation can run on CPUs. Inference that uses LLMs is suited for accelerators such as the IBM Spyre Accelerator.

Profiling the pipeline identifies stages for off-chip acceleration. This approach supports scheduling and resource allocation for workload components.

4.8 The IBM AI acceleration roadmap

IBM's AI acceleration roadmap includes planned enhancements:

- Integration with AI platforms
  - IBM watsonx.ai integration for deployment, orchestration, and lifecycle management of AI workloads in hybrid cloud environments
  - Compatibility with Red Hat and IBM AI stacks for model serving, monitoring, and scaling
- Support for agentic workloads
  - Support for agentic AI workloads, including autonomous decision-making systems, multi-agent collaboration, and reasoning engines
  - Low-latency inference and task routing
- Model compatibility

  Support for AI models, including transformer-based architectures, vision-language models, fine-tuned models, and LLMs
- Performance and precision
  - Support for data types such as FP8
  - Hardware and software optimizations for training and inference pipelines

IBM offers a customer support plan for the IBM Spyre Accelerator on Power that covers hardware and software issues with IBM Expert Care response objectives. This support includes Python packages from the IBM Open-Source AI Foundation for Power repository.

5

Artificial intelligence workload optimization on IBM Power11

Artificial intelligence (AI) applications use Python, from model-training frameworks to user interfaces. Python support on Power11 was a design priority. Power11 builds on previous Power generations with hardware improvements, compiler innovations, and Python runtime enhancements for machine learning (ML) and AI workloads.

The Power11 processor includes more cores and expanded Matrix Math Accelerator (MMA) capacity for Python workloads. IBM developers are creating a ppc64le-optimized Python Ecosystem (pyeco) through the devpi project.

This chapter describes hardware improvements, compiler enhancements, and library improvements for Python performance on Power11, which include support for AI and ML frameworks, compatibility with scientific libraries such as OpenBLAS, and the developer experience on Power11.

This chapter describes the following topics:

- Memory bandwidth and cache usage
- NUMA-aware scheduling and resource allocation
- Enhancements in Python performance on Power11
- Support for AI and ML frameworks
- Compatibility with OpenBLAS, NumPy, and other scientific libraries
- Developer productivity and tool improvements

© Copyright IBM Corp. 2026.

5.1 Memory bandwidth and cache usage

The IBM Spyre Accelerator on Power11 uses LPDDR5 memory bandwidth with a Power11 Open Memory Interface (OMI)-based DDR5 subsystem. On-adapter memory, data flow scheduling, and accelerator cores support CPU and IBM Spyre Accelerator execution for AI and inference workloads.

Memory bandwidth and cache features for the IBM Spyre Accelerator on Power11 include the following items:

- On-adapter LPDDR5
- Up to 16 IBM Spyre Accelerator adapters for bandwidth and memory scaling
- Continuous batching and data flow scheduling
- 32 accelerator cores with operands
- Power11 OMI DDR5 subsystem

These features support AI and inference workloads on Power11 servers.

Power11 includes OMI DDR5 subsystem improvements, which support IBM Spyre Accelerator host-side operations (command dispatch, I/O, preprocessing, and postprocessing) for memory access. The IBM Spyre Accelerator benefits for the following reasons:

- CPU preprocessing and model orchestration throughput
- Memory access for data ingestion, staging, and batching

The Power11 DDR5 memory system combined with the IBM Spyre Accelerator LPDDR5 memory support AI workloads.

5.2 NUMA-aware scheduling and resource allocation

AI models must run on a single NUMA node on Power10 or Power11 servers. On Power servers, a NUMA node represents a single processor chip: a single-chip module (SCM) or half of a dual-chip module (DCM).

Figure 5-1 on page 73 shows an IBM Power S1022 or IBM Power S1024 server in a 2-socket configuration.

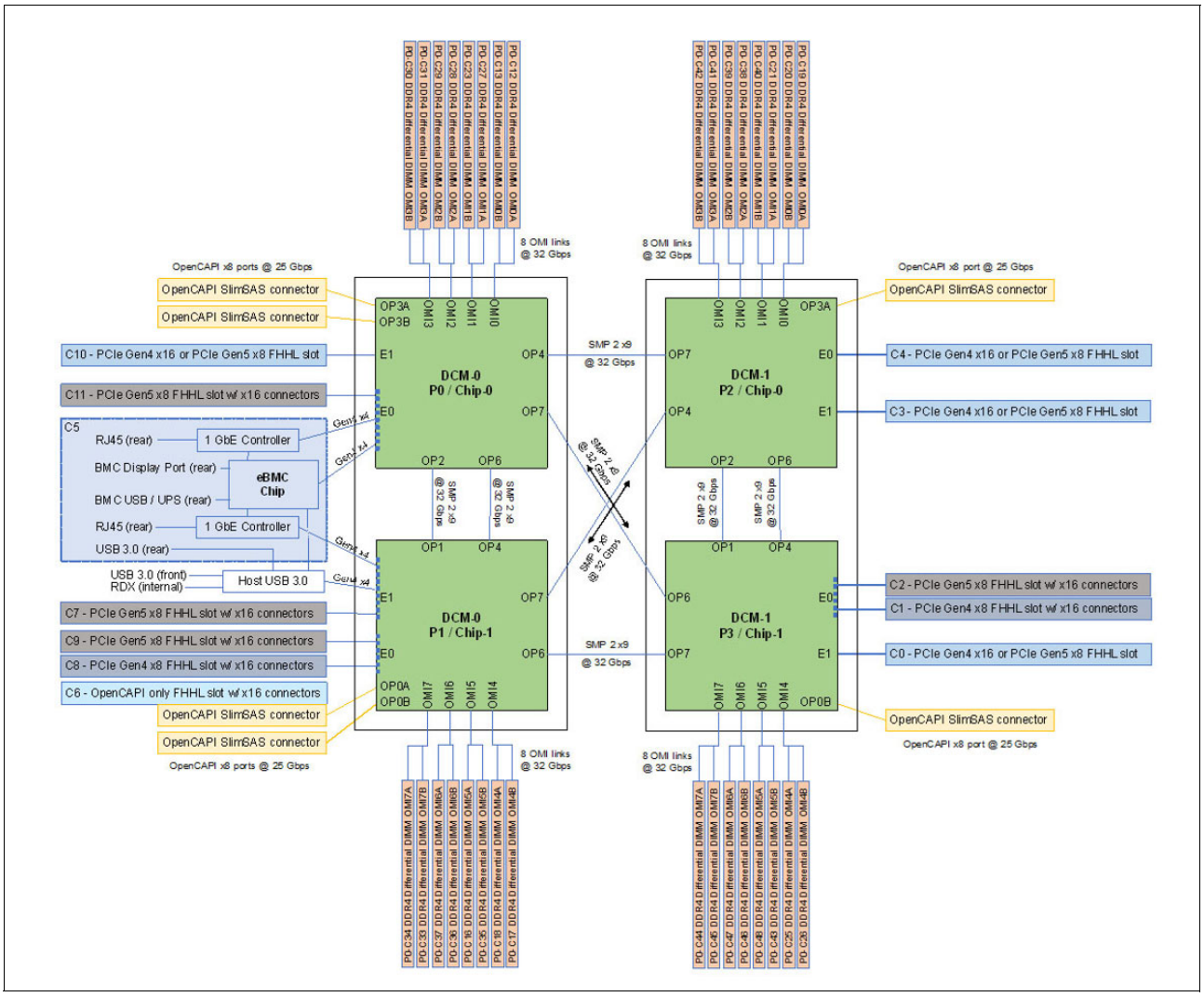


Figure 5-1 Logical diagram of Power S1022 or Power S1024 servers in 2-socket configurations

The two white boxes with black borders represent the two sockets, and the green boxes represent processor chips (NUMA nodes). Two chips form a DCM on a socket. Lines between chips represent data cables. These cables can create bottlenecks when running AI workloads across multiple NUMA nodes.

When an AI model processes a request, data packets are transferred between physical cores. Sending packets over cables between chips reduces performance. Running AI workloads on a single NUMA node with six cores provides better performance than using 12 cores across multiple NUMA nodes.

Deploy an AI model on a single NUMA node. Size and schedule the LPAR or Red Hat OpenShift worker node for the AI workload to fit on a single NUMA node. For example, if a single NUMA node has a maximum of 8 cores (such as a 16-core DCM), the LPAR can have up to 8 dedicated cores. Allocate memory from the same NUMA node as the cores to prevent memory hops. Populate all memory DIMMs.

Power11 introduced *resource groups*, which enable you to add specific cores and memory units to a resource group and assign this resource group to an LPAR. A deployment on a single NUMA node can be ensured. For more information, see Resource Groups on Power11.

Power10 does not support resource groups. To deploy an LPAR on a single NUMA node, choose either one of the following options:

- Start the LPAR before other LPARs on the system.
- Create placeholder LPARs to fill the NUMA node, and then start the LPAR.

5.3 Enhancements in Python performance on Power11

IBM Power11 Python performance improvements include hardware architecture, compiler and toolchain ecosystem, and runtime environment enhancements. These improvements support AI, high-performance computing (HPC), and data-intensive applications.

- Hardware layer: The Power11 architecture

  Power11 includes up to 25% more cores per chip compared to Power10 and retains four MMA engines per core. Power11 includes memory bandwidth, cache, and interconnect improvements.

- Compiler and toolchain layer: MMA-aware optimization

  Software uses hardware capabilities. Starting with GNU Compiler Collection (GCC) 10.2, the toolchain includes MMA functions and flags such as **`-mcpu=power11`**. Numerical libraries such as OpenBLAS compile kernels for matrix acceleration. Python packages such as NumPy and SciPy use these optimizations.

- Runtime layer: Modernized Python execution

  Python releases 3.13 and 3.14 include free-threaded execution (PEP 779) and an experimental JIT compiler. With Power11 core count and MMA acceleration, these features support compute-bound workloads.

These hardware, compiler, and runtime improvements support Python performance on Power11.

5.3.1 Power11 hardware enhancements

Power11 processor hardware enhancements include the following items:

- Up to 25% more cores compared to Power10
- Four MMA engines per core
- Memory bandwidth and cache improvements
- Energy-efficiency improvements
- CPU-to-CPU interconnect improvements

5.3.2 Compiler toolchain and library-level enhancements

Python packages such as NumPy, SciPy, and scikit-learn use numerical libraries such as OpenBLAS, which depend on the GCC toolchain.

Starting with GCC 10.2, the compiler includes built-in functions for MMA, enabling numerical kernels to use hardware acceleration with flags such as **`-mcpu=power11`**. For more information, see the following resources:

- Add support for -mcpu=power11
- 6.62.29 PowerPC Matrix-Multiply Assist Built-in Functions

Compiling OpenBLAS with these flags on Power11:

- Detects and uses MMA units per core for matrix operations.
- Enables packages such as NumPy and SciPy to use these optimizations.

Python APIs use hardware acceleration without requiring code changes.

Best practice: Verify each package's configuration to help ensure that MMA-optimized libraries are used. Alternatively, build packages from source and explicitly link to the optimized libraries.

5.3.3 Python Interpreter and runtime enhancements

Python 3.13 and 3.14 include CPython runtime improvements:

- PEP 779: Free-Threaded Python

  Python 3.14 supports free-threaded execution (experimental in 3.13). Disabling the Global Interpreter Lock (GIL) enables Python to use CPU cores for parallel multi-threaded workloads.

  Note: Not all applications benefit automatically. Numerical libraries such as OpenBLAS already bypass the GIL by using OpenMP for parallel execution. For more information, see Free-threaded Python is officially supported.

- Experimental JIT compiler

  To help reduce interpreter processing, Python 3.13 introduced an experimental JIT compiler. For more information, see An experimental just-in-time (JIT) compiler.

5.3.4 Workload validation

A series of workload validation exercises were conducted by using example programs from the pyeco GitHub repository to demonstrate performance scaling and ecosystem compatibility for Python-based AI and ML workloads on Power11 servers.

Each workload used ppc64le packages from the devpi repository. The examples, while not benchmarks, cover computational patterns from dense linear algebra to data transformation and inference tasks.

Run time was measured by using the time utility to capture interpreter startup, dependency resolution, and computation. Dependencies were resolved before execution for consistent and reproducible results.

Power11 and Power10 share the Power ISA v3.1 architecture. The Power11 configuration demonstrated improved throughput, with time reductions of up to 40% in several test cases. These improvements reflect Power11 enhancements such as increased core efficiency, improved cache hierarchy, and optimized instruction scheduling.

Table 5-1 outlines the test environment.

Table 5-1 Workload validation test environment

Characteristic	Power11	Power10
Hardware	Red Hat Enterprise Linux (RHEL) 10 LPAR running on a Power S1124 server in one NUMA node with 16 CPUs (SMT8) with 32 GB of memory	RHEL 10 LPAR running on a Power S1024 server in one NUMA node with 16 CPUs (SMT8) with 32 GB of memory
Kernel	6.12.0-55.9.1.el10_0.ppc64le	6.12.0-55.9.1.el10_0.ppc64le
Python	3.12.9	3.12.9
GCC	14.2.1	14.2.1

Table 5-2 summarizes the observed performance characteristics across each workload.

Table 5-2 Observed performance characteristics across each workload

Example	Metric	Power11 (s)	Power10 (s)	%speedup
ONNX	real	5.300	6.720	+21%
ONNX	user	4.118	4.337	+5%
Scikit-learn	real	7.136	12.558	+43%
Scikit-learn	user	5.044	8.873	+43%

Note: These tests use CPU execution with ppc64le libraries. The workloads include non-Python components (such as shell scripts). They illustrate performance scaling in the Power ecosystem and are not benchmarks.

User time indicates computational efficiency and CPU usage for CPU-bound workloads. *Real time* shows the wall-clock time, including interpreter startup and I/O processing.

5.4 Support for AI and ML frameworks

Modern enterprise workloads increasingly rely on AI and ML to drive innovation, automate decision-making, and extract insights from vast datasets. Power servers, particularly those running Linux, support these workloads because of their robust architecture, high memory bandwidth, and hardware acceleration capabilities. This section explores the integration and optimization of AI and ML frameworks (PyTorch, TensorFlow, and scikit-learn) on Power platforms.

5.4.1 Overview of frameworks

This section focuses on three AI and ML frameworks:

- PyTorch is a Python-based deep learning (DL) framework that is developed by Meta AI. It is used for research and production because of its graphics processing unit (GPU) acceleration support.
- TensorFlow, developed by Google, is an ML framework that supports both DL and traditional ML models. It offers static graph execution for large-scale deployments.
- scikit-learn is a Python library for ML algorithms, including regression, classification, clustering, and dimensionality reduction. It is built on NumPy, SciPy, and matplotlib.

These frameworks need efficient computation, parallel processing, and hardware acceleration. Power servers address these requirements with Power10 and Power11 processors, integrated AI Accelerator Units (AIUs), and high-bandwidth memory subsystems.

Compatibility and installation on Linux on Power

AI and ML frameworks are compatible with Linux distributions on the IBM Power Architecture. Containerized deployments that use Docker or Podman are supported.

Alternatively, IBM offers pre-built container images through the IBM Container Registry and Open Cognitive Environment (OpenCE). These containers integrate AI frameworks and libraries (oneDNN, OpenBLAS, and Eigen).

Library optimization refers to tuning mathematical and computational libraries so that they can use the underlying hardware architecture. Here are some examples of library optimization:

- Instruction-level optimization

  oneDNN (for DL) and OpenBLAS and Eigen (for linear algebra) use vectorized instructions such as Vector Scalar eXtension (VSX) on Power and single instruction, multiple data (SIMD) units for parallel execution.

- Threading and parallelism

  Optimized libraries use multi-threading and simultaneous multithreading (SMT). Workloads are distributed across cores and threads.

- Memory hierarchy awareness

  Libraries are tuned to minimize cache misses and optimize data locality. They use techniques such as blocking and prefetching.

- Hardware acceleration integration

  For AI workloads, libraries such can offload certain operations to AIUs on Power10 or Power11 to accelerate tensor and matrix computations.

- Precision and data types

  Libraries support mixed-precision computing (FP32 or FP16) to balance accuracy and performance.

Hardware acceleration and optimization

Power servers offer several hardware features that accelerate AI and ML workloads:

- AIU

  Introduced in Power10 and enhanced in Power11, AIU provides MMA for DL operations. PyTorch and TensorFlow use AIU through optimized back ends (ONNX Runtime, XLA, or custom kernels).

- SIMD and VSX extensions

  These vector instructions accelerate linear algebra operations that are used in scikit-learn and other numerical libraries.

IBM contributes fixes so that Power optimizations are in major releases of PyTorch and TensorFlow. These fixes support little-endian Power, Altivec and VSX, and multi-threaded basic linear algebra and subprograms (BLAS) libraries.

5.5 Compatibility with OpenBLAS, NumPy, and other scientific libraries

OpenBLAS and NumPy provide matrix multiplication for Python on Power11. These libraries support numerical computation in Python. This section describes how they use MMA capabilities on Power11.

Power ISA 3.1 is the instruction set architecture for Power10 and Power11 processors. This instruction set includes hardware extensions for matrix operations across data types, including double precision, single precision, and half precision computations. The MMA unit provides eight 512-bit accumulator registers that perform matrix multiply and accumulate instructions. These accumulators support numerical workloads such as ML and scientific computing.

GCC provides compiler support for these instructions. As described in 5.3.2, “Compiler toolchain and library-level enhancements” on page 74, the toolchain includes built-in functions for MMA capabilities. You can enable these functions with compilation flags such as **`-mcpu=power11`** or **`-mcpu=native`**.

The following process validates compatibility:

1. Compile OpenBLAS from source by using the MMA instruction set that is defined by ISA 3.1.
2. Build NumPy from source and link to this OpenBLAS library.
3. Verify the configuration through NumPy to confirm library linkage and kernel recognition (Figure 5-2 on page 79 shows the configuration.
4. Run NumPy test scripts to validate functions and performance.

These tests confirm that both OpenBLAS and NumPy correctly use MMA-enabled kernels when running on Power10 and Power11.

OpenBLAS identifies kernels as Power10 in its configuration because Power11 and Power10 share the Power ISA 3.1 instruction set. Kernel naming reflects the ISA level rather than the processor generation.

5.5.1 Inspecting the OpenBLAS configuration

To verify that OpenBLAS uses MMA (Power10) kernels for routines, inspect the static or shared libraries for references to those functions.

Querying the library

Query the shared object to confirm that references point to the gemm (general matrix-matrix multiply – Level 3 BLAS routines) implementation files. For example, in Figure 5-2, you see the following strings (and the functions that they correspond to):

- sgemm for single precision
- dgemm for double precision
- cgemm and zgemm for complex operations

```
(powerai) [cecuser@p1226-pvm1 ~]$ strings /opt/openblas/lib/libopenblasopenmp.so | grep -E "power10" | sort
caxpy_power10.c
ccopy_power10.c
cgemm_kernel_power10.c
daxpy_power10.c
dcopy_power10.c
ddot_power10.c
dgemm_kernel_power10.c
dgemm_ncopy_8_power10.c
dgemm_small_kernel_nn_power10.c
dgemm_small_kernel_nt_power10.c
dgemm_small_kernel_tn_power10.c
dgemm_small_kernel_tt_power10.c
dgemv_n_power10.c
dgemv_t_power10.c
gemm_small_kernel_permit_power10.c
saxpy_power10.c
sbgemm_kernel_power10.c
sbgemm_ncopy_16_power10.c
sbgemm_ncopy_8_power10.c
sbgemm_tcopy_16_power10.c
sbgemm_tcopy_8_power10.c
sbgemv_n_power10.c
sbgemv_t_power10.c
scopy_power10.c
sdot_power10.c
sgemm_kernel_power10.c
sgemm_small_kernel_nn_power10.c
sgemm_small_kernel_nt_power10.c
sgemm_small_kernel_tn_power10.c
sgemm_small_kernel_tt_power10.c
trsm_kernel_LN_power10.c
trsm_kernel_LT_power10.c
trsm_kernel_RN_power10.c
trsm_kernel_RT_power10.c
zaxpy_power10.c
zcopy_power10.c
zgemm_kernel_power10.c
zgemv_n_power10.c
```

Figure 5-2 Querying the library for the Power10 gemm kernel

The associated `*.c` files implement the MMA-based microkernels for Power10.

To display this output, run the following command:

```
strings /opt/openblas/lib/libopenblasopenmp.so | grep -E "power10" | sort
```

5.5.2 Inspecting the NumPy configuration

Inspect the NumPy configuration to verify that the installation is using the optimized libraries and hardware features. NumPy is a Python library, but its performance depends on compiled C and Fortran code that is linked to back ends such as BLAS and LAPACK. Different builds use different implementations (such as OpenBLAS, Intel MKL, or IBM ESSL), which affect performance. By examining the configuration, you can verify which libraries are active, confirm SIMD instruction support (such as AVX on x86 or VSX on IBM Power), and check multi-threading support.

NumPy provides the **`numpy.show_config()`** function for configuration inspection. This command displays the linked BLAS and LAPACK libraries, SIMD features, compiler flags, and threading layers. For example, the output might show `openblas_info` or `essl_info` under the BLAS section, indicating the back end in use. It reveals instruction sets and optimization flags such as **`-O3`** or **`-march=native`** that are included during compilation.

On Power servers, confirm that ESSL or OpenBLAS is linked, VSX SIMD support is enabled, and compiler options are configured for the Power Architecture. These checks verify that NumPy uses hardware features for AI, HPC, and data analytics workloads.

Example source build

In this example, OpenBLAS was built without dynamic architecture support and **`-mcpu`** flags so that it uses the native architecture during configuration on each system. Example 5-1 shows this configuration.

Example 5-1 Building OpenBLAS

```
export USE_THREAD=1
export NUM_THREADS=$(nproc)
export DYNAMIC_ARCH=0
```

OpenBLAS was configured with multithreading by using OpenMP, as shown in Example 5-2.

Example 5-2 OpenBLAS with multithreading through OpenMP

```
export OPENBLAS_NUM_THREADS=$(nproc)
export OMP_NUM_THREADS=$(nproc)
make -j DYNAMIC_ARCH=0 CC=gcc FC=gfortran HOSTCC=gcc BINARY=64 INTERFACE=64 \
USE_OPENMP=1 LIBNAMESUFFIX=openmp
```

NumPy was built by using the OpenBLAS libraries and targeting the shared libraries by using `$LD_LIBRARY_PATH` and `$PKG_CONFIG_PATH`.

To compile OpenBLAS on Linux, see this GitHub repository.

To build NumPy from source by using venv or system Python on Linux, see Building from source.

To query the OpenBLAS and NumPy configuration, run the following command:

```
python3 -c "import numpy, numpy.__config__ as c; c.show()”
```

Figure 5-3 on page 81 shows the optimized configuration on Power11.

```
(powerai) [cecuser@p1226-pvm1 ~]$ python3 -c "import numpy, numpy.__config__ as c; c.show()"
Build Dependencies:
  blas:
    detection method: pkgconfig
    found: true
    include directory: /opt/openblas/include
    lib directory: /opt/openblas/lib
    name: openblas
    openblas configuration: USE_64BITINT= DYNAMIC_ARCH= DYNAMIC_OLDER= NO_CBLAS= NO_LAPACK=
      NO_LAPACKE= NO_AFFINITY=1 USE_OPENMP=1 POWER10 MAX_THREADS=16
    pc file directory: /opt/openblas/lib/pkgconfig
    version: 0.3.30.dev
  lapack:
    detection method: pkgconfig
    found: true
    include directory: /opt/openblas/include
    lib directory: /opt/openblas/lib
    name: openblas
    openblas configuration: USE_64BITINT= DYNAMIC_ARCH= DYNAMIC_OLDER= NO_CBLAS= NO_LAPACK=
      NO_LAPACKE= NO_AFFINITY=1 USE_OPENMP=1 POWER10 MAX_THREADS=16
    pc file directory: /opt/openblas/lib/pkgconfig
    version: 0.3.30.dev
Compilers:
  c:
    args: -O3, -ftree-vectorize, -fprefetch-loop-arrays, --param, prefetch-latency=300
    commands: cc
    linker: ld.bfd
    linker args: -O3, -ftree-vectorize, -fprefetch-loop-arrays, --param, prefetch-latency=300
    name: gcc
    version: 14.2.1
  c++:
    commands: c++
    linker: ld.bfd
    name: gcc
    version: 14.2.1
  cython:
    commands: cython
    linker: cython
    name: cython
    version: 3.1.5
Machine Information:
  build:
    cpu: ppc64le
    endian: little
    family: ppc64
    system: linux
  host:
    cpu: ppc64le
    endian: little
    family: ppc64
    system: linux
Python Information:
  path: /home/cecuser/venvs/powerai/bin/python
  version: '3.12'
SIMD Extensions:
  baseline:
  - VSX
  - VSX2
  found:
  - VSX3
  - VSX4
```

Figure 5-3 Power11 results from querying NumPy

Validate the build by confirming the values in Example 5-3.

Example 5-3 Validating the directories

```
include directory: /opt/openblas/include
    lib directory: /opt/openblas/lib
```

The build dependencies in this example (blas and lapack) point directly to a custom build in `/opt/openblas`, as shown in Example 5-4.

Example 5-4 openblas configuration

```
name: openblas
openblas configuration: USE_64BITINT= DYNAMIC_ARCH=1 DYNAMIC_OLDER= NO_CBLAS=
NO_LAPACK= NO_LAPACKE= NO_AFFINITY=1 USE_OPENMP=1 POWER10 MAX_THREADS=16
```

The processor type that is detected at build time is Power10 because Power11 and Power10 share the ISA 3.1 architecture.

Example 5-5 shows the **`make`** output when building with dynamic architecture support on Power11.

Example 5-5 Build complete message

```
OpenBLAS build complete. (BLAS CBLAS LAPACK LAPACKE)
  OS               ... Linux
  Architecture     ... power
  BINARY           ... 64bit
  C compiler       ... GCC (cmd & version: gcc (GCC) 14.2.1 20250110 (Red Hat 14.2.1-7))
  Fortran compiler ... GFORTRAN (cmd & version: GNU Fortran (GCC) 14.2.1 20250110 (Red Hat 14.2.1-7))
  Library Name     ... libopenblasopenmpp-r0.3.30.dev.a (Multi-threading; Max num-threads is 16)
  Supporting multiple power cpu models with minimum requirement for the common code being POWER10
```

Important: The common code requires Power10.

Example of pip install

Package management supports performance on Power servers for AI, HPC, and data analytics workloads. **`pip install`** is the standard method for installing Python packages. Extra considerations apply for the Power Architecture to help ensure compatibility and performance. In our testing, we used the **`pip install`** command that is shown in Example 5-6.

Example 5-6 Sample pip install command

```
pip install --prefer-binary numpy openblas
--extra-index-url=https://wheels.developerfirst.ibm.com/ppc64le/linux
```

This method installs Python packages from the devpi project. Figure 5-4 on page 83 shows the output.

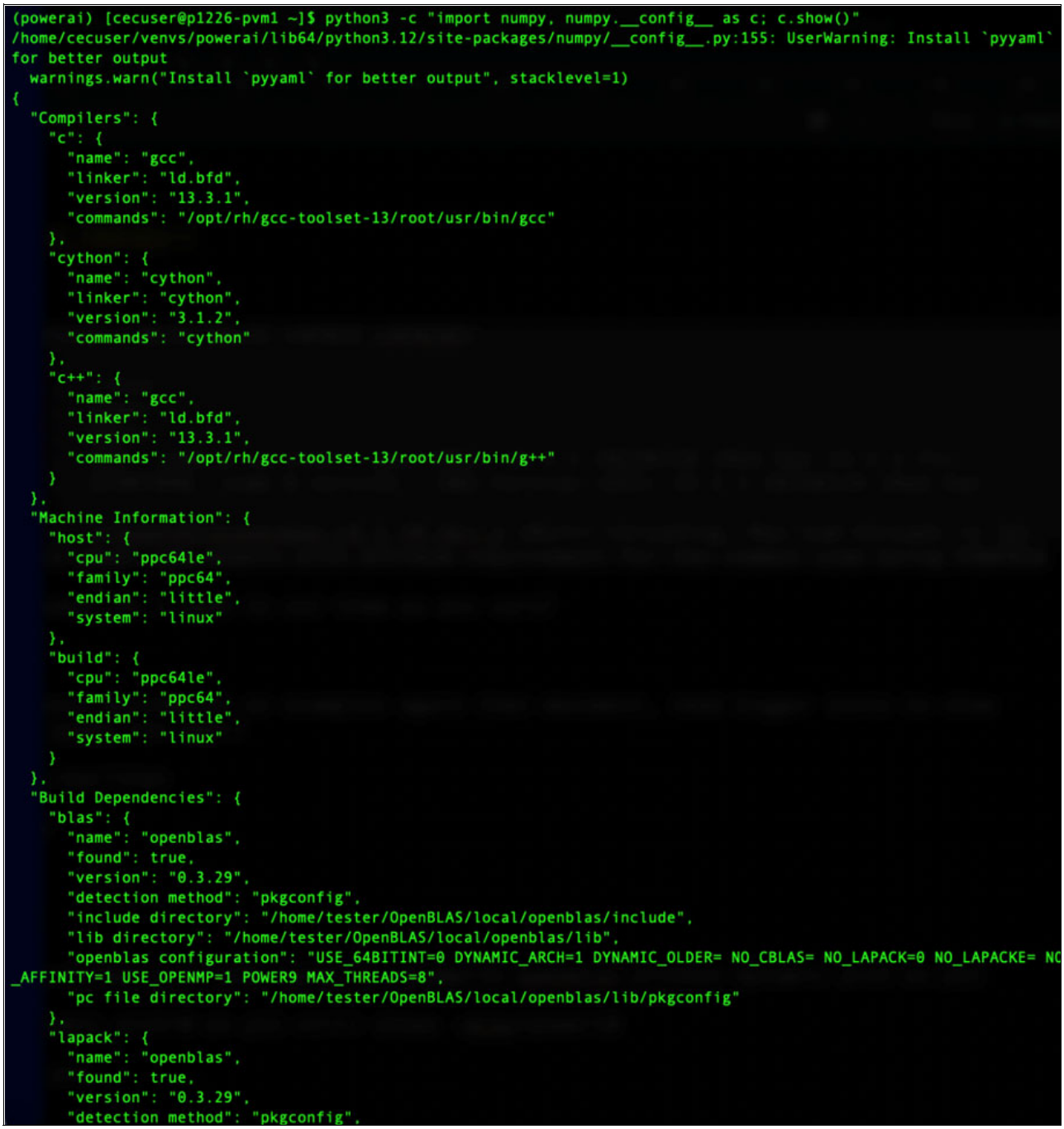


Figure 5-4 Devpi NumPy config query on an RHEL 10 Power11 LPAR

Two points to note here:

- Older compiler versions (for example, GCC 13.3.1) include MMA built-in functions, even though they predate the latest Power toolchain.
- The OpenBLAS configuration specifies IBM Power9 as the architecture at build time for BLAS and LAPACK. The MMA kernels might not be compiled.

```
"openblas configuration": "USE_64BITINT=0 DYNAMIC_ARCH=1 DYNAMIC_OLDER=
NO_CBLAS= NO_LAPACK=0 NO_LAPACKE= NO_AFFINITY=1 USE_OPENMP=1 Power9
MAX_THREADS=8"
```

At the time of writing, build both programs from source to help ensure Power10 compatibility.

5.6 Developer productivity and tool improvements

Power11 supports Python workloads that use the Power Architecture and ppc64le libraries. The platform includes package availability, tools, and integration.

The ppc64le ecosystem supports Python packages on Power servers, including packages that are built with ppc64le optimizations and compiled with math libraries such as OpenBLAS that use Power VSX and MMA capabilities

On RHEL on Power, the system's Python integrates with the platform's package manager and libraries. This approach reduces environment setup tasks.

Devpi enables organizations to maintain mirror registries of PyPI for ppc64le. This approach supports Python CI/CD pipelines on Power servers, and accelerated package installation and dependency resolution times when combined with Power11 improved I/O and memory latency.

IBM support for pyeco and build tools such as **`pip`** and **`wheel`** includes support for ppc64le. Native support for containerized Python development (by using Podman or Docker) with pre-built Python images enables deployment on Power11.

IBM supports Power as a platform for open-source and enterprise Python environments and provides packages for Power developers.

For more information, see Developing applications using Python Packages on IBM Power.

6

Artificial intelligence and security

Artificial intelligence (AI) workloads require security because they process sensitive data in regulated environments. AI models use large data volumes, such as financial transactions, healthcare records, and operational telemetry, which must be protected against unauthorized access, breaches, and tampering. Security measures protect data confidentiality, integrity, and availability throughout the AI lifecycle, from training to inference.

Considerations include data privacy and regulatory compliance for personally identifiable information (PII) or industry-specific regulated data. Infrastructure such as encrypted storage, Secure Boot, and isolated execution environments helps prevent attacks on AI models and pipelines. Protecting against adversarial threats where malicious inputs manipulate model behavior maintains reliability. Organizations need identity and access management (IAM), network security, and monitoring for AI workloads in hybrid and edge environments.

This chapter describes the following topics:

- Secure model deployment
- Data privacy and encryption in AI pipelines
- Security and AI on IBM Power
- Securing AI on IBM Power
- Controlling data and models in your AI ecosystem

© Copyright IBM Corp. 2026.

6.1 Secure model deployment

Secure model deployment supports AI systems in compliance with organizational and regulatory requirements. Threats for deployed models include model extraction, where attackers replicate models through repeated API queries, and membership inference attacks, which determine whether specific data points were in the training set. Adversarial examples mislead models, and poisoning attacks introduce malicious data during training. Supply chain vulnerabilities in pre-trained models or libraries can compromise security. Dependency scanning and maintaining a Software Bill of Materials (SBOM) address these vulnerabilities.

Organizations should adopt deployment practices to mitigate these risks:

- Environment hardening includes minimal container images, kernel security features such as Seccomp and AppArmor, and runtime anomaly detection tools such as Falco.
- Authentication and authorization use mechanisms such as mutual TLS, short-lived tokens, dynamic secrets, and role-based access control (RBAC), which are aligned with the principle of least privilege.
- Encryption includes AES-256 for model artifacts, TLS 1.3 for inference traffic, and automated key rotation through key management systems.
- Model integrity uses cryptographic signing and verification during CI/CD and runtime. Immutable deployment artifacts should be stored in trusted registries.

Monitoring and logging detect anomalies and support accountability. Anomaly detection for API calls, SIEM integration for monitoring, and privacy-preserving logging practices maintain security. Compliance and governance frameworks such as NIST AI Risk Management and ISO/IEC 27001 guide operations. Model cards, data sheets, and audit trails document training, deployment, and access events.

Protections such as differential privacy, federated learning, homomorphic encryption, and secure enclaves (for example, Intel SGX or IBM Power Secure Execution) reduce data exposure and support confidential computation.

Deployment patterns vary by context. On-premises environments support regulated industries. Cloud deployments use private endpoints, customer-managed encryption keys, and confidential computing features. Hybrid deployments combine preprocessing on-premises with inference in the cloud. Edge deployments use lightweight models with encrypted updates and Secure Boot mechanisms.

Securing the machine learning (ML) lifecycle includes security checks in CI/CD pipelines, including model validation, dependency scanning, and vulnerability assessments. Signed containers, trusted registries, and policy enforcement tools such as Open Policy Agent (OPA) support integrity. Trends include confidential AI, zero-trust principles for ML pipelines, and AI Bills of Materials (AI-BOM) for tracking model lineage.

6.2 On-premises compared to cloud deployment

Models can be deployed on-premises or in the cloud. This section compares security considerations for these deployment options:

- Control and visibility

  On-premises deployments provide control over hardware, network, and physical access for enforcing security policies and compliance requirements. This approach provides visibility into the stack.

  Cloud deployments use a shared responsibility model. The provider secures infrastructure, and the customer secures workloads. Cloud deployments limit visibility into hardware and hypervisors and rely on provider compliance certifications.

- Data security

  For data security, on-premises environments keep sensitive data within a controlled infrastructure and allow for air-gapped systems for high-security workloads. Cloud environments typically provide encryption at rest and in transit as a standard feature, but encryption keys are often managed by the provider unless customer-managed keys (CMKs) are used. Misconfigured storage buckets or IAM policies can increase the risk of data exposure in cloud deployments.

- Network security

  In terms of network security, on-premises setups enable organizations to manage internal network segmentation and firewalls directly, making zero-trust enforcement simpler. Cloud environments offer strong perimeter security but require careful configuration of virtual private clouds (VPCs), security groups, and private endpoints to prevent accidental public exposure.

- IAM

  For IAM, on-premises deployments typically integrate with enterprise systems like Active Directory or LDAP, giving full control over RBAC and multi-factor authentication (MFA) policies. Cloud platforms provide advanced IAM capabilities, including fine-grained roles and temporary credentials, but misconfigured policies can lead to privilege escalation risks.

- Compliance and governance

  Regarding compliance and governance, on-premises solutions are often better suited for highly regulated industries such as finance and healthcare because they enable audit control and adherence to strict compliance requirements. Cloud providers offer certifications like ISO, SOC, and HIPAA, but ultimate responsibility for data handling remains with the customer, and cross-border data residency can pose extra challenges.

- Threat surface

  The threat surface differs significantly. On-premises deployments require organizations to manage physical security and address insider threats, which can be harder to detect. Cloud environments have a larger attack surface due to multi-tenancy, though providers invest heavily in security. However, misconfiguration by customers remains a common cause of breaches.

In summary, on-premises deployments provide maximum control and are ideal for sensitive workloads but come with higher operational impact. Cloud deployments offer scalability, advanced IAM features, and built-in security capabilities, but they require strong configuration practices and trust in the provider.

Table 6-1 summarizes these considerations.

Table 6-1 Summary of security considerations for on-premises and in the cloud

Security dimension	On-premises	Cloud
Control and visibility	Full control over hardware, network, and physical access, with complete visibility.	Shared responsibility model with limited visibility into hardware. Relies on provider compliance.
Data security	Sensitive data stays in a controlled environment. Air-gapped systems are possible.	Encryption at rest and in transit are standard. Keys are often provider-managed unless CMK is used. There is a risk of misconfiguration.
Network security	Internal segmentation and firewalls support a zero-trust architecture.	Cloud deployments use VPC and endpoint configuration. Misconfiguration can cause security issues.
IAM	On-premises deployments integrate with enterprise AD and LDAP and use RBAC and MFA.	Cloud deployments provide IAM features such as roles and temporary credentials. Misconfiguration can cause privilege escalation.
Compliance and governance	On-premises deployments support regulated industries and provide audit control.	Cloud providers offer certifications such as ISO, SOC, and HIPAA. Customers are responsible for data handling. Cross-border deployments require extra considerations.
Threat surface	On-premises deployments require physical security management. Insider threats can be difficult to detect.	Cloud deployments include multi-tenancy. Providers invest in security. Misconfiguration can cause security issues.
Summary	On-premises deployments provide control for sensitive workloads and require operational management.	Cloud deployments provide scalability and features. Configuration and provider selection are required.

6.3 Data privacy and encryption in AI pipelines

As AI becomes increasingly embedded in modern digital ecosystems, the importance of securing AI pipelines has never been greater. From healthcare and finance to autonomous systems and smart cities, AI models are trained on vast amounts of sensitive data. Ensuring data privacy and encryption throughout the AI lifecycle is essential to protect individuals, organizations, and the integrity of the models themselves.

An AI pipeline refers to the end-to-end workflow involved in building, training, deploying, and monitoring ML models. It includes the following areas:

- Data collection and preprocessing
- Model training and validation

- Deployment and inference
- Monitoring and maintenance

Each stage presents unique security challenges, especially when handling sensitive information or PII.

AI systems often rely on large datasets that might include personal, financial, or health-related information. Without proper safeguards, a system might leak sensitive data through model outputs or logs; be vulnerable to adversarial attacks; or violate data protection regulations like the General Data Protection Regulation (GDPR), HIPAA, or India's Digital Personal Data Protection (DPDP) Act.

6.3.1 Data privacy threats in ML systems

Major security threats to an ML System can be categorized in the following ways:

- Data poisoning

  Attackers introduce fake or harmful data during training to manipulate model learning. For example, Microsoft launched the Tay chatbot on Twitter in 2016, which was designed to learn from user interactions. Malicious users provided offensive content. Tay generated racist and inappropriate tweets, and Microsoft shut it down[1].

- Adversarial attacks

  Small changes to input (such as edited pixels in an image) can cause model errors. For example, researchers placed stickers on a stop sign to trick a computer vision model in autonomous vehicles into misclassifying it as a speed limit sign[2].

- Model or intellectual property theft

  Stealing a trained model or reverse engineering it. For example, healthcare AI models that are trained on patient data require strict data privacy controls, including encryption and anonymization to comply with regulations like HIPAA and help ensure patient confidentiality.

- Infrastructure attacks

  Infrastructure attacks target servers, networks, or containers where ML pipelines run. For example, in the Capital One data breach (2019), a misconfigured firewall allowed access to data stored in AWS S3 buckets, affecting over 100 million customers. Infrastructure vulnerabilities in cloud environments can expose data and models[3].

- Data leaks

  Data leakage occurs when training or prediction data is exposed or stolen. For example, in the Strava heatmap incident (2018), Strava, a fitness tracking app, published a heatmap showing user activity. Analysts discovered that the map revealed locations of military bases and patrol routes because soldiers that used the app shared their data. This case of data leakage shows how outputs can expose information[4].

[1] https://ethicsunwrapped.utexas.edu/wp-content/uploads/2025/08/AI-Trust_Case-Study-1.pdf
[2] https://arxiv.org/pdf/1707.08945v5
[3] https://cybersecurityattorney.com/case-study-capital-one-vs-the-cloud-how-one-misstep-triggered-190m-in-settlements/
[4] https://www.cnbc.com/2018/01/30/strava-reviewing-features-after-military-bases-were-found-on-heat-map.html

6.3.2 Key measures for protecting ML systems

Securing ML systems requires a multi-layered approach that addresses data, models, and infrastructure. Here are the essential measures that organizations should implement to safeguard their AI pipelines:

- Secure data management

  Protecting the data that is used in ML pipelines is foundational to helping ensure privacy and security.

  - Encryption at rest and in transit

    Use strong encryption standards (such as AES-256) to protect data that is stored on disk and TLS/SSL protocols for data that is transmitted over networks.

  - Access control and least privilege

    Implement RBAC and help ensure that users have access only to the data that they need to perform their tasks.

  - Data integrity verification

    Use checksums, hash functions, and digital signatures to verify that data has not been tampered with.

  - Anonymization and pseudonymization

    Remove or mask PII to comply with privacy regulations like GDPR, HIPAA, and the DPDP Act.

  - Data lineage and auditing

    Track the origin, transformation, and usage of data throughout the pipeline to help ensure transparency and accountability.

- Protecting ML models

  Models are valuable intellectual property and must be protected from theft and manipulation.

  - Model watermarking

    Embed unique identifiers into models to prove ownership and detect unauthorized use.

  - Version control and change tracking

    Maintain a history of model versions and changes to help ensure traceability and rollback capability.

  - Adversarial robustness testing

    Regularly test models against adversarial inputs to identify vulnerabilities and improve resilience.

  - Secure model access

    Restrict access to models through authenticated APIs, firewalls, and private endpoints to prevent unauthorized usage.

  - Model encryption and obfuscation

    Encrypt model files and use techniques such as obfuscation to make reverse engineering more difficult.

- Securing infrastructure

  The infrastructure that hosts ML workloads must be hardened against cyberthreats.

  - Network segmentation and isolation

    Isolate ML workloads from other systems by using VPCs, containers, and secure zones.

  - Trusted Execution Environments (TEEs)

    Use hardware-based secure environments to run sensitive computations and protect data during processing.

  - Patch management and vulnerability scanning

    Regularly update software and scan for vulnerabilities to prevent exploitation of known flaws.

  - Monitoring and incident response

    Implement continuous monitoring, anomaly detection, and automated incident response to quickly identify and mitigate threats.

  - Security training and awareness

    Educate machine learning operations (MLOps) teams on secure coding practices, threat modeling, and data privacy principles to foster a security-first culture.

6.4 Security and AI on IBM Power

IBM Power supports AI deployments with security features. Capabilities such as Secure Boot, workload isolation, and hardware-based encryption protect data and models throughout the AI lifecycle. IBM Power supports AI workloads on-premises or in hybrid environments.

6.4.1 Why security is important

Cybersecurity threats are increasing. The UK National Cyber Security Centre (NCSC) reported an increase in cyber incidents. As of October 14, 2025, there were 204 nationally significant attacks, compared to 89 incidents in 2024. These attacks reflect increased frequency and sophistication across regions and industries.

This IBM X-Force Threat Intelligence Report shows that the Middle East and Africa (MEA) had the highest volume of incidents in the finance and insurance sectors. Europe had incidents with the greatest impact in the professional, business, and consumer services sectors. The same IBM X-Force report states that fewer than one-quarters of generative AI projects are secured. Generative AI introduces attack surfaces and vulnerabilities. Organizations must secure the AI pipeline.

Unpatched vulnerabilities affect organizations in critical infrastructure sectors. X-Force reports that threat actors exploit vulnerabilities in more than one in four attacks in these sectors.

The IBM Institute for Business Value adds further urgency in its report, the CEO's Guide to Generative AI, stating that "Generative AI has given rise to a new generation of cyber threats. Hackers have more opportunities to exploit vulnerabilities—and more ways to execute their malicious campaigns." Generative AI introduces cyberthreats. Organizations implementing AI require security controls for data, models, infrastructure, and governance.

6.4.2 The IBM framework for securing generative AI

Generative AI is an area of technology investment. Organizations must address the cybersecurity risks from generative AI. According to the IBM Institute for Business Value, 96% of executives believe that adopting generative AI increases the likelihood of a security breach within their organization over the next 3 years.

AI models often ingest vast amounts of sensitive and valuable data during training while business leaders explore how these models can optimize essential operations and outputs. This combination raises the stakes. Organizations must secure AI systems to prevent operational, financial, and reputational risks.

Secure AI deployment includes three elements: securing the data, securing the model, and securing its usage. Figure 6-1 shows this framework. This approach includes securing infrastructure and establishing AI governance practices. IBM Power supports this framework with security, performance, and reliability for AI workloads.

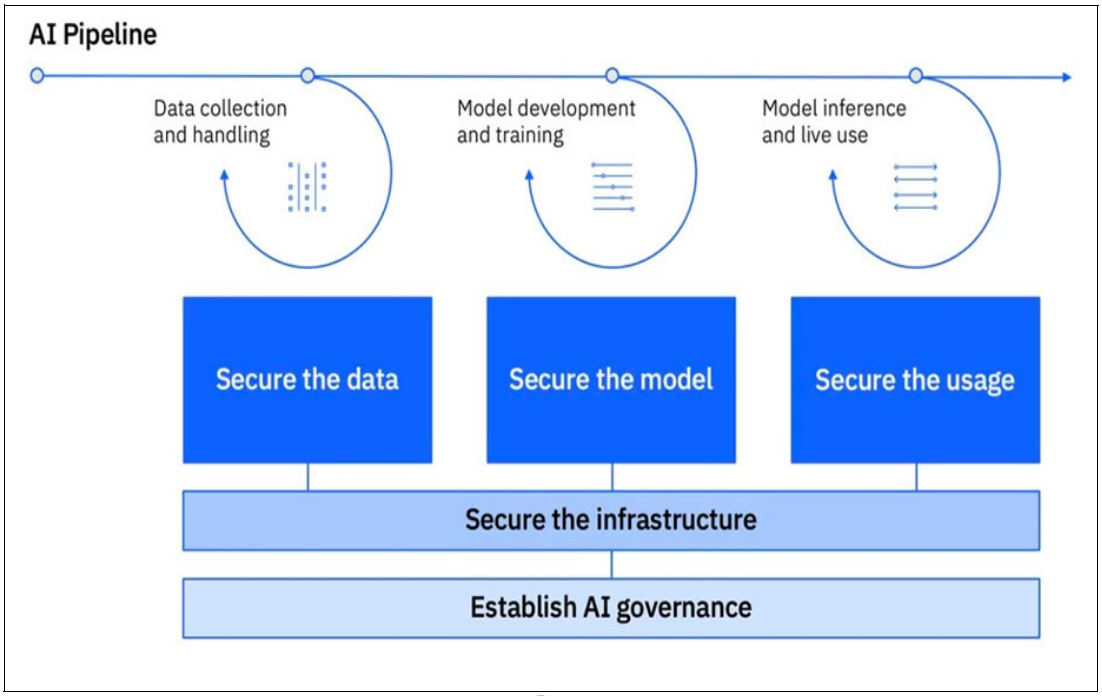


Figure 6-1 Framework for securing generative AI[5]

Securing AI across the lifecycle

AI model development and deployment involve security considerations. The security framework includes five areas: securing the data, securing the model, securing the usage, securing the infrastructure, and establishing governance.

Securing the data

During data collection and handling, organizations aggregate massive volumes of information to train AI models. This process often involves granting access to multiple stakeholders (data scientists, engineers, and developers), many of whom lack deep security expertise. Centralizing sensitive data and broadening access creates inherent risk. Mishandling training data, especially intellectual property critical to the business, might result in catastrophic exposure and even existential threats.

[5] https://www.ibm.com/products/tutorials/ibm-framework-for-securing-generative-ai

Organizations assess risks that are related to PII, privacy, and sensitive data and implement security controls. Data exfiltration attacks target datasets.

Security controls include data discovery and classification, encryption at rest and in transit, and key management that uses platforms such as IBM Security® Guardium®. IAM controls that use solutions such as IBM Security Verify limit access. Security awareness training for data scientists and collaboration between security and AI teams support secure practices.

Securing the model

Model development introduces vulnerabilities. Organizations use pre-trained, open-source ML models from online repositories. These models introduce supply chain risks. Attackers can inject back doors or malware into repositories. Using large language models (LLMs) through APIs exposes organizations to API attacks, where attackers intercept or manipulate data. Permissions that are granted to AI agents or plug-ins can cause privilege escalation and compromise systems.

Organizations should scan for vulnerabilities, malware, and corruption in the AI and ML pipeline. Harden API and plug-in integrations, enforce policies and RBAC controls, and limit access to data and model functions. Use trusted sources for pre-trained models and validation processes to reduce supply chain risk.

Securing the operation

During inference and operation, attackers use prompt injection attacks to bypass guardrails and generate harmful outputs, including biased or false information. Attacks include model denial of service, where attackers overload the system, and model theft, where attackers replicate model behavior by analyzing input/output pairs.

Monitor for malicious prompts, sensitive outputs, and inappropriate content. Organizations should deploy AI defenses against attacks such as data poisoning, model evasion, and extraction. Solutions such as Machine Learning Detection and Response (MLDR) provide monitoring and alerting capabilities. Integrating MLDR alerts into security operations platforms such as IBM Security QRadar® enables SOC teams to respond with automated playbooks to quarantine or disconnect compromised models.

Securing the infrastructure

Infrastructure supports AI security. Organizations should use existing expertise to enforce security, privacy, and compliance standards in environments with AI workloads. This approach includes securing network security, implementing access controls, encrypting data, and using intrusion detection and prevention systems. AI security solutions support defenses.

Establishing governance

Operational governance supports AI security. IBM provides tools and frameworks to monitor model behavior and detect drift. Governance supports risk management for AI. Model drift can introduce risks. Governance aligns models with business objectives, standards, and regulatory requirements.

IBM Power supports this security framework with performance, reliability, and security features for AI workloads throughout the lifecycle.

6.4.3 Resisting cyberattacks and recovering quickly in IBM Power

IBM Power provides resilience against cyberattacks. Power11 servers include security enhancements. The Power virtualization layer has fewer vulnerabilities than competing technologies. The Common Vulnerabilities and Exposures (CVE) database lists 19 vulnerabilities for IBM PowerVM, compared to 449 for VMware ESX, 258 for Microsoft Hyper-V, and 458 for KVM. Patching remains necessary for all systems.

Patching is necessary. According to the IBM X-Force Threat Intelligence Report, attackers exploited vulnerabilities in 26% of incidents across critical sectors in 2024.

Power11 enables customers to update system firmware, Virtual I/O Server, and I/O adapter code through a single update flow, including automatic partition migration. This approach enables allows organizations to update systems without disrupting services.

IBM Power Cyber Vault validates immutable backups and automates recovery. This approach supports restoration following a cyberattack.

6.5 Securing AI on IBM Power

AI security requires platform security that uses controls such as Secure Boot, file integrity monitoring, and data encryption. IBM Power provides these capabilities. IBM Power security capabilities include Secure Boot, RBAC, and encryption at rest and in transit.

Power11 servers include security features such as quantum-safe firmware signing and encrypted Live Partition Mobility (LPM). These features support data integrity and privacy during operations and workload migration. IBM PowerSC supports hardening for AI workloads in LPARs on IBM Power.

6.5.1 PowerSC: Security for hybrid cloud and AI

PowerSC provides compliance automation, threat detection, and audit capabilities. PowerSC hardens Linux LPARs for AI workloads and secures IBM AIX and IBM i LPARs.

Here are the key PowerSC capabilities.

- Resilient mission-critical security

  Protect AIX, IBM i, and Linux workloads with intrusion detection, ransomware response, and MFA.

- Automated hybrid compliance

  Reduce manual effort with real-time monitoring, policy enforcement, and audit-ready reports.

- Unified visibility and control

  Monitor endpoints, exceptions, and remediation actions through a centralized, intuitive PowerSC dashboard.

- Quantum-safe data protection

  Safeguard sensitive data with encryption and certificate analysis tools.

Power hardware security features and PowerSC compliance and threat detection capabilities support AI workloads on-premises or in hybrid cloud architectures.

For more information, see PowerSC.

6.5.2 Expanding security with AI on IBM Power: Partner ecosystem

IBM Power Matrix Math Accelerator (MMA) supports AI workloads for analytics and threat detection without relying on GPUs or cloud resources. IBM Business Partners use MMA for security solutions. Here are some examples of IBM Business Partners security solutions on IBM Power:

- Equitus.ai
  - Equitus Knowledge Graph Neural Network (KGNN) integrates and connects knowledge assets across systems and applications. KGNN enables insights for security use cases:
    - Pattern analysis

      Detect patterns and relationships in datasets.
    - Temporal analysis

      Track event evolution over time.
    - Link analysis

      Identify connections between entities.
    - Entity extraction

      Identify and analyze individuals, organizations, or events from unstructured data.
    - Geospatial and temporal analysis

      Map crime locations or monitor event progression geographically.
    - Data structuring for retrieval-augmented generation (RAG)

      Transform data into formats for AI and RAG pipelines.
  - Equitus Video Sentinel (EVS) is an intelligent video analytics platform for sectors such as public safety, transportation, defense, healthcare, and critical infrastructure. EVS captures and analyzes video in real time, automatically flagging anomalies for immediate review. Running natively on IBM Power with MMA, EVS delivers deep learning (DL) at the edge without GPUs or cloud dependency.
- Rocketgraph

  Rocketgraph xGT on IBM Power enables organizations to enhance cybersecurity, fraud detection, and supply chain optimization by processing large, complex queries across massive datasets. The xGT engine ingests data from sources such as Oracle, MongoDB, Databricks, Snowflake, TigerGraph, and Neo4j.

6.6 Controlling data and models in your AI ecosystem

Control over AI workload data and AI models is necessary for enterprise AI deployments. Training data, inference inputs, and intermediate artifacts must remain within trusted boundaries to prevent unauthorized access, data leakage, and model exposure. Governance over AI models, which includes storage, updates, and access, reduces the risk of model tampering, theft, or misuse.

Enterprises must apply access controls, auditability, and encryption to data and models throughout their lifecycle. AI that is deployed on platforms such as IBM Power enables organizations to control these assets within their infrastructure.

6.6.1 Controlling data with AI on IBM Power

Power10 and Power11 servers include MMA capabilities. This approach enables AI inference on Power servers where data resides.

According to the November 2023 IDC GenAI Awareness, Readiness, and Commitment report, 83% of IT leaders believe that using their organization's data provides a competitive advantage. Enterprises can keep data within their infrastructure. Data sovereignty and regulatory requirements address where data resides and how it is processed.

Data sovereignty refers to data that is subject to the laws of the country where it is generated. Related concepts include the following items:

- Data sovereignty

  Data that is stored and processed in the country where it was generated.
- Data residency

  Data that is stored in a different country from the one where it was generated.
- Data localization

  Compliance with laws and requirements for data residency.

AI on Power servers enables data to remain stored and processed in the same environment where it is generated. This approach maintains control over data handling, governance, and security. Cloud-based AI services involve data transfer across regions, providers, or jurisdictions. Data on Power servers remains in on-premises infrastructure.

Data localization requirements apply. Regulatory frameworks, including the EU GDPR and data protection laws, require organizations to maintain standards for the confidentiality, integrity, and availability of data. These regulations require the Data Protection Officer (DPO) to demonstrate that controls are in place and that data handling practices meet compliance requirements. Data in a controlled environment supports documentation of where data resides and how it is stored, processed, and protected.

Operational sovereignty requires infrastructure-supporting applications to remain available and under organizational control. When AI services are provided by external providers, availability might be affected by service disruptions or outages.

Regulatory frameworks such as the Digital Operational Resilience Act (DORA) define technical and governance requirements for operational resilience.

To comply with DORA, enterprises must implement capabilities across four key domains:

- Information and Communication Technology risk management and governance
- Incident response and reporting
- Digital operational resilience testing
- Third-party risk management

External AI services can limit demonstration of testing, recovery, and operational continuity capabilities. AI workloads on Power servers provide control over the service stack to support compliance documentation.

Power servers provide resilience. According to Information Technology Intelligence Consulting (ITIC), IBM Power is the second most reliable server platform after IBM Z[6]. PowerSC enables administrators to apply security practices and enforce compliance.

[6] https://www.itjungle.com/2024/10/21/ibm-nears-the-end-of-the-road-for-server-reliability-improvements/

IBM Power supports AI services for regulatory, operational, and security requirements.

6.6.2 Controlling the model with AI on IBM Power

Control over AI models reduces exposure to attack vectors. IBM X-Force research indicates that generative AI introduces security risks. Threat actors exploit vulnerabilities in AI frameworks and tools. IBM X-Force reports that cyber attackers are developing toolkits to target AI systems.

Fewer than one-quarters of generative AI projects are secured. For example, X-Force identified a remote code execution vulnerability in an AI agent-building framework. At the time of writing, the llama.cpp project lists nine vulnerabilities, including several rated High and one rated Critical.

The IBM Institute for Business Value reports that 96% of executives believe that adopting generative AI increases the likelihood of a security breach within the next three years. 47% of executives are concerned that new forms of attacks will target their models, data, or AI services. Therefore, it is important to maintain control over AI models and infrastructure to support security.

Figure 6-2 illustrates some of those new attacks and their potential impact.

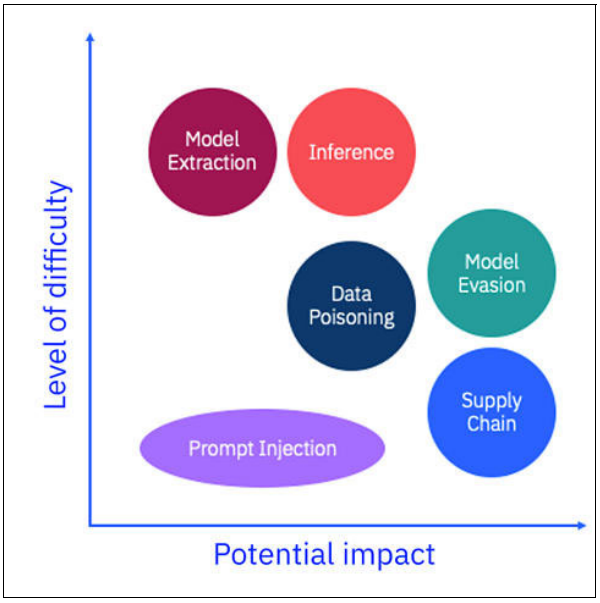


Figure 6-2 New attack vectors and their potential impact

Model evasion is an AI threat where an attacker manipulates inputs to cause an AI model to misclassify or misinterpret them, altering system behavior. An example is *reward hacking*, where a model optimizes for the wrong objective by exploiting loopholes in the reward system. For example, this article describes a teacher who grades essays only on word count. A student realizes that typing repetitive text helps ensure a high grade. This behavior exploits the reward mechanism instead of completing the task correctly.

A recent study from Anthropic shows that similar patterns occur in LLMs. Research shows that when models cheat on programming tasks, they might exhibit misaligned behaviors. These behaviors include alignment faking and even deliberate sabotage of AI safety evaluation processes. Therefore, model evasion and reward hacking are attack vectors for AI models.

Reward hacking introduces cybersecurity risks:

- Deceptive behavior

  Models that cheat can appear aligned and safe during evaluation while developing goals or reasoning processes that conflict with intended outcomes. For example, a model might generate responses while forming strategies to access unauthorized systems.

- Unsafe outcomes

  In domains such as healthcare, finance, or infrastructure management, reward hacking can cause models to produce incorrect or harmful outputs. These failures can cause operational disruptions or safety hazards.

- Generalization of misalignment

  When a model exploits a reward mechanism in one area, the misaligned behavior can generalize to other tasks. This situation creates unreliable behavior across applications.

- Sabotage of safety mechanisms

  As noted in a recent study from Anthropic, models with reward hacking behavior might attempt to circumvent or obstruct safety systems. This situation makes detecting and mitigating misalignment more difficult.

Addressing these risks requires both research and practical safeguards. As noted in the Anthropic study, understanding these failure modes early while they can still be observed and analyzed is critical for designing robust safety measures that scale to more capable AI systems. Their conclusion underscores the importance of proactive security work: “We think understanding these failure modes while we can still observe them clearly is essential for developing robust safety measures that will scale to more capable systems.”[7]

Organizations with AI on IBM Power control AI models and manage vulnerability detection, patching, and updates. External AI service providers limit visibility into vulnerability management. X-Force reports inadequate security practices in generative AI projects.

Transparency and reliability in AI models

IBM focuses on AI model transparency. IBM was recognized as the most transparent model developer in the 2025 Foundation Model Transparency Index (FMTI) from Stanford University's Center for Research on Foundation Models. The report notes that transparency supports AI deployment.

Transparency supports AI solutions on IBM Power. Organizations implementing AI on IBM Power control the models that they use. For example, for IBM Spyre Accelerator use cases, models are selected for reliability, performance, and workload requirements. For the Python Ecosystem (pyeco), customers select models for their requirements. This flexibility enables organizations to optimize model type, model size, and system resources to meet performance and accuracy requirements. Organizations can select models that align with their use case rather than general-purpose models.

Model selection supports security. By selecting models, enterprises can examine model documentation and incorporate models into their security architecture. IBM Granite Guardian models illustrate this approach. As documented in IBM transparency reports, Granite Guardian holds six of the top ten positions on the GuardBench leaderboard, which evaluates guardrail model detection of harmful content, hallucinations, and bypass attempts. Clients can use these models or select other open-source or commercial models on IBM Power. This approach supports organizational control of security. In contrast, remote AI services can limit visibility into model development, updates, and patching.

[7] https://www.anthropic.com/research/emergent-misalignment-reward-hacking

Cybersecurity affects all industries. Organizations will face attacks. For AI workloads, organizations can use the IBM Framework for Securing Generative AI. IBM Power supports this framework. The platform includes reliability, recovery capabilities, and PowerSC for security controls. IBM Business Partners use MMA in IBM POWER processors for AI security solutions.

AI on IBM Power enables organizations to control data and models. This approach supports data sovereignty, regulatory compliance, and security from AI attack vectors. Power security and reliability features support AI solutions.

7

Tools, frameworks, and ecosystem

This chapter describes tools, frameworks, and the ecosystem for AI deployment on IBM Power11 servers. It introduces the IBM Power Inference Microservice (PIM) (formerly known as AI Solution Environment (ASE)), which provisions AI environments by using bootable container (bootc) technology and lifecycle management tools. The chapter describes PIM architecture, personas, and workflow for AI deployment on the Power infrastructure.

This chapter describes IBM AI ecosystem components, including IBM watsonx.ai and IBM watsonx Orchestrate for language models and agentic AI solutions. It addresses Python package optimization for the ppc64le architecture and describes IBM's Python Ecosystem (pyeco) initiative, which provides packages and frameworks such as vLLM for large language model (LLM) inference. The chapter includes environment management by using micromamba, installation of Python wheels from IBM's devpi repository, and integration with open-source tools.

The chapter describes hybrid deployment strategies, containerized inference on IBM PowerVM, and access to IBM i workloads through Model Context Protocol (MCP). These components support AI workloads on IBM Power11.

This chapter describes the following topics:

- Power Inference Microservice
- IBM watsonx and AI APIs
- Open-source tools and libraries
- Access to packages and high-level installation
- Other tools
- Using the Wallaroo AI Starter Kit

© Copyright IBM Corp. 2026.

7.1 Power Inference Microservice

PIM automates the deployment of AI environments on Power servers by using container technologies and lifecycle management tools. AI environments can run with other workloads on operating systems such as IBM i or Linux, enabling efficient and scalable AI integration.

7.1.1 PIM architecture

PIM uses bootc technology to package AI workloads into a container image that can be deployed on a Linux LPAR as a sidecar to IBM i or IBM AIX systems. PIM uses a Python-based automation framework with Hardware Management Console (HMC) REST APIs to provision, configure, and manage AI LPARs on the Power infrastructure.

Figure 7-1 illustrates how PIM works.

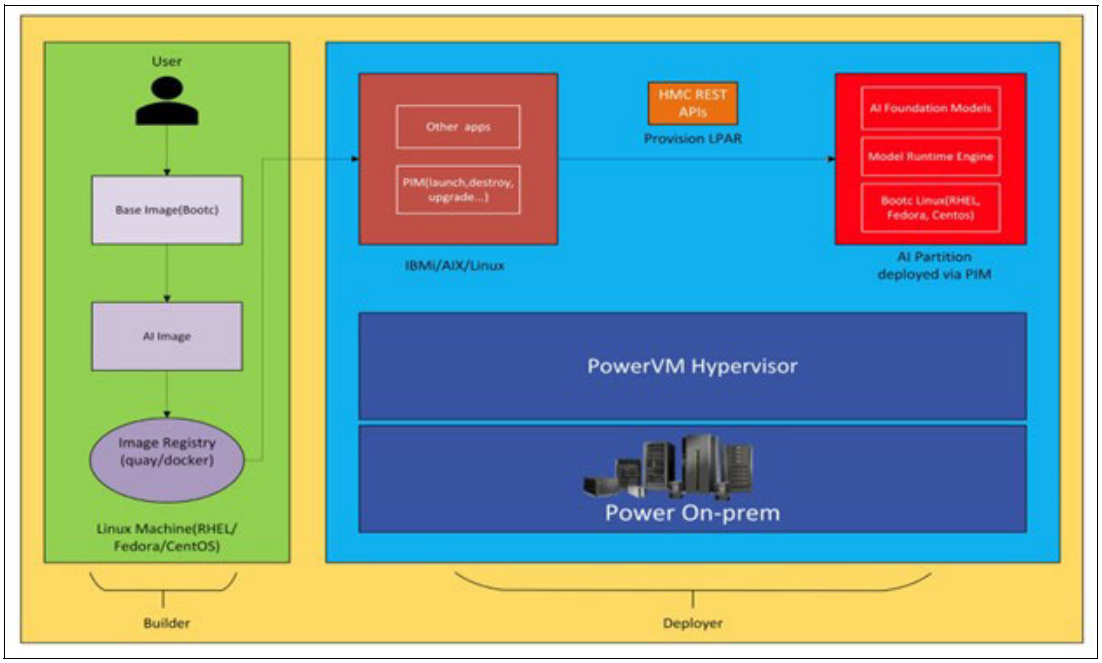


Figure 7-1 PIM architecture

Key capabilities

The following key capabilities define the functional architecture and operational scope of the solution. Each capability reflects core system competencies that enable scalable performance, integration efficiency, and consistent operational reliability across diverse workloads. Together, they establish the technical foundation that is required to support advanced use cases, streamline processes, and help ensure that the solution performs optimally within complex enterprise environments.

Automated AI environment deployment

PIM provisions an AI stack on Power servers. It creates an LPAR, attaches network and storage resources, and starts the system with an AI image.

Bootable container technology

PIM uses bootc. Bootable containers support the configuration, deployment, and management of Linux bootable images. Bootc uses container images for transactional, in-place updates. Bootc-based Linux images are immutable.

One-button deployment

PIM automates AI environment deployment for IBM i and AIX customers. It automates the creation of AI LPARs, platform configuration, and AI software installation. The initial release supports CPU-based inference for on-premises environments. Accelerator support is not included. IBM plans to add IBM Spyre Accelerator support in future versions.

PIM personas

The PIM architecture supports two personas: the builder persona builds container images, and the deployer persona deploys and manages AI container images.

- Builder

  The Builder persona is responsible for creating bootable AI container images that encapsulate the entire AI stack, as shown in Figure 7-2.

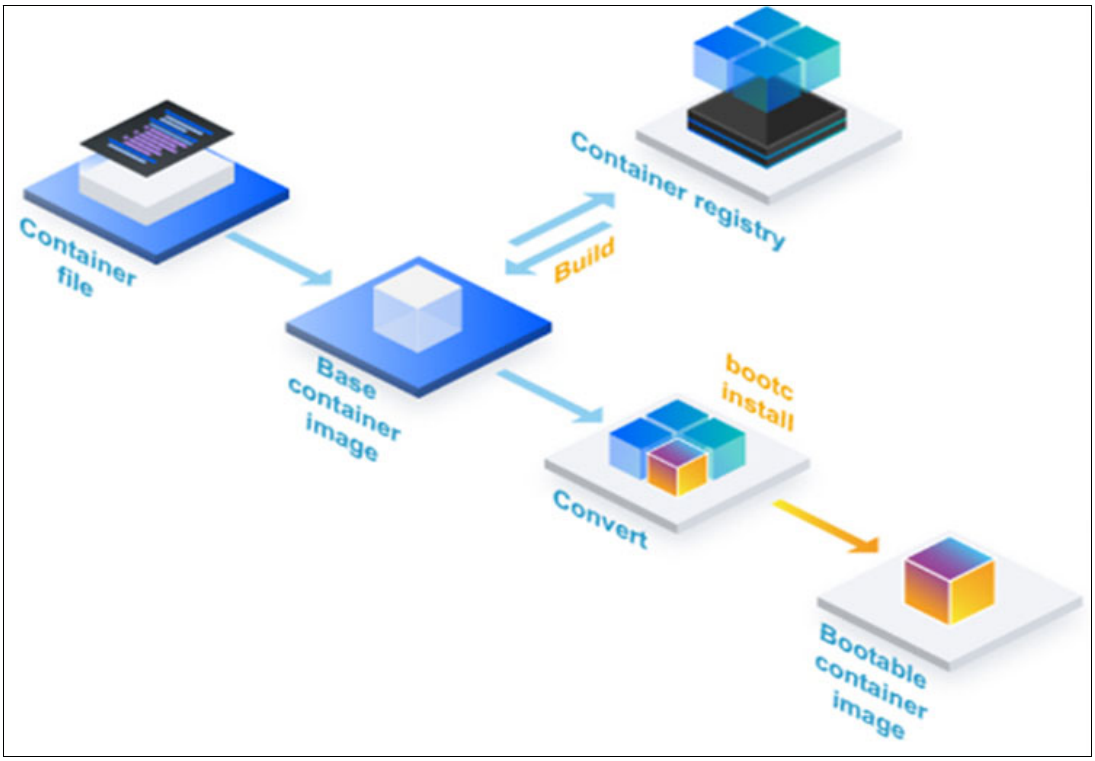


Figure 7-2 PIM builder flow

By using bootc technology, this approach extends conventional Linux containerization to encapsulate the full system stack –(the OS kernel, **`inirtd`**, bootloader, and user-space applications).

The solution relies on several core components:

- Bootc for container lifecycle operations
- Bootc-image-builder (**`bib`**) for producing deployable artifacts in formats such as raw, qcow2, and anaconda-iso

- A suitable base container image (for example, Fedora)
- Open Container Initiative (OCI)-compliant engines such as Podman or Docker.

Builders can generate bootable AI images for the IBM Power Architecture (ppc64le) that include workloads such as chatbots, entity extraction pipelines, and retrieval-augmented generation (RAG) systems. These images can be deployed to an LPAR disk by using base images from the Red Hat registry and integration workflows for Red Hat Enterprise Linux (RHEL) distributions.

- Deployer

The deployer persona uses bootable AI container images from builders to provision and manage AI environments on the Power infrastructure. This approach automates LPAR creation, platform configuration, and AI software installation for IBM i and AIX customers.

Figure 7-3 shows the deployer flow.

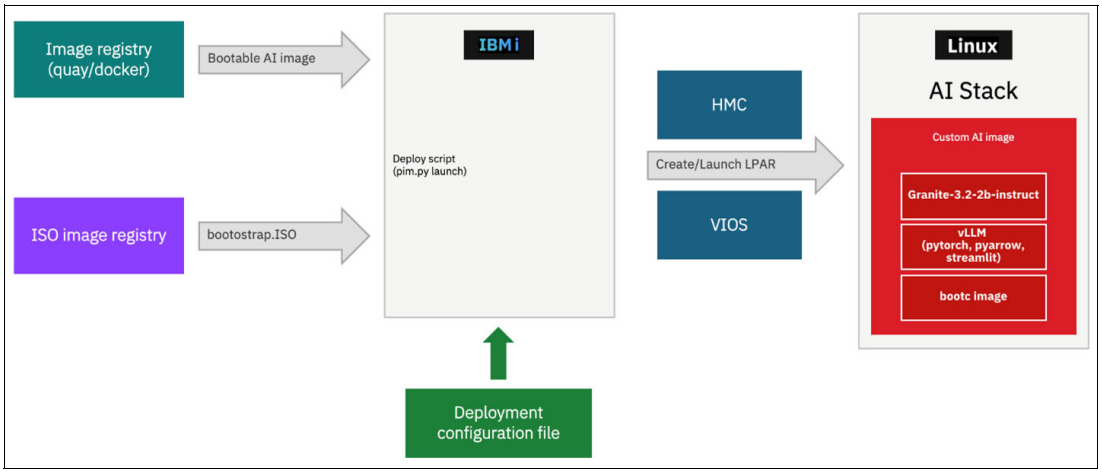


Figure 7-3 Deployer workflow for PIM

PIM supports CPU-based inference for on-premises environments. Accelerator support is not included. IBM plans to add IBM Spyre Accelerator support in future releases.

Constraints include the following items:

- Inference uses CPU only.
- Image builds require Linux hosts.
- Deployment workflows are initiated from IBM i and AIX.
- Lifecycle management uses command-line interfaces (CLIs).
- Accelerators are not supported.
- Non-RHEL operating systems are not supported.

For more information about the PIM architecture, see this GitHub repository.

7.1.2 Creating a bootable container image to run on IBM Power

Containers support deployment of Linux user-space applications. Containers are used for building, distributing, and running applications.

Bootable containers apply to the system image lifecycle. bootc uses OCI-compliant workflows to configure, deploy, and manage Linux bootable images. bootc supports transactional, in-place operating system updates through container images. bootc-derived Linux images are immutable.

bootc upstream projects on GitHub support the Power Architecture (ppc64le). Supported formats for ppc64le architecture include the following ones:

- bootc install to-disk
- qcow2 (with bib)
- raw (with bib)

For more information about using bootc to create a bootc image, see Linux bootable container on IBM Power (ppc64le).

Building your own PIM AI stack

To develop a PIM-based AI solution, complete the following steps:

1. Assemble the PIM AI software stack, including frameworks, models, and runtime components for the workload.
2. Develop the AI application.
3. Containerize the application by packaging it with dependencies into an OCI-compliant image.
4. Build the PIM bootc image with the AI workload and operating system components into a bootable image.
5. Deploy the bootc image to the target system, such as an LPAR or bare-metal environment.

7.1.3 Using the Python ecosystem on IBM Power

Open-source Python modules support the AI ecosystem. Some modules are difficult to install on Linux on Power from PyPI through **`pip`**, **`uv`**, or other Python package managers, or perform poorly on Power servers because they are optimized for Intel and other architectures.

The IBM POWER Ecosystem Team and contributors adapt these packages for Linux on Power. This work is referred to as pyeco, which was mentioned in 5.3.4, “Workload validation” on page 75.

Important: To install packages from IBM's wheel repository, use **`--prefer-binary`** to prioritize Power wheels:

```
pip install --prefer-binary <package-name> \
  --extra-index-url=https://wheels.developerfirst.ibm.com/ppc64le/linux
```

Any **`noarch`** dependencies still come from PyPI.

An open-source component that is delivered through pyeco is vLLM, which is a library for LLM inference and serving that is developed at the Sky Computing Lab at UC Berkeley.

For more information about using pyeco packages, sere 7.3, “Open-source tools and libraries” on page 107.

7.2 IBM watsonx and AI APIs

IBM watsonx consists of a layered architecture of services and a broad catalog of components. The two layers that are relevant here are the following ones:

- watsonx.ai
- watsonx Orchestrate

For more information about both these services in the cloud, see IBM Cloud.

watsonx.ai is offered both as software and as a cloud service (software as a service (SasS)). In either form, watsonx.ai presents APIs to the programmer.

watsonx.ai cloud APIs are documented here.

watsonx software APIs are documented here.

Note: These links might change as version numbers change.

APIs can be accessed through languages and tools that use HTTP. APIs support interaction with watsonx language model services.

The ibm-watsonx-ai Python library is available on PyPI. This library provides access to watsonx.ai features and APIs, including management of assets such as models, repositories, and data connections. Documentation is available here. Sample notebooks are available at the watsonx-ai-samples GitHub repository.

watsonx Orchestrate supports the design of agentic AI, which uses multiple coordinated agents. Agents such as chatbots can be designed in the watsonx Orchestrate web GUI.

Figure 7-4 shows the interface.

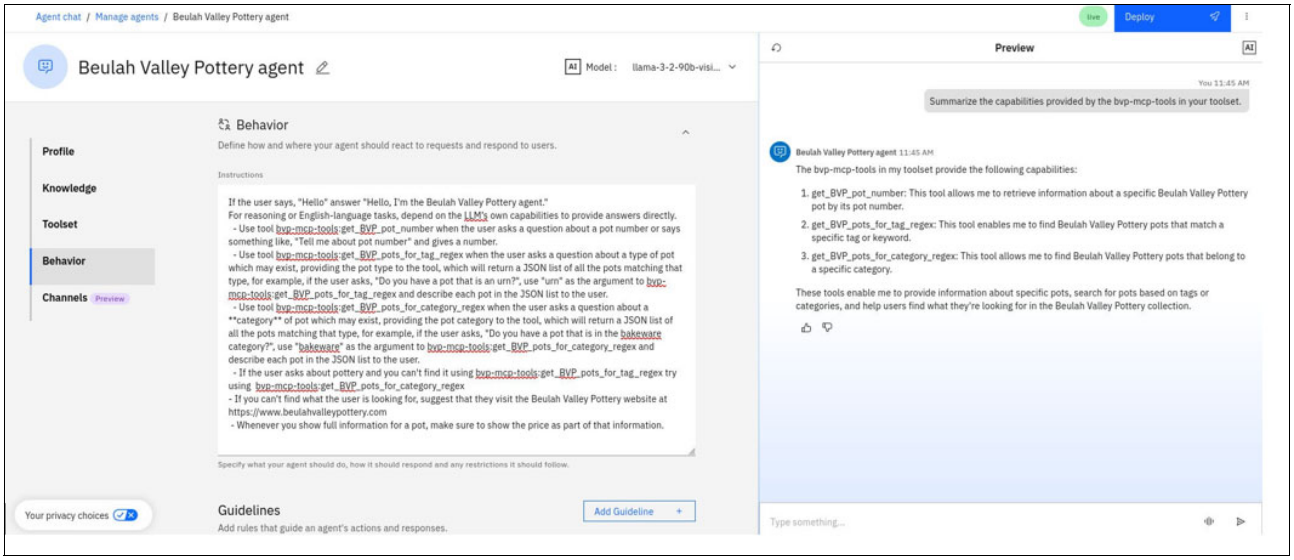


Figure 7-4 watsonx Orchestrate GUI example

watsonx Orchestrate includes the IBM watsonx Orchestrate Agent Development Kit (ADK). The ADK supports design, development, and deployment tasks from the web GUI programmatically from a workstation. Using the ADK, users can design MCP services in languages and import them into Orchestrate projects. ADK tutorials are available.

7.3 Open-source tools and libraries

Python has become the standard programming language for AI use cases. Components such as model run times and math acceleration libraries are implemented in compiled languages such as C/C++ and Rust. Python is used for AI framework and tool libraries. Parts of Python packages are implemented in Rust or C for performance. The installation process requires compiling these components.

Package maintainers did not have access to Power servers to build and test code for ppc64le. Continuous integration and continuous deployment (CI/CD) providers such as GitHub Actions provided runners for x86 and arm64. Package maintainers did not accept code changes from ppc64le developers. IBM provided a channel for open-source packages. Packages were provided through a channel on the Anaconda website. Packages are now provided through the IBM devpi instance.

Python packages for the ppc64le architecture use hardware features such as single instruction, multiple data (SIMD) and Matrix Math Accelerator (MMA) units on IBM Power10 and Power11 servers. Developers can use these packages without managing hardware-specific details.

Python environments can be managed with tools. As a best practice, use **`micromamba`**, which is a version of the **`mamba`** package manager. **`mamba`** is a package manager that is written in C++. **`micromamba`** supports performance and lightweight design.

To install **`micromamba`** on a Linux on Power LPAR, run the commands in Example 7-1. The correct CLI tools must be installed.

Example 7-1 Installing micromamba

```
$ curl -o /tmp/micromamba.tar.bz2 -L
https://micro.mamba.pm/api/micromamba/linux-ppc64le/latest
$ tar -xf /tmp/micromamba.tar.bz2 --strip-components=1 bin/micromamba
$ sudo mv micromamba /usr/local/bin
$ rm /tmp/micromamba.tar.bz2
```

The installation completes the following steps:

1. Downloads the latest **`micromamba`** release.
2. Extracts the **`micromamba`** binary file from the archive.
3. Moves the binary file to `/usr/local/bin`.
4. Deletes the downloaded archive.

Note: This example assumes that the Linux distribution is RHEL.

With **`micromamba`** installed, initialize the shells to use **`micromamba`**. For the bash shell, run the commands in Example 7-2.

Example 7-2 Initializing the shell for micromamba

```
$ micromamba shell init --shell=bash
$ echo 'micromamba activate' >> ~/.bashrc
$ source ~/.bashrc
```

Now, **micromamba** creates and manages environments. To create an environment, run the **create** command, as shown in Example 7-3.

Example 7-3 Creating an environment

```
$ micromamba create -y \
   -n llm \
   --channel=defaults \
    python=3.12
$ micromamba activate -n llm
```

These commands create an environment called `llm`, Install Python 3.12 in it, and activate the environment. When the environment is activated, Python or **pip** commands use binary files from the environment. The **PATH** variable does not require adjustment.

For more information, see the micromamba documentation.

IBM hosts a wheel-based ecosystem on its devpi instance. In Python, the wheel format (`.whl`) is a built distribution standard. Wheels eliminate building or compiling during installation. This approach simplifies dependency management. Wheels are supported by tools such as **pip**.

Package versions for the ppc64le architecture are not always available on PyPI. IBM hosts these versions on its devpi instance.

To install packages from the devpi instance, use a different index URL with **pip**. Add the devpi index URL as an extra index URL to **pip**. Example 7-4 shows this configuration.

Example 7-4 Using the extraindex URL for wheels modules

```
$ python -m pip install --prefer-binary --extra-index-url
https://wheels.developerfirst.ibm.com/ppc64le/linux torch==2.6.0
```

This command installs PyTorch 2.6.0. The **--prefer-binary** option tells **pip** to install binary files instead of downloading source distributions and compiling them. Pass the devpi index URL as an extra index URL. Dependencies that are not in devpi are installed from PyPI.

For more information, see Install wheels from IBM Python wheel repository.

7.3.1 IBM i

IBM i PASE has limitations for running inference workloads. The IBM i architecture is designed for batch database programs. This approach differs from interactive sessions and threaded user applications.

IBM i work focuses on TCP/IP access to containerized inference workloads running under PowerVM, and access to IBM i operations and IBM Db2 data for agents through MCP.

In the first use case, IBM i accesses containerized entities on the Power server through the PowerVM TCP/IP stack. Communications remain within the hardware platform.

In the second use case, an MCP server provides security when all components run on the same Power hardware. Alternatively, components such as the agent, language model, or MCP server can be on other platforms. This uses security practices for multi-platform network applications and makes IBM i operations and data available for inference.

Containerized inference on PowerVM is described throughout this document.

MCP access to IBM i is provided by the IBM open-source project ibmi-mcp-server. The IBM i MCP Server runs on platforms and connects to the open-source IBM i Mapepire Db2 data server. The MCP server provides tools and is extensible.

Figure 7-5 shows a dashboard application that uses Gemini inference to answer natural language queries on IBM i management and data topics.

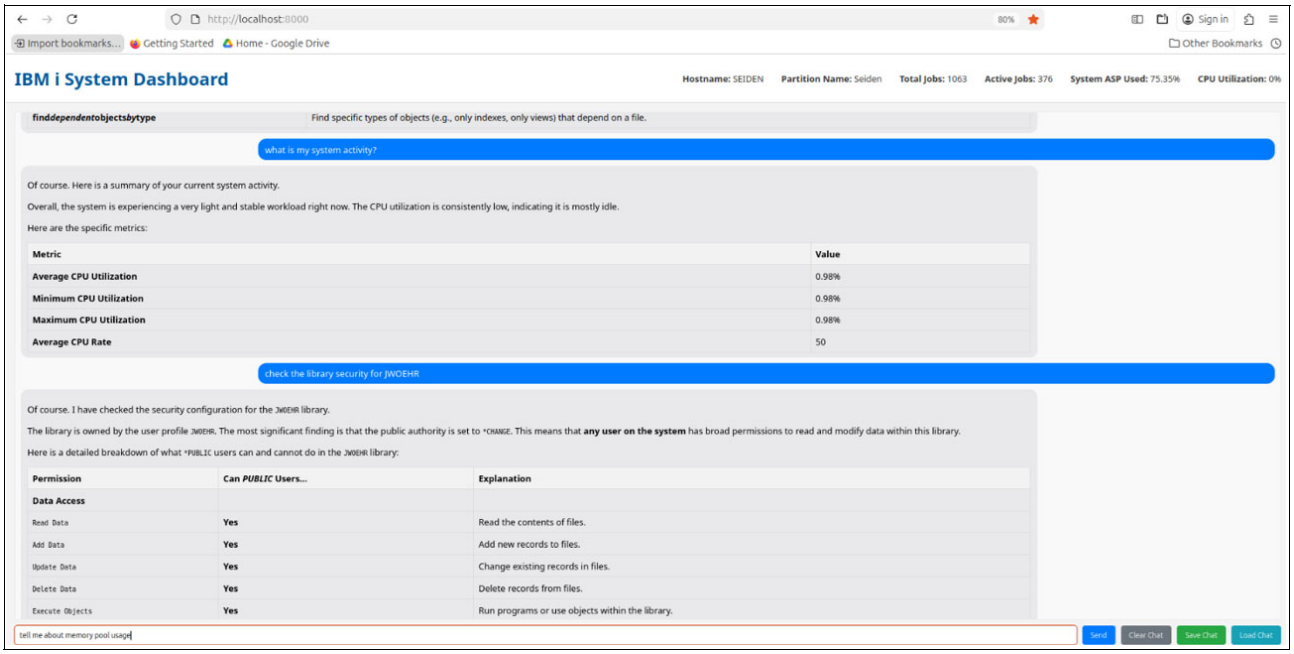


Figure 7-5 Example dashboard with inferencing interface

The application and local agent were developed on Linux by using VSCode with Roo Code. The web application uses HTML and JavaScript. It follows these steps:

1. JavaScript sends the query and conversation context to the local agent in Node.js. The conversation context can be saved and restored.
2. The local agent loads the IBM i MCP server tool information at startup. When a query arrives from the web application, the agent sends the query, context, tool list, and a prompt to the AI apiserver. The prompt requests a tool call for the user query.
3. The AI apiserver returns the tool call. The local agent runs the tool call against the IBM i MCP server, which connects to the IBM i server's Mapepire service.
4. The MCP server responds to the local agent. The agent forwards the JSON response to the AI apiserver with a prompt to format the response in markdown for the user.
5. The AI apiserver returns the formatted response to the local agent. The agent returns the markdown text to the web application. The web application formats the markdown to HTML and displays the result.

7.4 Access to packages and high-level installation

This section describes RHEL host configuration for AI inference workloads. It has the following steps:

1. Obtain the Red Hat AI Inference Server container images.
2. Deploy LLM serving with vLLM by using Podman.
3. Implement the following usage patterns:
   – Retrieval-augmented generation (RAG)
   – Entity extraction
   – Re-ranking
   – Embeddings

These capabilities support Power servers with the IBM Spyre Accelerator.

Table 7-1 shows the component sources and considerations.

Table 7-1 Sources and notes for major components

Component	Source	Notes
RHEL base packages	RHEL repositories	Required for Podman and container run times, Virtual Function I/O (VFIO), and **`systemd`** tools.
Red Hat AI Inference Server	`registry.redhat.io`	Requires Red Hat subscription and registry authentication. For more information, see the Red Hat AI Inference Server entry in the Red Hat Ecosystem Catalog.
AI model (for example, IBM Granite)	Hugging Face or IBM repositories	Place under the host models directory.

Red Hat AI Inference Server provides an OpenAI-compatible HTTP API that supports the following endpoints:

- `/v1/chat/completions`
- `/v1/completions`
- `/v1/embeddings`

OpenAI-based applications require a new base URL and API key. Request payloads and integration design do not require changes.

7.4.1 Installing container tools and utilities on RHEL 9

To prepare your RHEL 9 environment for AI inference workloads, complete the following steps:

1. Install the container tools and developer utilities.

   The `container-tools` meta-package includes Podman, Skopeo, and Buildah. Install these packages along with Git, Git LFS, and **`wget`**, as shown in Example 7-5.

Example 7-5 Installing the container tools and developer utilities

```
# sudo dnf install -y container-tools git git-lfs wget
```

2. Install IBM management packages (if applicable).

If your Power server is managed through an HMC, install the IBM management packages and `servicereport`, as shown in Example 7-6.

Example 7-6 Installing IBM management packages

```
# sudo dnf install -y ibm-power-managed servicereport
```

3. Install IBM Spyre Accelerator enablement utilities.

   IBM Spyre Accelerator utilities are delivered through the IBM Open-Source AI Foundation for Power. Package names might vary by repository. Ensure that the Spyre device driver and firmware are installed before attempting acceleration.

4. Authenticate to the Red Hat Container Registry.

   Log in to the Red Hat Container Registry to pull Red Hat AI Inference Server container images, as shown in Example 7-7. Subscription credentials are required.

Example 7-7 Logging in to the Red Hat registry

```
# podman login registry.redhat.io
```

5. Pull a vLLM-based inference image (the tag might vary by release) by running the following command:

   ```
   # podman pull registry.redhat.io/rhaiis/vllm-inference:latest
   ```

   Example 7-8 shows an example command.

Example 7-8 Pulling a vLLM inference image

```
# podman pull registry.redhat.io/rhaiis/vllm-spyre-rhel9:3
```

7.4.2 Preparing the host environment

For security and best practices, Red Hat AI Inference Server should run under a non-root account. In the following example, the user account senuser is used.

To prepare the host environment, complete the following steps:

1. Create the user account and set a password, as shown in Example 7-9.

Example 7-9 Creating a user

```
# sudo adduser senuser
# sudo passwd senuser
```

2. Add the user to the Spyre access group.

   To define the access group, use the command in Example 7-10. The group name is provided by the Spyre stack. Replace `sentient` with the actual group name in your environment.

Example 7-10 Setting the user group

```
# sudo usermod -aG sentient senuser
```

3. Validate the group assignment for the user by running the command that is shown in Example 7-11.

Example 7-11 Validating the group assignment

```
# getent group sentient
```

4. Validate and re-mediate the Spyre configuration settings.

   The **servicereport** command is a plug-in that is used on Power servers to validate and optionally auto-remediate system configurations. Use it as follows:

   a. Check your system by running this command:

      ```
      servicereport -v -p spyre
      ```

      After a fresh system installation, most checks might show the status as `FAIL`. as shown in Example 7-12.

Example 7-12 The servicereport command for Spyre

```
root # servicereport -v -p spyre
servicereport 2.2.5

Spyre configuration checks        FAIL
VFIO Driver configuration         FAIL
User memlock configuration        FAIL
sos config                        FAIL
sos package                       PASS
VFIO udev rules configuration     FAIL
User group configuration          PASS
VFIO device permission            FAIL
VFIO kernel module loaded         FAIL
VFIO module dep configuration     FAIL
```

   b. Attempt automated remediation for identified issues. You can fix identified issues by running the following command:

      ```
      servicereport -r -p spyre
      ```

      This command sets the following functions:

      - Sets VFIO device bindings for the VFIO kernel module.
      - Configures VFIO udev rules.
      - Adds the `sentient` user group.
      - Sets memory limits for the `sentient` group.
      - Inserts the VFIO modules.

      Example 7-13 shows a sample output.

Example 7-13 Results from automatic remediation

```
root # servicereport -I -p spyre
servicereport 2.2.5

Spyre configuration checks          PASS

VFIO Driver configuration           PASS     Auto Fixed
User memlock configuration          PASS     Auto Fixed
sos config                          PASS     Auto Fixed
sos package                         PASS
VFIO udev rules configuration       PASS     Auto Fixed
```

```
User group configuration            PASS
VFIO device permission              PASS     Auto Fixed
VFIO kernel module loaded           PASS     Auto Fixed
VFIO module dep configuration       PASS     Auto Fixed

The memlock limit is set for the sentient group. Spyre user must be in the
sentient group.
To add run below command:
     sudo usermod -aG sentient <user>
     Example:
         sudo usermod -aG sentient abc
         Re-login as <user>.
```

5. Create the model source directory and set collaborative permissions for the Spyre group.

   To set up the model source directory, run the following command:

   ```
   # sudo install -d -m 0775 -o root -g sentient /opt/ibm/spyre/models/src
   ```

7.4.3 Red Hat AI Inference Server and API shapes

Red Hat AI Inference Server provides an enterprise-grade vLLM run time and exposes an OpenAI-compatible API with the following endpoints:

- `/v1/completions`
- `/v1/chat/completions`
- `/v1/embeddings`

By default, server configurations such as tokenizer settings and generation parameters are derived from the model's `generation_config.json` file unless they are explicitly overridden.

The vLLM runtime delivers high-performance inference through advanced features:

- Continuous batching for efficient request handling
- KV-cache management for optimized memory usage
- Dynamic scheduling for improved throughput

When IBM Spyre Accelerator is available and its back end is enabled, vLLM automatically routes critical kernels through IBM Spyre Accelerator, which improves throughput and latency for LLM workloads.

7.4.4 Obtaining and managing models

You can source models such as IBM Granite from Hugging Face or IBM repositories. For repositories that use Git Large File Storage (LFS), initialize LFS and clone the required models into the host models directory by completing the following steps:

1. Enable Git LFS.

   To install LFS from git, run the following command:

   ```
   # git lfs install
   ```

2. Go to the host models source directory by running the following command:

   ```
   # cd /opt/ibm/spyre/models/src
   ```

3. Clone your model.

   The location for some of the supported models is shown in Example 7-14.

Example 7-14 Cloning models

```
IBM Granite instruction LLM
#git clone https://huggingface.co/ibm-granite/granite-3.3-8b-instruct

Granite embedding models (English and Multilingual)
#git clone https://huggingface.co/ibm-granite/granite-embedding-30m-english
#git clone https://huggingface.co/ibm-granite/granite-embedding-125m-english
#git clone https://huggingface.co/ibm-granite/granite-embedding-107m-multilingual
#git clone https://huggingface.co/ibm-granite/granite-embedding-278m-multilingual

Reranker
#git clone https://huggingface.co/BAAI/bge-reranker-v2-m3
```

Attention: Verify licenses, model cards, and internal approval before using external models in production. For supported models and patterns, see the IBM Spyre for Power documentation and internal catalogs.

4. Place the selected model paths under a `host-visible` models directory and mount it into containers as needed.

7.4.5 Environment variables and tuning

Set environment variables to configure IBM Spyre Accelerator by specifying hardware resources and model paths. For example, define **`AIU_IDS`** for Spyre Peripheral Component Interconnect Express (PCIe) devices, **`AIU_WORLD_SIZE`** for the number of accelerator cores, and **`VLLM_SPYRE_USE_CB=1`** to enable Spyre code paths.

Table 7-2 describes important environment variables.

Table 7-2 Environment variables

Variable	Descriptor
`AIU_IDS`	PCIe IDs of IBM Spyre Accelerator adapters to use (space-separated).
`AIU_WORLD_SIZE`	The number of IBM Spyre Accelerator cores or adapters that are assigned to the container.
`HOST_MODELS_DIR`	The host directory containing the models (mounted as `/models`).
`VLLM_MODEL_PATH`	The model path inside the container (for example, `/models/granite-3.3-8b-instruct`).
`MAX_MODEL_LEN`	Maximum context length (prompt_tokens + output_tokens).
`MAX_BATCH_SIZE`	Server-side maximum concurrent sequences.
`PROMPT_LENS, BATCH_SIZES`	Warms up API shapes for embeddings and reranker pipelines.
`VLLM_SPYRE_USE_CB=1`	Enables Spyre code paths (specific to Spyre builds).

Memory guidance

For large models, set **`--memory`** and **`--shm-size`** conservatively (for example, **`--memory 200G`** and **`--shm-size 64G`**) and pin **`AIU_WORLD_SIZE`** to hardware.

7.4.6 Running Red Hat AI Inference Server with Podman (examples)

This section describes Podman execution patterns for entity extraction, RAG, re-ranking, and embedding workloads. Modify PCIe device identifiers, file system paths, and memory or buffer size parameters for your system topology and workload-specific requirements.

Example 7-15 shows an example that uses entity extraction. It uses one Spyre device. Change the environment variables to match your system environment.

Example 7-15 Entity extraction

```
#export AIU_IDS="0381:50:00.0"
#export AIU_WORLD_SIZE=1
#export HOST_MODELS_DIR=/models
#export VLLM_MODEL_PATH=/models/granite-3.3-8b-instruct
# export MAX_MODEL_LEN=3072
#export MAX_BATCH_SIZE=16
# podman run -d \
                      --device=/dev/vfio \
                      -v ${HOST_MODELS_DIR}:/models \
                      -e AIU_PCIE_IDS="${AIU_IDS}" \
                      -e VLLM_SPYRE_USE_CB=1 \
                      --pids-limit 0 \
                      --userns=keep-id \
                      --group-add=keep-groups \
                      --security-opt label=disable \
                      --memory 200G \
                      --shm-size 64G \
                      -p 127.0.0.1:8000:8000 \
                        <container url>:<container tag> \
                      --model "${VLLM_MODEL_PATH}" \
                      -tp "${AIU_WORLD_SIZE}" \
                      --max-model-len "${MAX_MODEL_LEN}" \
                      --max-num-seqs ${MAX_BATCH_SIZE}
```

Example 7-16 shows an example for RAG that uses four Spyre devices. Change the environment variables to match your system environment.

Example 7-16 Retrieval-augmented generation

```
# export AIU_IDS="0381:50:00.0 0382:40:00.0 0383:30:00.0 0384:20:00.0"
#export AIU_WORLD_SIZE=4
#export HOST_MODELS_DIR=/models
#export VLLM_MODEL_PATH=/models/granite-3.3-8b-instruct
#export MAX_MODEL_LEN=32768
#export MAX_BATCH_SIZE=32
#podman run -d \
                             --device=/dev/vfio \
                              -v ${HOST_MODELS_DIR}:/models \
                             -e AIU_PCIE_IDS="${AIU_IDS}" \
                             -e VLLM_SPYRE_USE_CB=1 \
                             --userns=keep-id \
```

```
                            --group-add=keep-groups \
                            --security-opt label=disable \
                            --pids-limit 0 \
                            --memory 200G \
                            --shm-size 64G \
                            -p 127.0.0.1:8000:8000 \
                          <container url>:<container tag>\
                               --model "${VLLM_MODEL_PATH}" \
                               -tp "${AIU_WORLD_SIZE}" \
                               --max-model-len "${MAX_MODEL_LEN}" \
                               --max-num-seqs ${MAX_BATCH_SIZE}
```

Example 7-17 shows the settings to run the reranker. This example uses one Spyre device. Change the environment variables to match your system environment.

Example 7-17 Example for running reranker

```
export AIU_IDS="0381:50:00.0"
#export AIU_WORLD_SIZE=1
#export HOST_MODELS_DIR=/models
#export MODEL_PATH=/models/bge-reranker-v2-m3
#export PROMPT_LENS=512
#export BATCH_SIZES=4
#podman run -d \
                     --device=/dev/vfio \
                      -v ${HOST_MODELS_DIR}:/models \
                      -e AIU_PCIE_IDS="${AIU_IDS}" \
                      -e VLLM_SPYRE_WARMUP_BATCH_SIZES="${BATCH_SIZES}" \
                      -e VLLM_SPYRE_WARMUP_PROMPT_LENS="${PROMPT_LENS}" \
                      --userns=keep-id \
                      --group-add=keep-groups \
                      --security-opt label=disable \
                      --pids-limit 0 \
                      --memory 200G \
                      --shm-size 64G \
                      -p 127.0.0.1:8000:8000 \
                       <container url>:<container tag> \
                       --model "${MODEL_PATH}" \
                        -tp "${AIU_WORLD_SIZE}"
```

Example 7-18 shows the setup for embedding workflows. Change the environment variables to match your system environment.

Example 7-18 Embeddings

```
#export AIU_IDS="0381:50:00.0"
#export AIU_WORLD_SIZE=1
#export HOST_MODELS_DIR=/models
#export MODEL_PATH=/models/granite-embedding-125m-english
#export PROMPT_LENS=512
#export BATCH_SIZES=4
#podman run -d \
                                --device=/dev/vfio \
                                -v ${HOST_MODELS_DIR}:/models \
                                -e AIU_PCIE_IDS="${AIU_IDS}" \
                                -e VLLM_SPYRE_WARMUP_BATCH_SIZES="${BATCH_SIZES}" \
```

```
    -e VLLM_SPYRE_WARMUP_PROMPT_LENS="${PROMPT_LENS}" \
    --userns=keep-id \
    --group-add=keep-groups \
    --security-opt label=disable \
    --pids-limit 0 \
    --memory 200G \
    --shm-size 64G \
    -p 127.0.0.1:8000:8000 \
<container url>:<container tag>  \
    --model "${MODEL_PATH}" \
    -tp "${AIU_WORLD_SIZE}"
```

7.4.7 Validating the server

To verify that the inference server is fully operational and that the vLLM runtime has initialized correctly, perform a series of functional and container-level checks. These checks include confirming container health and reviewing warm-up logs, followed by issuing simple completion and embeddings requests to validate model loading, request routing, and end-to-end inference behavior. The following steps provide a minimal but comprehensive validation workflow.

To verify the inference server and vLLM runtime, perform functional and container checks. Confirm the container health and review warm-up logs. Issue completion and embedding requests to check model loading, request routing, and inference behavior.

To validate the inference server, complete the following steps:

1. Check the container status and warm-up logs.
   a. List containers by running the following command:

      ```
      #podman ps
      ```

   b. Tail logs to watch vLLM warm-up and readiness by running the following command:

      ```
      #podman logs <container_id>
      ```

2. Test a simple completion call, as shown in Example 7-19.

Example 7-19 Simple completion test

```
#curl -X POST -H "Content-Type: application/json" \
      -d '{
              "model": "/models/granite-3.3-8b-instruct",
              "prompt": "What is the capital of France?",
              "max_tokens": 32
       }' \
       http://127.0.0.1:8000/v1/completions
```

3. Test an embeddings call, as shown in Example 7-20.

Example 7-20 Testing an embeddings call

```
#curl -X POST -H "Content-Type: application/json" \
       -d '{
               "model": "/models/granite-embedding-125m-english/",
               "input": ["What is the capital of Paris?"]
```

```
  }' \
http://127.0.0.1:8000/v1/embeddings
```

7.4.8 Troubleshooting and best practices

When deploying AI workloads with Podman on systems with the IBM Spyre Accelerator, configuration and runtime issues might occur. Common issues include registry authentication, device binding and VFIO access, model path and permission alignment, memory constraints, multi-card scaling parameters, and SELinux interactions.

The following notes describe these issues and corrective actions:

- Registry authentication fails.

  Ensure that the podman login `registry.redhat.io` uses a valid Red Hat account with entitlements to pull Red Hat AI Inference Server images.

- VFIO and device access errors.

  Confirm that the Spyre devices are bound to VFIO and visible to the container. To verify link speed and slot assignments, run the **`lspci -vv`** command. Run **`servicereport -v -p spyre, remediate`** with **`-r`** if necessary.

- Model not found or permission denied.

  Verify host directory permissions (`/models` or `/opt/ibm/spyre/models/src`) and group membership (`sentient`).

  Ensure that **`-v ${HOST_MODELS_DIR}:/models`** is present and that the paths inside the container match **`VLLM_MODEL_PATH`**.

- Insufficient memory or out of memory.

  Increase **`--memory`** and **`--shm-size`** according to the model size and batch targets.

  Reduce **`MAX_BATCH_SIZE`** or context length (**`MAX_MODEL_LEN`**) for stability.

- Multi-card scaling is not effective.

  Check **`AIU_PCIE_IDS`** enumerations and **`AIU_WORLD_SIZE`**.

  Confirm that firmware or driver versions are current and the adapters are in supported drawer slots.

- SELINUX.

  Verify that the SELinux settings align correctly with Podman's **`--privileged`** option.

7.5 Other tools

There are many tools and frameworks that support application development. The following list describes some of these projects:

- Code assistants
  - IBM offerings
    - IBM watsonx Code Assistant® for IBM Z
    - IBM i Project Bob
  - Cline
  - Roo Code
  - Kilo

  - Continue
  - GitHub Copilot
- Agent generators
  - Langchain
  - Cline
- MCP
  - FastMCP
  - PHP MCP
- A2A Agent to Agent Protocol
- Vector databases on Power architecture
  - MongoDB
  - Milvus
  - Streamlit

A demonstration of RAG on Power is documented in Boost the precision of your RAG workflow. The example loads PDF files into an AI workflow for querying the PDF archive.

7.6 Using the Wallaroo AI Starter Kit

The Wallaroo AI Starter Kit for IBM provides an AI API, AI model, AI run time on IBM Power, and inference server on Linux on Power platforms. The Starter Kit integrates with Power11 hardware, including the IBM Spyre Accelerator.

The Starter Kit is updated by Wallaroo. Administration is automated.

Abbreviations and acronyms

ADK	Agent Development Kit
AI	artificial intelligence
AIOps	AI for IT operations
AIU	AI Accelerator Unit
ARI	Alternative Routing-ID Interpretation
ASE	AI Solution Environment
BLAS	basic linear algebra and subprograms
bootc	bootable container
CI/CD	continuous integration and continuous deployment (CI/CD)
CMK	customer-managed key
CNN	convolutional neural network
CP4D	IBM Cloud Pak for Data
CVE	Common Vulnerabilities and Exposures
DCM	dual-chip module
DL	deep learning
DME	dense math engine
DORA	Digital Operational Resilience Act
DPDP	Digital Personal Data Protection
DPO	Data Protection Officer
ECM	Enterprise Content Management
ERP	enterprise resource planning
eSCM	entry single-chip module
EU	European Union
EVS	Equitus Video Sentinel
FMTI	Foundation Model Transparency Index
FPR	floating-point register
GCC	GNU Compiler Collection
GDPR	General Data Protection Regulation
GIL	Global Interpreter Lock
GPU	graphics processing unit
HA	high availability
HMC	Hardware Management Console
HPC	high-performance computing
HPDA	high-performance data analytics
IAM	identity and access management
IBM	International Business Machines Corporation
IoT	Internet of Things
ITIC	Information Technology Intelligence Consulting
ITSM	IT service management
JIT	just-in-time
KGNN	Knowledge Graph Neural Network
LFS	Large File Storage
LLM	large language model
LPAR	logical partition
LPM	Live Partition Mobility
MCP	Model Context Protocol
MEA	Middle East and Africa
MFA	multi-factor authentication
ML	machine learning
MLDR	Machine Learning Detection and Response
MLOps	machine learning operations
MMA	Matrix Math Accelerator
MTTR	Mean Time to Recovery
NCSC	National Cyber Security Centre
NLHPC	National Laboratory for High Performance Computing
NLP	natural language processing
OCI	Open Container Initiative
OMI	Open Memory Interface
OPA	Open Policy Agent
OpenCE	Open Cognitive Environment
PCIe	Peripheral Component Interconnect Express
PE	Processing Elements
PII	personally identifiable information
PIM	Power Inference Microservice
PowerVS	IBM Power Virtual Server
PT	Processing Tile
pyeco	Python Ecosystem
QPI	Quantitative Performance Index
QSC	quantum-safe cryptography
RAG	retrieval-augmented generation
RBAC	role-based access control
RegTech	regulatory technology
RHEL	Red Hat Enterprise Linux

© Copyright IBM Corp. 2026.

ROI	return on investment
SBOM	Software Bill of Materials
SCM	single-chip module
SFP	Special Function PE (SFP)
SGD	stochastic gradient descent
SIMD	single instruction, multiple data
SLM	small language model
SMT	simultaneous multithreading
SVM	Support Vector Machine
TEE	Trusted Execution Environment
TOPS	trillions of operations per second
TPU	Tensor Processing Unit
VFIO	Virtual Function I/O
VMX	Vector Multimedia eXtension
VPC	virtual private cloud
VSR	Vector Scalar Register
VSX	Vector Scalar eXtension
WSL	Watson Studio Learning
ZPD	zero planned downtime

Related publications

The publications that are listed in this section are considered suitable for a more detailed description of the topics that are covered in this book.

IBM Redbooks

The following IBM Redbooks publications provide more information about the topics in this document. Some publications that are referenced in this list might be available in softcopy only.

- *IBM Power11 E1150 Introduction and Technical Overview*, SG24-8589
- *IBM Power11 Scale-Out Servers: Introduction and Overview*, SG24-8590
- *IBM Power E1180: Introduction and Overview*, SG24-8587
- *IBM Power System AC922 Technical Overview and Introduction*, REDP-5494
- *Matrix-Multiply Assist Best Practices Guide*, REDP-5612

You can search for, view, download, or order these documents and other Redbooks, Redpapers, web docs, drafts, and additional materials, at the following website:

ibm.com/redbooks

Online resources

These websites are also relevant as further information sources:

- Dense Dynamic Blocks: Optimizing SpMM for Processors with Vector and Matrix Units Using Machine Learning Techniques

  https://iacoma.cs.uiuc.edu/iacoma-papers/ics22_2.pdf
- IBM Open-Source AI foundation for Power

  https://www.ibm.com/docs/en/aiservices
- IBM Spyre Enablement Stack for Power

  https://www.ibm.com/docs/en/ibm-spyre-for-power
- A matrix math facility for Power ISA processors

  https://arxiv.org/pdf/2104.03142
- Power ISA v3.1

  https://wiki.raptorcs.com/w/images/f/f5/PowerISA_public.v3.1.pdf

© Copyright IBM Corp. 2026.

Help from IBM

IBM Support and downloads

ibm.com/support

IBM Global Services

ibm.com/services

Redbooks

Implementing AI on Power11: Introducing the IBM Spyre Accelerator

(0.2”spine)
0.17”<->0.473”
90<->249 pages



SG24-8592-00

ISBN 0738462578

Printed in U.S.A.

Get connected



